

CÁTEDRA FINAL

Clasificación de enfermedades foliares del tomate mediante Deep Learning: comparación entre MLP y CNN sobre PlantVillage

Ítem	Detalle
Asignatura	Deep Learning
Ponderación	60 % de la nota final
Modalidad	Grupos del proyecto
Fecha de entrega	Sábado, 29 de agosto de 2026

Alumnos : Guido Ríos Ciaffaroni > **Correo** : guidoriosciaffaroni@gmail.com

Alumnos : Eduardo Opazo > **Correo** : edo.opazo.d@gmail.com

Fase 0 - Preparación y organización del proyecto

Contexto del problema y variable objetivo

1. Contexto del problema

La identificación oportuna de enfermedades en cultivos agrícolas constituye una tarea relevante para la sanidad vegetal, debido a que distintas patologías pueden afectar el desarrollo de las plantas, reducir el rendimiento de los cultivos y generar pérdidas productivas. En este contexto, el análisis automatizado de imágenes mediante técnicas de **Visión por Computador y Deep Learning** representa una alternativa para apoyar la clasificación de enfermedades a partir de características visuales presentes en las hojas.

Las enfermedades foliares pueden manifestarse mediante diferentes patrones visuales, entre ellos:

- Cambios de coloración.
- Aparición de manchas o lesiones.
- Alteraciones en la textura de la superficie foliar.
- Variaciones en la forma y distribución de las zonas afectadas.
- Presencia de patrones característicos asociados a determinadas enfermedades.

A diferencia de un conjunto de datos tabular, donde las características suelen encontrarse representadas explícitamente mediante variables numéricas o categóricas, una imagen contiene información distribuida espacialmente entre sus píxeles. Por esta razón, características como bordes, texturas, manchas, formas y patrones de lesión deben ser **aprendidas automáticamente por el modelo** a partir de los datos visuales.

En este proyecto se utiliza el conjunto de datos **PlantVillage**, restringiendo el análisis exclusivamente a imágenes correspondientes a hojas de **tomate (Tomato)**. Después del proceso de selección, verificación de integridad y eliminación de archivos duplicados exactos, el conjunto de datos utilizado contiene **18.146 imágenes únicas**, distribuidas en **10 clases** que representan diferentes enfermedades foliares y una categoría correspondiente a hojas saludables.

Las imágenes presentan características homogéneas de almacenamiento: se encuentran en formato **JPEG**, utilizan representación de color **RGB** y poseen una resolución original de **256 × 256 píxeles**. Estas características permiten establecer un pipeline uniforme de preprocesamiento antes del entrenamiento de los modelos.

La **variable objetivo** corresponde a la categoría sanitaria asignada a cada imagen, es decir, la enfermedad foliar o el estado saludable de la hoja de tomate. Matemáticamente, para cada imagen (X_i), se desea predecir una etiqueta:

$$[y_i \in c_1, c_2, \dots, c_{10}]$$

donde cada (c_k) representa una de las diez clases presentes en el conjunto de datos.

Por lo tanto, el problema corresponde a una tarea de **clasificación supervisada multiclase**, ya que cada imagen debe ser asignada a una única categoría entre varias clases posibles.

El propósito general del proyecto es desarrollar y comparar dos arquitecturas de Deep Learning estudiadas en la asignatura: una **Red Neuronal Multicapa (MLP)** y una **Red Neuronal Convolutiva (CNN)**. Ambas utilizarán las mismas imágenes como fuente de información, pero representarán hipótesis diferentes respecto de cómo aprovechar su estructura.

La MLP utilizará una representación vectorizada de la imagen, mientras que la CNN conservará y explotará explícitamente las relaciones espaciales entre los píxeles mediante operaciones convolucionales. De esta manera, el proyecto buscará determinar si la capacidad de una CNN para aprender patrones locales y jerarquías espaciales permite obtener una mejor capacidad de clasificación y generalización que una MLP aplicada sobre las mismas imágenes.

En consecuencia, la pregunta experimental central puede formularse como:

¿En qué medida una red neuronal convolutiva (CNN), al conservar y explotar la estructura espacial de las imágenes, mejora la clasificación y la capacidad de generalización de enfermedades foliares del tomate respecto de una red neuronal multicapa (MLP) que utiliza una representación vectorizada de las mismas imágenes?

Librerías

Reproducibilidad

Descargar o cargar el conjunto de datos mediante código reproducible.

Descarga del dataset PlantVillage desde Kaggle mediante curl

En esta etapa se descargará directamente el archivo comprimido del dataset **PlantVillage** utilizando curl y el endpoint de descarga proporcionado por Kaggle.

El dataset será almacenado en:

/content/plantvillage.zip

Esta estrategia permite mantener inicialmente el conjunto de datos comprimido, verificar su descarga e integridad y posteriormente descomprimirlo solamente cuando sea necesario para el análisis de imágenes.

Filtrado de clases del dataset PlantVillage

En esta etapa se realizará un **filtrado de las clases originales del dataset PlantVillage** con el objetivo de conservar únicamente las categorías correspondientes a hojas de **tomate (Tomato)**, que constituyen el objeto de estudio del proyecto.

La eliminación se aplicará tanto sobre el conjunto original train como sobre el conjunto val, de manera que ambos subconjuntos mantengan el mismo criterio de selección.

Se eliminarán todas las carpetas cuyo nombre comience con alguno de los siguientes prefijos:

- Apple
- Blueberry
- Cherry
- Corn
- Grape
- Orange
- Peach
- Pepper
- Potato
- Raspberry
- Soybean
- Squash
- Strawberry

De esta forma, después del filtrado se conservarán exclusivamente las clases asociadas a **tomate**, incluyendo distintas enfermedades foliares y la categoría correspondiente a hojas saludables.

Antes de realizar la eliminación definitiva, el código verificará y mostrará qué directorios cumplen los criterios establecidos. Esta comprobación previa permite validar la selección y reducir el riesgo de eliminar accidentalmente alguna clase que deba formar parte del experimento.

El resultado esperado de esta etapa es obtener un conjunto de datos compuesto únicamente por las **10 clases de tomate** que serán utilizadas posteriormente en las etapas de auditoría, preprocesamiento, entrenamiento y comparación de los modelos **MLP y CNN**.

Conteo de imágenes por clase

En esta etapa se contabilizará la cantidad de imágenes disponibles en cada directorio de clase para los conjuntos `train` y `val`.

El objetivo es verificar:

- cuántas clases permanecen en el dataset;
- cuántas imágenes contiene cada clase;
- si existe desbalance entre las categorías;
- el número total de imágenes por conjunto;
- la consistencia entre entrenamiento y validación.

Se considerarán como imágenes los archivos con extensiones:

`.jpg`, `.jpeg`, `.png`, `.bmp`, `.tif`, `.tiff` y `.webp`.

Consolidación de imágenes por clase

El dataset PlantVillage se encuentra actualmente dividido en dos conjuntos:

- `train`
- `val`

En esta etapa se eliminará dicha separación y se consolidarán todas las imágenes pertenecientes a una misma clase en un único directorio.

Por ejemplo:

`train/Potato___Early_blight`

y

`val/Potato___Early_blight`

serán fusionados en:

`PlantVillage/Potato___Early_blight`

El procedimiento contempla:

1. Identificar automáticamente todas las clases.
2. Contar las imágenes antes de la consolidación.
3. Crear un directorio único para cada clase.
4. Mover las imágenes de `train` y `val`.
5. Evitar sobrescrituras en caso de nombres de archivos duplicados.
6. Verificar que el número total de imágenes se conserve.
7. Eliminar las antiguas carpetas `train` y `val`.

Verificación de integridad de las imágenes

Antes de dividir el dataset en los conjuntos de entrenamiento, validación y prueba, se realizará una auditoría de integridad de todas las imágenes.

El objetivo es detectar archivos que puedan generar errores durante el entrenamiento de los modelos de Deep Learning.

Para cada imagen se verificará:

1. Que el archivo exista y tenga un tamaño mayor que cero.
2. Que pueda ser reconocido como una imagen válida.
3. Que la estructura interna del archivo no esté corrupta.
4. Que la imagen pueda ser decodificada completamente.
5. Que posea dimensiones válidas.
6. Que su formato sea compatible con el pipeline de procesamiento.
7. Que se pueda identificar correctamente la clase a la que pertenece.

La verificación se realizará sin eliminar ni modificar archivos. Los archivos problemáticos serán registrados para su posterior análisis.

Separación de imágenes duplicadas

Después de verificar la integridad del dataset, se realizará una detección de duplicados exactos utilizando el algoritmo criptográfico SHA-256.

Dos archivos serán considerados duplicados cuando posean exactamente el mismo contenido binario y, por tanto, generen el mismo hash SHA-256.

Para cada grupo de imágenes duplicadas:

1. Se conservará una imagen como copia principal dentro del dataset.
2. Las copias adicionales serán trasladadas a un directorio separado.
3. Se mantendrá información sobre la clase original de cada imagen.
4. No se eliminará ningún archivo.
5. Se generará un informe con los archivos conservados y trasladados.

El objetivo es evitar que una misma imagen pueda aparecer posteriormente en entrenamiento, validación y prueba, lo que produciría fuga de información (data leakage) y una evaluación artificialmente optimista del modelo.

Conteo de archivos originales y duplicados por clase

Después de separar las imágenes duplicadas del dataset principal, se realizará una comparación entre:

- las imágenes que permanecen en el dataset limpio;
- las imágenes duplicadas que fueron apartadas;
- el número total de imágenes que existía antes de la depuración.

El análisis se realizará por clase y permitirá determinar:

1. Número de imágenes originales conservadas.
2. Número de imágenes duplicadas apartadas.
3. Número total de imágenes antes de la depuración.
4. Porcentaje de duplicados por clase.
5. Número total de imágenes del dataset antes y después de la limpieza.

Esta información permite documentar cuantitativamente el proceso de depuración del conjunto de datos.

Fase 1 - Definición formal del problema

Esta fase establece qué problema se resolverá, cuál es la variable objetivo y por qué la solución tiene utilidad práctica

1.1 Contexto

Proyecto Final - Deep Learning | PlantVillage | MLP vs CNN - Importancia de identificar enfermedades vegetales. - Utilidad de automatizar la clasificación a partir de imágenes. - Características visuales que pueden diferenciar estados sanitarios. - Uso de imágenes de hojas como modalidad principal de datos.

1.1 Contexto

La identificación de enfermedades en plantas constituye un problema relevante dentro del ámbito agrícola, debido a que la aparición de patologías puede afectar el desarrollo de los cultivos, disminuir su rendimiento y generar pérdidas productivas. En particular, muchas enfermedades vegetales producen modificaciones visibles en las hojas, por lo que el análisis de imágenes puede utilizarse como una fuente de información para reconocer diferentes estados sanitarios.

En este proyecto se trabaja con imágenes de hojas de **tomate** provenientes del dataset **PlantVillage**, considerando distintas enfermedades foliares y una categoría correspondiente a hojas saludables. El objetivo es utilizar estas imágenes para construir modelos capaces de distinguir automáticamente entre diferentes clases sanitarias.

La automatización de este proceso mediante técnicas de **Deep Learning y Visión por Computador** permite analizar directamente las imágenes sin necesidad de definir manualmente todas las características visuales relevantes. En lugar de proporcionar al modelo variables previamente diseñadas, las redes neuronales pueden aprender representaciones a partir de los píxeles disponibles en las imágenes.

Las enfermedades foliares pueden manifestarse mediante diferentes características visuales, entre ellas:

- cambios de coloración;
- presencia de manchas o lesiones;
- variaciones en la textura de la hoja;
- alteraciones en los bordes;
- diferencias en la distribución espacial de las zonas afectadas;
- patrones visuales asociados a determinadas enfermedades.

Estas características no se encuentran representadas como variables independientes dentro del dataset, sino que están distribuidas espacialmente en las imágenes. Por esta razón, el problema requiere que el modelo aprenda automáticamente patrones visuales capaces de diferenciar una clase de otra.

De forma conceptual, el proceso de aprendizaje puede representarse como:

[Píxeles → bordes → texturas → formas → patrones foliares → clase sanitaria]

La modalidad principal de datos utilizada en este proyecto corresponde, por tanto, a **imágenes digitales de hojas de tomate**. Después de las etapas de filtrado, verificación de integridad y eliminación de duplicados exactos realizadas en el notebook, el conjunto de datos de trabajo contiene **18.146 imágenes únicas distribuidas en 10 clases**, correspondientes a distintas enfermedades y al estado saludable de la planta.

Este contexto proporciona una base adecuada para comparar dos estrategias de aprendizaje profundo: una **MLP**, que utilizará una representación vectorizada de las imágenes, y una **CNN**, que conservará y explotará explícitamente su estructura espacial. La comparación permitirá estudiar si el aprendizaje de patrones espaciales propios de una red convolucional proporciona una ventaja observable en la clasificación de enfermedades foliares del tomate.

1.2 Tipo de problema

Clasificación multiclase

- Cada imagen X_i deberá asociarse con una categoría y_i perteneciente al conjunto de K clases disponibles.

1.2 Tipo de problema

El problema abordado en este proyecto corresponde a una tarea de **clasificación supervisada multiclase**.

Se considera **aprendizaje supervisado** porque cada imagen del conjunto de datos posee previamente una etiqueta que identifica su correspondiente **estado sanitario o categoría foliar**. Durante el entrenamiento, el modelo utilizará pares de datos de la forma:

$$[(X_i, y_i)]$$

donde:

- (X_i) representa la imagen de una hoja de tomate.
- (y_i) representa la clase asociada a dicha imagen.

Después de las etapas de filtrado, consolidación, verificación de integridad y eliminación de duplicados desarrolladas en el notebook, el conjunto de datos definitivo contiene:

$$[N = 18,146]$$

imágenes únicas distribuidas en:

$$[K = 10]$$

clases.

La hoja de ruta establece precisamente que cada imagen (X_i) debe asociarse con una categoría (y_i) perteneciente al conjunto de (K) clases disponibles.

Por lo tanto, para cada imagen se debe determinar una única etiqueta perteneciente al conjunto:

$$[\mathcal{C} = c_1, c_2, \dots, c_{10}]$$

En los datos procesados en el notebook, las diez clases corresponden a:

1. Tomato__Bacterial_spot
2. Tomato__Early_blight
3. Tomato__Late_blight
4. Tomato__Leaf_Mold
5. Tomato__Septoria_leaf_spot
6. Tomato__Spider_mites Two-spotted_spider_mite
7. Tomato__Target_Spot
8. Tomato__Tomato_Yellow_Leaf_Curl_Virus
9. Tomato__Tomato_mosaic_virus
10. Tomato__healthy

De esta forma, el objetivo puede expresarse matemáticamente mediante una función de clasificación:

$$[f_\theta : X_i \rightarrow y_i]$$

donde (f_θ) representa el modelo parametrizado por (θ), encargado de aprender patrones visuales presentes en las imágenes para asignarlas a una de las diez categorías posibles.

La tarea es **multiclase** porque existen más de dos categorías posibles y cada imagen debe ser asignada a una única clase. Esto la diferencia de una clasificación binaria, donde solamente existen dos posibles resultados, y de un problema de regresión, donde la salida corresponde a un valor numérico continuo.

Conceptualmente, el proceso puede representarse como:

$$[\text{Imagen de hoja de tomate} \rightarrow \text{Modelo de clasificación} \rightarrow \text{una de 10 clases sanitarias}]$$

En las etapas posteriores del proyecto, tanto la **MLP** como la **CNN** resolverán exactamente la misma tarea de clasificación y utilizarán las mismas categorías de salida. Esto permitirá realizar posteriormente una comparación experimental bajo un criterio común, tal como exige el enunciado para la combinación **MLP + CNN** aplicada a imágenes.

En síntesis, el problema queda formalmente definido como:

$$[\text{Clasificación supervisada multiclase de imágenes de hojas de tomate en 10 categorías}]$$

1.3 Variable objetivo

La variable objetivo corresponde a la clase de enfermedad o estado sanitario de la hoja, inferida a partir del directorio de clase al que pertenece cada imagen.

1.3 Variable objetivo

La **variable objetivo** del proyecto corresponde a la **clase de enfermedad o estado sanitario de cada hoja de tomate**. En el dataset PlantVillage, esta información se encuentra determinada por el **directorio o carpeta de clase** al que pertenece cada imagen, tal como establece la hoja de ruta del proyecto.

Por lo tanto, para cada imagen (X_i) existe una etiqueta asociada (y_i), donde:

$[X_i = \text{imagen de una hoja de tomate}]$

y:

$[y_i = \text{clase sanitaria asociada a la imagen}]$

Después de las etapas de filtrado, consolidación, verificación de integridad y eliminación de archivos duplicados exactos realizadas en el notebook, el conjunto de datos utilizado contiene:

$[N = 18,146]$

imágenes únicas, distribuidas en:

$[K = 10]$

clases diferentes.

Las categorías que conforman la variable objetivo son:

1. Tomato__Bacterial_spot
2. Tomato__Early_blight
3. Tomato__Late_blight
4. Tomato__Leaf_Mold
5. Tomato__Septoria_leaf_spot
6. Tomato__Spider_mites Two-spotted_spider_mite
7. Tomato__Target_Spot
8. Tomato__Tomato_Yellow_Leaf_Curl_Virus
9. Tomato__Tomato_mosaic_virus
10. Tomato__healthy

De estas diez categorías, **nueve representan diferentes enfermedades o afecciones foliares**, mientras que una corresponde al estado **saludable (healthy)** de la hoja.

Matemáticamente, la variable objetivo puede representarse como:

$[y_i \in \mathcal{C}]$

donde:

$[\mathcal{C} = c_1, c_2, \dots, c_{10}]$

representa el conjunto de las diez clases posibles.

El objetivo de los modelos será, por tanto, aprender una función:

$[f_\theta(X_i) = \hat{y}_i]$

donde:

- (X_i) corresponde a la imagen de entrada.
- (f_θ) representa el modelo de Deep Learning parametrizado por (θ) .
- $(\hat{y}_i)$ corresponde a la clase predicha por el modelo.
- (y_i) corresponde a la clase real utilizada como referencia durante el entrenamiento y evaluación.

Conceptualmente:

[Imagen de hoja $\rightarrow$ MLP o CNN $\rightarrow$ Probabilidades para 10 clases $\rightarrow$ Clase predicha]

En la etapa de modelamiento, la capa de salida de ambas redes deberá contener **10 neuronas**, una por cada clase:

$[\mathbf{p} = [p_1, p_2, \dots, p_{10}]]$

donde cada (p_k) representa la probabilidad estimada de que la imagen pertenezca a la clase (k) . La predicción final corresponderá a la categoría con mayor probabilidad:

$$[\hat{y} = \arg \max_k p_k]$$

Esta definición permite que tanto la **MLP** como la **CNN** resuelvan exactamente la misma tarea predictiva y utilicen la misma variable objetivo, condición necesaria para realizar posteriormente una comparación experimental justa entre ambas arquitecturas.

En síntesis, la variable objetivo del proyecto queda definida como:

Clase de enfermedad o estado saludable de la hoja de tomate

con **10 categorías posibles** obtenidas directamente de la estructura de clases del dataset PlantVillage procesado en el notebook.

1.4 Pregunta central

- ¿La utilización de una CNN permite clasificar las enfermedades de las hojas con mayor capacidad de generalización que una MLP entrenada sobre una representación vectorizada de las mismas imágenes?

1.4 Pregunta central

La pregunta central del proyecto busca determinar si la **estructura arquitectónica de una Red Neuronal Convolutiva (CNN)** proporciona una ventaja real frente a una **Red Neuronal Multicapa (MLP)** cuando ambas son utilizadas para resolver exactamente el mismo problema de clasificación de enfermedades foliares del tomate.

La hoja de ruta establece como pregunta experimental:

¿La utilización de una CNN permite clasificar las enfermedades de las hojas con mayor capacidad de generalización que una MLP entrenada sobre una representación vectorizada de las mismas imágenes?

Esta pregunta es coherente con el propósito del proyecto, ya que la comparación **MLP vs CNN** debe representar dos hipótesis diferentes acerca de cómo utilizar la información contenida en una imagen. La hoja de ruta plantea explícitamente que la MLP trabaja sobre una imagen aplanada, mientras que la CNN conserva y explota su estructura espacial.

Considerando los datos obtenidos y procesados en el notebook, el experimento se desarrollará sobre un conjunto definitivo de:

$$[N = 18,146]$$

imágenes únicas de hojas de tomate, distribuidas en:

$$[K = 10]$$

clases correspondientes a diferentes enfermedades foliares y al estado saludable.

Las imágenes presentan una resolución original de:

$$[256 \times 256]$$

píxeles, utilizan tres canales de color **RGB** y han sido previamente verificadas en términos de integridad y duplicados. De esta forma, ambas arquitecturas podrán ser evaluadas sobre la misma fuente de información visual.

Hipótesis representada por la MLP La MLP utilizará una representación vectorizada de cada imagen. Conceptualmente:

$$[X_i \in \mathbb{R}^{H \times W \times C}]$$

será transformada mediante una operación de aplanamiento:

$$[\text{Flatten}(X_i) \in \mathbb{R}^{HWC}]$$

Por ejemplo, manteniendo la resolución original:

$$[256 \times 256 \times 3 = 196,608]$$

valores de entrada por imagen.

El principal efecto de esta transformación es que la estructura bidimensional de la imagen deja de encontrarse representada explícitamente. Para la MLP, los píxeles pasan a formar parte de un único vector y las relaciones de vecindad espacial deben ser aprendidas indirectamente a partir de conexiones densas.

Conceptualmente:

[Imagen $\rightarrow$ Flatten $\rightarrow$ MLP $\rightarrow$ Clase]

Hipótesis representada por la CNN La CNN, en cambio, recibe la imagen preservando sus dimensiones espaciales:

$$[X_i \in \mathbb{R}^{H \times W \times C}]$$

y utiliza filtros convolucionales para aprender patrones locales.

Conceptualmente, el aprendizaje puede representarse mediante:

[píxeles $\rightarrow$ bordes $\rightarrow$ texturas $\rightarrow$ formas $\rightarrow$ patrones de lesión $\rightarrow$ clase]

La hoja de ruta identifica precisamente como ventajas de la CNN la **detección de patrones locales, la compartición de pesos y el aprendizaje jerárquico de características**, mientras que señala que el aplanamiento realizado por una MLP elimina la representación espacial explícita.

Comparación experimental La pregunta central no pretende simplemente identificar cuál modelo obtiene una mayor exactitud, sino determinar si la incorporación explícita de estructura espacial produce una **mejora observable y generalizable**.

Por ello, el experimento deberá mantener constantes, en la medida de lo posible, las condiciones comunes de comparación:

[mismas imágenes]

[mismos subconjuntos de Train, Validation y Test]

[misma variable objetivo]

[mismas 10 clases]

[métricas de evaluación comunes]

El propio enunciado señala que una comparación MLP + CNN sobre imágenes debe contrastar una **imagen aplanada** frente a una arquitectura que **explora la estructura espacial**, y exige que ambas redes sean comparadas de forma significativa y bajo criterios adecuados.

Capacidad de generalización En este proyecto, la generalización se entenderá como la capacidad del modelo para clasificar correctamente **imágenes que no fueron utilizadas para ajustar sus parámetros**.

Por tanto, el rendimiento sobre entrenamiento no será suficiente para responder la pregunta central. Será necesario analizar posteriormente:

- desempeño sobre validación;
- desempeño final sobre test;
- diferencia entre entrenamiento y validación;
- presencia de sobreajuste;

- comportamiento por clase;
- matriz de confusión;
- errores de clasificación.

Las métricas recomendadas por el enunciado para tareas de clasificación incluyen **Accuracy**, **Balanced Accuracy**, **Precision**, **Recall** y **F1**, además de matrices de confusión y otras métricas cuando corresponda.

La comparación podrá expresarse posteriormente mediante:

$$[\Delta M = M_{\text{CNN}} - M_{\text{MLP}}]$$

donde (M) representa una métrica determinada. Por ejemplo:

$$[\Delta F1_{\text{macro}} = F1_{\text{CNN}} - F1_{\text{MLP}}]$$

Una diferencia positiva favorecería experimentalmente a la CNN para dicha métrica, pero la conclusión definitiva deberá considerar conjuntamente rendimiento, sobreajuste, errores y costo computacional.

Hipótesis experimental A partir de las propiedades de ambas arquitecturas se plantea como hipótesis de trabajo:

$$[H_1 : \text{La CNN presentará una mejor capacidad de generalización que la MLP}]$$

debido a que puede aprovechar explícitamente las relaciones espaciales existentes entre los píxeles y aprender patrones visuales locales y jerárquicos asociados a las enfermedades foliares.

Como hipótesis alternativa de ausencia de ventaja arquitectónica puede plantearse:

$$[H_0 : \text{La CNN no presentará una mejora relevante de generalización frente a la MLP}]$$

Estas hipótesis **no se consideran demostradas en esta fase**. La Fase 1 solamente establece la pregunta experimental. La respuesta deberá obtenerse posteriormente mediante entrenamiento, validación y evaluación final de ambas arquitecturas.

En consecuencia, la pregunta central queda operacionalizada como:

$$[¿\text{La preservación y explotación de la estructura espacial mediante una CNN produce una mejora observable?}]$$

Esta formulación permitirá que las etapas posteriores del proyecto respondan la pregunta mediante **evidencia experimental**, en lugar de asumir anticipadamente que una arquitectura es superior a la otra.

Fase 2 - Justificación del uso de Deep Learning

El proyecto exige demostrar que Deep Learning es una decisión metodológica fundamentada. La justificación debe construirse antes del entrenamiento y luego respaldarse con evidencia experimental.

2.1 ¿Existe suficiente cantidad y complejidad?

La evaluación debe considerar tanto el número total de imágenes como su distribución entre clases. Un volumen elevado no es suficiente si algunas categorías presentan muy pocos ejemplos.

$N_{\text{total}} = \text{suma de } N_i \text{ para } i = 1, \dots, K$

- Cantidad total de imágenes válidas y únicas después de la depuración. Número final de clases.
- Cantidad mínima, máxima y mediana de imágenes por clase.
- Porcentaje de participación de cada clase.
- Existencia de clases significativamente sobrerrepresentadas o subrepresentadas.

La complejidad también proviene de la alta dimensionalidad y de la estructura espacial de cada imagen: color, textura, manchas, bordes, lesiones y formas no aparecen como variables explícitas, sino que deben aprenderse a partir de los píxeles.

Píxeles -> bordes -> texturas -> formas -> patrones de lesión -> clase

2.1 ¿Existe suficiente cantidad y complejidad?

Uno de los requisitos centrales del proyecto consiste en justificar que el uso de **Deep Learning** responde a las características reales del problema y del conjunto de datos, y no solamente a una elección de modelamiento. El enunciado exige determinar si existe una cantidad y complejidad suficiente de datos para aprender patrones no triviales, mientras que la hoja de ruta establece que este análisis debe considerar tanto el volumen total como la distribución de observaciones entre las clases.

Cantidad total de imágenes Después del filtrado de PlantVillage se conservaron exclusivamente las categorías correspondientes a hojas de tomate. Antes de la eliminación de duplicados, el conjunto consolidado contenía:

$$[N_{\text{antes}} = 18,160]$$

imágenes.

La verificación de integridad realizada en el notebook determinó que las **18.160 imágenes eran válidas**, sin archivos problemáticos detectados, obteniéndose un porcentaje de integridad del:

$$[100]$$

Posteriormente, mediante comparación de hashes **SHA-256**, se identificaron y apartaron **14 copias redundantes exactas**, equivalentes aproximadamente al:

$$[\frac{14}{18,160} \times 100 \approx 0.08]$$

del conjunto original.

Por tanto, el dataset definitivo utilizado para continuar el experimento contiene:

$$[N_{\text{total}} = 18,146]$$

imágenes válidas y únicas.

De acuerdo con la formulación propuesta en la hoja de ruta:

$$[N_{\text{total}} = \sum_{i=1}^K N_i]$$

donde (N_i) corresponde al número de imágenes pertenecientes a la clase (i) y:

$[K = 10]$ representa el número final de clases.

En este caso se verifica:

$$[18,146 = \sum_{i=1}^{10} N_i]$$

Distribución de imágenes entre clases Después de la depuración, las imágenes se encuentran distribuidas de la siguiente manera:

Clase	Imágenes	Participación
Tomato__Bacterial_spot	2.127	11,72 %
Tomato__Early_blight	1.000	5,51 %
Tomato__Late_blight	1.901	10,48 %
Tomato__Leaf_Mold	952	5,25 %
Tomato__Septoria_leaf_spot	1.771	9,76 %

Clase	Imágenes	Participación
Tomato__Spider_mites Two-spotted_spider_mite	1.676	9,24 %
Tomato__Target_Spot	1.404	7,74 %
Tomato__Tomato_Yellow_Leaf_Curl_Virus	5.357	29,52 %
Tomato__Tomato_mosaic_virus	373	2,06 %
Tomato__healthy	1.585	8,73 %
Total	18.146	100 %

La cantidad mínima de imágenes por clase corresponde a:

$$[N_{\min} = 373]$$

para:

Tomato__Tomato_mosaic_virus

mientras que la cantidad máxima corresponde a:

$$[N_{\max} = 5,357]$$

para:

Tomato__Tomato_Yellow_Leaf_Curl_Virus.

Al ordenar las cantidades de imágenes:

[373, ; 952, ; 1000, ; 1404, ; 1585, ; 1676, ; 1771, ; 1901, ; 2127, ; 5357]

la mediana corresponde a:

$$[\text{Mediana}(N_i) = \frac{1585+1676}{2} \boxed{1630.5}]$$

imágenes por clase.

Análisis del desbalance Aunque el dataset posee un volumen total considerable, su distribución no es uniforme.

La clase Tomato__Tomato_Yellow_Leaf_Curl_Virus representa aproximadamente:

$$\boxed{29.52\%}$$

del conjunto completo, mientras que Tomato__Tomato_mosaic_virus representa solamente:

$$\boxed{2.06\%}$$

Una forma de cuantificar esta diferencia es mediante la razón de desbalance:

$$[IR = \frac{N_{\max}}{N_{\min}}]$$

por lo que:

$$[IR = \frac{5357}{373} \approx \boxed{14.36}]$$

Esto significa que la clase mayoritaria contiene aproximadamente **14,4 veces más imágenes que la clase minoritaria.**

Por lo tanto, se identifica una **sobrerrepresentación importante** de Tomato__Tomato_Yellow_Leaf_Curl_Virus y una **subrepresentación relativa** de Tomato__Tomato_mosaic_virus.

Este resultado no impide utilizar Deep Learning, pero deberá ser considerado en las fases posteriores, especialmente durante:

- la división estratificada de train, validation y test;

- el entrenamiento de los modelos;
- la interpretación de las métricas;
- el análisis de errores por clase.

En consecuencia, posteriormente no será conveniente evaluar los modelos exclusivamente mediante accuracy, ya que una métrica global puede ocultar un rendimiento deficiente sobre las clases minoritarias. La hoja de ruta contempla precisamente métricas como **Balanced Accuracy, Precision, Recall y F1**, además de la matriz de confusión, para analizar el comportamiento de los modelos ante esta situación.

Complejidad de los datos

La justificación de Deep Learning no depende únicamente de disponer de 18.146 ejemplos. También es necesario analizar la **complejidad de cada observación**.

Las imágenes procesadas en el notebook presentan características homogéneas:

$[256 \times 256]$

píxeles de resolución y tres canales de color:

$[RGB]$

Por tanto, cada imagen puede representarse inicialmente como un tensor:

$[X_i \in \mathbb{R}^{256 \times 256 \times 3}]$

lo que supone:

$[256 \times 256 \times 3 = \boxed{196,608}]$

valores de intensidad por imagen.

Esta alta dimensionalidad diferencia claramente el problema de un conjunto tabular compuesto por unas pocas variables predictoras.

Sin embargo, la complejidad más importante no procede solamente del número de píxeles. La información necesaria para identificar una enfermedad se encuentra **distribuida espacialmente dentro de la imagen**.

Variables como:

- coloración;
- textura;
- manchas;
- lesiones;
- bordes;
- formas;
- tamaño y distribución de las zonas afectadas;

no se encuentran disponibles como columnas explícitas del dataset.

El modelo recibe fundamentalmente los valores de los píxeles y debe aprender qué combinaciones de ellos resultan útiles para distinguir los diferentes estados sanitarios.

Conceptualmente, el aprendizaje esperado puede representarse mediante:

$\boxed{\text{Píxeles} \rightarrow \text{bordes} \rightarrow \text{texturas} \rightarrow \text{formas} \rightarrow \text{patrones de lesión} \rightarrow \text{clase}}$

Esta característica es precisamente uno de los argumentos centrales establecidos en la hoja de ruta para justificar Deep Learning: color, textura, manchas, bordes, lesiones y formas **no aparecen como variables explícitas**, sino que deben aprenderse a partir de la representación visual.

Cantidad y complejidad consideradas conjuntamente

La evaluación del dataset muestra entonces dos características simultáneas.

Por una parte, existe un volumen de:

[18, 146 imágenes únicas]

distribuidas entre:

[10 clases]

lo que proporciona una base de datos suficientemente amplia para plantear un experimento acotado de Deep Learning en Google Colab.

Por otra parte, cada observación posee una estructura:

[$256 \times 256 \times 3$]

y contiene relaciones espaciales que no pueden reducirse directamente a unas pocas variables previamente definidas.

Por tanto, la complejidad del problema puede resumirse como:

[volumen de datos + alta dimensionalidad + estructura espacial + patrones visuales no explícitos + clasificación]

El propio enunciado considera apropiada la combinación **MLP + CNN para imágenes**, específicamente para contrastar una representación aplanada con una arquitectura que explota la estructura espacial.

Conclusión de la sección 2.1

A partir de los resultados obtenidos en el notebook, se considera que **sí existe una cantidad y complejidad suficientes para justificar el desarrollo del experimento mediante Deep Learning**.

La afirmación se fundamenta en que:

[$N_{\text{total}} = 18,146$ $K = 10$ $N_{\text{min}} = 373$ $N_{\text{max}} = 5,357$ $\text{Mediana}(N_i) = 1,630.5$ $IR \approx 14.36$]

Además, cada imagen contiene **196.608 valores de entrada en su resolución original**, organizados espacialmente en tres canales RGB, y las características relevantes para la clasificación deben ser aprendidas a partir de los píxeles.

No obstante, el análisis también identifica una limitación que deberá mantenerse presente durante el proyecto: **la distribución entre clases es desbalanceada**. Esta situación no invalida el dataset, pero exige una partición estratificada y métricas que permitan evaluar adecuadamente el rendimiento sobre las categorías minoritarias.

En consecuencia:

[Sí existe suficiente cantidad y complejidad para estudiar Deep Learning,]

pero la verdadera ventaja de las arquitecturas profundas —y particularmente de la CNN— deberá ser **demostrada posteriormente mediante evidencia experimental**, comparando su capacidad de generalización con la MLP y no asumiéndola de antemano.

2.2 ¿Por qué las imágenes justifican Deep Learning?

Una imagen conserva relaciones espaciales entre píxeles. Las CNN están diseñadas para explotar esa estructura mediante filtros convolucionales y aprendizaje jerárquico de representaciones.

2.2 ¿Por qué las imágenes justifican Deep Learning?

La utilización de **Deep Learning** se encuentra especialmente justificada cuando los datos de entrada poseen una estructura compleja cuya información relevante no está disponible como un conjunto reducido de variables previamente definidas. Este es precisamente el caso del proyecto, ya que la modalidad principal de los datos corresponde a **imágenes digitales de hojas de tomate**.

El enunciado del proyecto establece que una de las preguntas centrales para justificar Deep Learning es determinar si la modalidad de los datos favorece el **aprendizaje automático de representaciones**. Asimismo, propone explícitamente la combinación **MLP + CNN** para problemas

basados en imágenes, comparando una representación aplanada con una arquitectura que explota su estructura espacial.

Después de las etapas de filtrado, verificación de integridad y eliminación de duplicados realizadas en el notebook, el conjunto de datos de trabajo contiene:

[18, 146 imágenes únicas]

distribuidas en:

[10 clases]

correspondientes a diferentes enfermedades foliares del tomate y a la categoría de hojas saludables.

Las imágenes presentan una resolución original de:

[256×256]

píxeles y tres canales de color **RGB**, por lo que cada observación puede representarse como un tensor:

[$X_i \in \mathbb{R}^{256 \times 256 \times 3}$]

Esto significa que una sola imagen contiene:

[$256 \times 256 \times 3 = 196,608$]

valores de intensidad.

Sin embargo, la principal razón para utilizar Deep Learning **no es solamente la elevada cantidad de valores por imagen**, sino la forma en que estos valores se encuentran organizados.

La información de una imagen posee estructura espacial En una imagen, la posición de cada píxel importa. Un píxel no constituye una observación independiente: su significado visual depende de los píxeles que se encuentran a su alrededor.

Formalmente, una imagen RGB puede representarse como:

[$X(i, j, c)$]

donde:

- (i) representa la posición vertical;
- (j) representa la posición horizontal;
- (c) representa el canal de color.

Por lo tanto, valores próximos como:

[$X(i, j, c)$]

y:

[$X(i + 1, j, c)$]

mantienen una relación de proximidad espacial.

Esta relación es fundamental para reconocer estructuras visuales como:

- bordes;
- manchas;
- cambios de coloración;
- texturas;
- lesiones;
- contornos;
- formas;
- distribución espacial de zonas afectadas.

En consecuencia:

[la posición relativa de los píxeles contiene información]

y esa información debe ser considerada por el modelo.

Las características relevantes no están definidas explícitamente A diferencia de un problema tabular, el dataset no proporciona columnas como:

```
intensidad_de_manchas  
color_de_lesion  
textura_foliar  
forma_de_la_lesion  
porcentaje_de_superficie_afectada
```

Estas variables **no existen directamente en los datos**.

El modelo recibe esencialmente una matriz de píxeles y debe descubrir qué características resultan útiles para diferenciar las diez clases.

Por ejemplo, una determinada enfermedad puede caracterizarse por una combinación de:

[color + textura + forma + localización]

y no por una única característica aislada.

Por tanto, el problema requiere un mecanismo de **aprendizaje de representaciones**, es decir, que el modelo construya internamente características cada vez más informativas a partir de los datos originales.

Este principio coincide directamente con la hoja de ruta, que establece que características como color, textura, manchas, bordes, lesiones y formas no aparecen como variables explícitas, sino que deben ser aprendidas a partir de los píxeles.

Aprendizaje jerárquico de representaciones

Una de las principales capacidades del Deep Learning es aprender características de manera jerárquica.

Conceptualmente, el proceso puede representarse como:

[Píxeles → Bordes → Texturas → Formas → Patrones de lesión → Clase]

Las primeras capas de una red pueden aprender patrones visuales relativamente simples, mientras que capas posteriores combinan estas representaciones para detectar estructuras progresivamente más complejas.

Por ejemplo:

Nivel 1 — Información básica [píxeles → cambios de intensidad y color]

Nivel 2 — Patrones locales [cambios locales → bordes y pequeñas texturas]

Nivel 3 — Estructuras visuales [bordes + texturas → manchas o lesiones]

Nivel 4 — Patrones de mayor nivel [lesiones + formas + distribución → patrón foliar]

Nivel 5 — Clasificación [patrón foliar → enfermedad o estado saludable]

Esta capacidad de aprender automáticamente una jerarquía de características constituye una de las razones principales por las que Deep Learning resulta apropiado para el problema.

La hoja de ruta establece precisamente que las CNN están diseñadas para aprovechar las relaciones espaciales mediante **filtros convolucionales y aprendizaje jerárquico de representaciones**.

¿Cómo aprovecha una CNN la estructura espacial?

Una **Convolutional Neural Network (CNN)** utiliza filtros o kernels que recorren regiones locales de la imagen.

De forma simplificada, una operación convolucional puede representarse como:

$$[Y(i, j) = \sum_m \sum_n X(i + m, j + n), K(m, n)]$$

donde:

- (X) representa la imagen de entrada;
- (K) corresponde al filtro convolucional;
- (Y) es el mapa de características obtenido.

El filtro analiza pequeños vecindarios de píxeles en lugar de tratar todos los valores de la imagen como elementos independientes.

Por ejemplo, un kernel de:

$$[3 \times 3]$$

analiza grupos locales de nueve posiciones espaciales a la vez.

Conceptualmente:

$$[[x_1 \ x_2 \ x_3 \ x_4 \ x_5 \ x_6 \ x_7 \ x_8 \ x_9]]$$

Esto permite detectar estructuras locales que posteriormente pueden combinarse para formar patrones de mayor complejidad.

Relación con las imágenes del proyecto

Las enfermedades incluidas en el dataset se manifiestan mediante información visual presente directamente en las hojas.

Por ello, el modelo deberá aprender relaciones como:

$$[\text{coloración local} + \text{textura} + \text{forma de manchas} + \text{distribución espacial} \rightarrow \text{clase sanitaria}]$$

El hecho de que todas las imágenes procesadas sean homogéneas en términos de resolución, canales de color y formato facilita la construcción posterior de un pipeline común de procesamiento, pero **no transforma el problema en una tarea sencilla**.

Aunque todas posean dimensiones:

$$[256 \times 256 \times 3]$$

el contenido espacial varía entre imágenes y entre clases. La dificultad consiste precisamente en aprender qué configuraciones visuales son discriminativas para cada enfermedad.

Deep Learning frente a extracción manual de características

Una estrategia convencional podría intentar construir manualmente variables como:

$$[\text{color promedio}, \text{contraste}, \text{histograma RGB}, \text{textura}]$$

y utilizar posteriormente estas características en un clasificador tradicional.

El problema es que este enfoque requiere decidir previamente:

¿Qué características visuales son realmente importantes para distinguir las enfermedades?

Deep Learning modifica este procedimiento:

$$[\text{Imagen} \rightarrow \text{aprendizaje automático de características} \rightarrow \text{clasificación}]$$

En lugar de:

$$[\text{Imagen} \rightarrow \text{características diseñadas manualmente} \rightarrow \text{clasificador}]$$

Esta capacidad resulta especialmente pertinente para PlantVillage, ya que las diferencias entre categorías se encuentran codificadas en patrones visuales que pueden involucrar múltiples niveles de información.

Justificación específica para el experimento MLP vs CNN

La modalidad de los datos permite además construir una comparación metodológicamente significativa entre las dos arquitecturas seleccionadas.

Ambas redes recibirán información procedente de las mismas imágenes, pero la representación utilizada será diferente:

[Imagen $\rightarrow$ {MLP: representación vectorizada CNN: representación espacial}]

La CNN está diseñada específicamente para explotar:

[localidad espacial + filtros convolucionales + aprendizaje jerárquico]

mientras que el comportamiento de la MLP permitirá establecer una referencia experimental respecto de una representación que no explota explícitamente esa geometría.

Precisamente, el enunciado considera razonable para imágenes la comparación entre una **MLP con la imagen aplanada** y una **CNN que explota su estructura espacial**.

Conclusión de la sección 2.2

La modalidad de los datos proporciona una justificación técnica clara para utilizar Deep Learning.

El dataset no está compuesto por unas pocas variables independientes, sino por **18.146 imágenes**, cada una con una estructura original de:

[$256 \times 256 \times 3$]

y, por tanto:

[196, 608]

valores organizados espacialmente.

La información necesaria para diferenciar las diez categorías se encuentra distribuida entre los píxeles y se manifiesta mediante combinaciones de color, bordes, texturas, manchas, lesiones y formas que no están disponibles como variables explícitas.

Por ello:

[Imagen $\rightarrow$ representaciones visuales aprendidas $\rightarrow$ clasificación]

constituye una formulación natural del problema.

En consecuencia:

[las imágenes justifican el uso de Deep Learning porque contienen relaciones espaciales y patrones visuales]

La CNN constituye una arquitectura especialmente adecuada para estudiar esta hipótesis debido a su capacidad para explotar la **localidad espacial y el aprendizaje jerárquico de representaciones**. No obstante, que esta ventaja teórica produzca efectivamente una mayor capacidad de generalización sobre PlantVillage deberá demostrarse posteriormente mediante la comparación experimental con la MLP.

2.3 Limitación de una MLP

La MLP requiere aplanar la imagen. Al convertir una matriz $H \times W \times C$ en un vector, la organización espacial deja de estar representada explícitamente.

2.3 Limitación de una MLP

Una **Red Neuronal Multicapa (MLP, Multilayer Perceptron)** puede utilizarse para resolver problemas de clasificación de imágenes. Sin embargo, presenta una limitación estructural im-

portante: antes de introducir una imagen en capas densas, normalmente es necesario transformar su representación tridimensional en un **vector unidimensional** mediante una operación de aplanamiento (Flatten).

Esta transformación conserva los valores numéricos de los píxeles, pero hace que la estructura espacial original de la imagen deje de estar representada explícitamente en la arquitectura de la red.

Representación original de las imágenes

De acuerdo con los resultados obtenidos durante la preparación y auditoría del dataset, las imágenes utilizadas en el proyecto poseen una resolución original de:

$$256 \times 256$$

píxeles y tres canales de color:

$$RGB$$

Por lo tanto, cada imagen puede representarse matemáticamente como un tensor:

$$X_i \in \mathbb{R}^{256 \times 256 \times 3}$$

donde:

- X_i representa la imagen i .
- 256×256 corresponde a sus dimensiones espaciales.
- 3 corresponde a los canales de color rojo, verde y azul.

La cantidad total de valores presentes en una imagen es:

$$256 \times 256 \times 3 = 196,608$$

Por tanto, cada imagen original contiene:

196,608 valores

Estos valores no se encuentran organizados arbitrariamente. Cada píxel posee una posición específica dentro de la imagen y mantiene relaciones espaciales con los píxeles cercanos.

Aplanamiento de la imagen

Para utilizar directamente una imagen como entrada de una MLP, se aplica normalmente una operación de aplanamiento:

$$X_i \in \mathbb{R}^{H \times W \times C}$$

que transforma la imagen en:

$$\text{Flatten}(X_i) \in \mathbb{R}^{HWC}$$

En el caso particular de las imágenes originales del proyecto:

$$\mathbb{R}^{256 \times 256 \times 3} \longrightarrow \mathbb{R}^{196,608}$$

Conceptualmente, el proceso puede representarse como:

$$\text{Imagen} \rightarrow \text{Flatten} \rightarrow \text{Vector} \rightarrow \text{MLP} \rightarrow \text{Clase}$$

La estructura típica de la MLP podría expresarse como:

$$\text{Imagen} \rightarrow \text{Flatten} \rightarrow \text{Dense} + \text{ReLU} \rightarrow \text{Dropout} \rightarrow \text{Dense} + \text{ReLU} \rightarrow \text{Softmax}$$

Pérdida de representación espacial explícita

La principal limitación del aplanamiento no consiste en eliminar los valores de los píxeles. Todos ellos permanecen en el vector resultante.

La dificultad es que las relaciones espaciales entre los píxeles dejan de estar representadas explícitamente.

Antes del aplanamiento, un píxel puede expresarse como:

$$X(i, j, c)$$

donde:

- i representa la posición vertical.
- j representa la posición horizontal.
- c representa el canal de color.

Después del aplanamiento, la imagen pasa a representarse como:

$$\mathbf{x} * i = [x_1, x_2, x_3, \dots, x * 196608]$$

La MLP recibe entonces un vector de valores y sus capas densas no incorporan de manera explícita información acerca de:

- qué píxeles se encuentran próximos;
- qué píxeles pertenecen a una misma región;
- dónde aparecen las manchas;
- cómo se distribuyen las lesiones;
- cómo se organizan las texturas;
- qué relación espacial existe entre distintas partes de la hoja.

Por ejemplo, en una pequeña región de una imagen:

$$\begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{2,1} & x_{2,2} & x_{2,3} & x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix}$$

el píxel $x_{2,2}$ posee una relación espacial directa con los píxeles que lo rodean.

Al aplanar esa matriz se obtiene una representación similar a:

$$[x_{1,1}, x_{1,2}, x_{1,3}, x_{2,1}, x_{2,2}, \dots]$$

En esta nueva representación, la geometría bidimensional original ya no forma parte explícita de la arquitectura del modelo.

Implicancia para la clasificación de enfermedades foliares

Esta limitación resulta especialmente importante en este proyecto, porque las enfermedades de las hojas de tomate pueden manifestarse mediante patrones como:

- manchas;
- lesiones;
- cambios de coloración;
- bordes irregulares;
- texturas;
- formas;
- distribución espacial de zonas afectadas.

En consecuencia, no solo interesa conocer el valor de determinados píxeles, sino también:

dónde aparecen y cómo se organizan espacialmente

Una determinada lesión podría caracterizarse mediante una combinación de:

Color + Forma + Textura + Distribución espacial

Al utilizar una MLP, estas relaciones deben ser aprendidas indirectamente mediante conexiones densas, ya que la arquitectura no incorpora de manera natural el concepto de vecindad espacial.

Elevado número de parámetros

Una segunda limitación de la MLP aplicada a imágenes es el rápido crecimiento del número de parámetros.

En una capa densa, cada neurona se conecta con todas las entradas.

Si se mantuviera la resolución original de las imágenes:

$$256 \times 256 \times 3$$

la entrada contendría:

$$196,608$$

valores.

Si la primera capa densa tuviera 256 neuronas, el número de pesos sería:

$$196,608 \times 256 = 50,331,648$$

Además, cada neurona tendría un término de sesgo, por lo que el número total de parámetros de esa primera capa sería:

$$50,331,648 + 256 = 50,331,904$$

Por lo tanto:

50,331,904 parámetros

serían necesarios únicamente para conectar la entrada con una primera capa de 256 neuronas.

Este ejemplo muestra que una MLP aplicada directamente sobre imágenes de alta resolución puede generar una arquitectura excesivamente grande.

Reducción de la resolución

Para mantener un experimento computacionalmente manejable en Google Colab, posteriormente será conveniente reducir la resolución de entrada.

Por ejemplo, utilizando imágenes de:

$$64 \times 64 \times 3$$

la cantidad de valores de entrada sería:

$$64 \times 64 \times 3 = 12,288$$

Después del aplanamiento:

$$\mathbb{R}^{64 \times 64 \times 3} \longrightarrow \mathbb{R}^{12,288}$$

Si nuevamente se utilizara una primera capa de 256 neuronas:

$$12,288 \times 256 = 3,145,728$$

pesos.

Incluyendo los sesgos:

$$3,145,728 + 256 = 3,145,984$$

parámetros.

Esto representa una reducción considerable respecto de la resolución original:

$$50,331,904 \longrightarrow 3,145,984$$

aunque continúa siendo una arquitectura con una cantidad significativa de conexiones.

Funcionamiento matemático de una capa densa

La MLP puede aprender relaciones no lineales complejas mediante transformaciones sucesivas de la forma:

$$\mathbf{h} = \sigma(W\mathbf{x} + b)$$

donde:

- $\mathbf{x}$ corresponde al vector de entrada.
- W representa la matriz de pesos.
- b representa el vector de sesgos.
- σ representa una función de activación.
- $\mathbf{h}$ representa la salida de la capa.

Si se utiliza la función **ReLU**, esta se define como:

$$\text{ReLU}(z) = \max(0, z)$$

Por tanto, una MLP sí posee capacidad para aprender relaciones complejas a partir de los píxeles.

La limitación no consiste en que sea incapaz de clasificar imágenes, sino en que su arquitectura **no incorpora un mecanismo específico para explotar la estructura espacial de estas**.

En otras palabras:

MLP limitada para explotar estructura espacial $\neq$ MLP incapaz de clasificar imágenes

Consecuencias potenciales para el aprendizaje

En el contexto del dataset utilizado, compuesto por:

$$N = 18,146$$

imágenes únicas distribuidas en:

$$K = 10$$

clases, el uso de una MLP presenta varias posibles limitaciones.

1. Estructura espacial no explícita El proceso:

$$H \times W \times C \rightarrow HWC$$

elimina la representación matricial explícita de la imagen.

2. Gran cantidad de parámetros Las capas densas requieren conexiones entre todas las entradas y todas las neuronas:

$$N_{\text{pesos}} = N_{\text{entrada}} \times N_{\text{neuronas}}$$

Esto puede aumentar considerablemente la complejidad del modelo.

3. Menor eficiencia para detectar patrones locales Una característica visual similar que aparezca en diferentes posiciones de la hoja no es procesada automáticamente mediante el mismo conjunto reducido de pesos.

4. Mayor riesgo de sobreajuste Cuando aumenta excesivamente el número de parámetros, el modelo puede desarrollar una gran capacidad para ajustarse al conjunto de entrenamiento:

Mayor capacidad $\rightarrow$ posible memorización $\rightarrow$ riesgo de sobreajuste

Este fenómeno deberá evaluarse posteriormente mediante las curvas de entrenamiento y validación.

5. Arquitectura no especializada en imágenes La MLP no incorpora explícitamente propiedades como:

Localidad espacial

Vecindad entre píxeles

Compartición de pesos

Estas características sí forman parte del diseño de las redes convolucionales.

Papel de la MLP dentro del experimento

La MLP constituye una parte fundamental del experimento porque permite analizar cuánto puede aprenderse utilizando los mismos valores de los píxeles, pero sin explotar explícitamente su estructura espacial.

La hipótesis representada por la MLP puede expresarse como:

Imagen $\rightarrow$ Vectorización $\rightarrow$ Aprendizaje mediante capas densas

La CNN representará una hipótesis diferente:

Imagen $\rightarrow$ Preservación espacial $\rightarrow$ Aprendizaje mediante convoluciones

Por tanto, el núcleo experimental será comparar:

MLP con imagen aplanada vs. CNN con estructura espacial

Conclusión de la sección 2.3

La principal limitación de una MLP aplicada a las imágenes de PlantVillage es la necesidad de transformar cada imagen desde:

$$X_i \in \mathbb{R}^{H \times W \times C}$$

hacia:

$$\mathbf{x}_i \in \mathbb{R}^{HWC}$$

En las imágenes originales del proyecto:

$$256 \times 256 \times 3 \rightarrow 196,608$$

Aunque los valores de los píxeles se conservan, la organización espacial deja de estar representada explícitamente en la arquitectura.

El proceso puede resumirse como:

Imagen $\rightarrow$ Flatten $\rightarrow$ Vector $\rightarrow$ Capas densas

Como consecuencia, la MLP debe aprender indirectamente las relaciones necesarias para identificar manchas, texturas, bordes, lesiones y formas.

Además, el uso de capas completamente conectadas puede incrementar rápidamente el número de parámetros, aumentando el costo computacional y potencialmente el riesgo de sobreajuste.

Estas limitaciones proporcionan el fundamento técnico para comparar posteriormente la MLP con una CNN. Sin embargo, la superioridad de la CNN **no debe asumirse previamente**; deberá comprobarse mediante los resultados de entrenamiento, validación y evaluación final sobre el conjunto de prueba.

2.4 Ventaja específica de una CNN

- Detección de patrones locales.
- Compartición de pesos.
- Aprendizaje jerárquico de características.
- Mayor eficiencia para explotar estructura espacial que una capa densa equivalente.

2.4 Ventaja específica de una CNN

Una **Red Neuronal Convolutiva (CNN, Convolutional Neural Network)** constituye una arquitectura especialmente adecuada para el procesamiento de imágenes porque está diseñada para aprovechar explícitamente la **estructura espacial** presente en los datos.

A diferencia de una MLP, que transforma la imagen en un vector mediante Flatten, una CNN mantiene durante sus primeras etapas la organización bidimensional de los píxeles y aprende características mediante operaciones de convolución.

En el conjunto de datos procesado para este proyecto se dispone de:

$$N = 18,146$$

imágenes únicas, distribuidas en:

$$K = 10$$

clases de hojas de tomate.

Las imágenes originales poseen dimensiones:

$$256 \times 256 \times 3$$

donde los tres canales corresponden a la representación de color **RGB**.

Por tanto, cada imagen puede expresarse como:

$$X_i \in \mathbb{R}^{256 \times 256 \times 3}$$

La CNN puede procesar esta representación manteniendo explícitamente las relaciones espaciales entre píxeles.

1. Detección de patrones locales

Una de las principales ventajas de una CNN es que sus filtros convolucionales analizan **regiones locales de la imagen**.

En lugar de conectar cada neurona con todos los píxeles, como ocurre en una capa densa, una convolución utiliza una pequeña ventana o **kernel**, por ejemplo:

$$3 \times 3$$

que se desplaza sobre la imagen.

De manera simplificada, una convolución puede representarse como:

$$Y(i, j) = \sum_u \sum_v X(i + u, j + v), W(u, v) + b$$

donde:

- X corresponde a la imagen o mapa de características de entrada.
- W corresponde al kernel o filtro convolucional.
- b corresponde al término de sesgo.
- $Y(i, j)$ representa la activación obtenida en la posición (i, j) .

Para una imagen con varios canales, la operación puede expresarse de manera más general como:

$$Y_k(i, j) = b_k + \sum_{c=1}^{C_{in}} \sum_{u=1}^{K_h} \sum_{v=1}^{K_w} W_k(u, v, c), X(i + u, j + v, c)$$

donde:

- C_{in} corresponde al número de canales de entrada.
- K_h y K_w representan las dimensiones del kernel.
- k identifica el filtro utilizado.
- Y_k corresponde al mapa de características producido por dicho filtro.

Esta operación permite que la CNN aprenda detectores especializados en estructuras locales.

En las imágenes de hojas de tomate, estos patrones pueden estar relacionados con:

- pequeños cambios de color;
- bordes de lesiones;
- manchas;
- texturas;
- variaciones locales de intensidad;
- estructuras foliares características.

Conceptualmente:

Región local de píxeles → Filtro convolucional → Característica detectada

Por ejemplo:

Píxeles → Borde

o:

Píxeles → Textura

Esta propiedad resulta particularmente relevante porque las enfermedades foliares no necesariamente afectan de manera uniforme toda la imagen, sino que pueden manifestarse mediante **patrones localizados**.

2. Compartición de pesos

Una segunda ventaja fundamental de las CNN es la **compartición de pesos**.

En una capa convolucional, un mismo filtro se utiliza repetidamente sobre diferentes posiciones de la imagen.

Si un kernel aprende a detectar una determinada estructura visual, por ejemplo el borde de una lesión, los mismos parámetros pueden utilizarse para buscar esa característica en diferentes zonas de la hoja.

Conceptualmente:

Mismo filtro $\rightarrow$ {zona superior zona central zona inferior}

Esto es diferente de una capa densa, donde las conexiones entre entradas y neuronas poseen parámetros independientes.

La cantidad de parámetros de una capa convolucional puede expresarse como:

$$P_{\text{CNN}} = (K_h K_w C_{\text{in}} + 1) C_{\text{out}}$$

donde:

- K_h y K_w representan el tamaño del kernel.
- C_{in} es el número de canales de entrada.
- C_{out} es el número de filtros.
- El término $+1$ representa el sesgo asociado a cada filtro.

Por ejemplo, si la primera convolución utilizara hipotéticamente:

$$K_h = K_w = 3$$

$$C_{\text{in}} = 3$$

y:

$$C_{\text{out}} = 32$$

entonces el número de parámetros sería:

$$P_{\text{CNN}} = (3 \times 3 \times 3 + 1) \times 32$$

$$P_{\text{CNN}} = 28 \times 32$$

$$P_{\text{CNN}} = 896$$

Es importante señalar que este cálculo constituye **un ejemplo arquitectónico**; la arquitectura definitiva todavía deberá definirse y entrenarse en fases posteriores.

La idea fundamental es que esos **896 parámetros se reutilizan en toda la imagen**, en lugar de crear un conjunto diferente de pesos para cada posición espacial.

3. Aprendizaje jerárquico de características

Las CNN permiten construir representaciones progresivamente más complejas mediante la combinación sucesiva de capas convolucionales.

Las primeras capas pueden aprender características simples:

Píxeles $\rightarrow$ Bordes

Las capas intermedias pueden combinar esos patrones:

Bordes $\rightarrow$ Texturas

Posteriormente:

Texturas $\rightarrow$ Manchas y formas

y las capas profundas pueden integrar esas representaciones:

Manchas + texturas + formas $\rightarrow$ Patrones de lesión

hasta obtener finalmente:

Patrones de lesión $\rightarrow$ Clase sanitaria

El proceso completo puede resumirse como:

Píxeles $\rightarrow$ Bordes $\rightarrow$ Texturas $\rightarrow$ Formas $\rightarrow$ Patrones de lesión $\rightarrow$ Clase

Esta propiedad se denomina **aprendizaje jerárquico de representaciones**.

El modelo no necesita recibir explícitamente variables como:

```
cantidad_de_manchas
tipo_de_textura
forma_de_la_lesion
color_de_la_zona_afectada
```

sino que intenta aprender automáticamente representaciones relevantes a partir de los píxeles.

4. Mayor eficiencia para explotar la estructura espacial

La diferencia entre una capa densa y una convolucional puede observarse también mediante el número de parámetros.

Para una capa densa, si existen D valores de entrada y M neuronas, el número de parámetros es:

$$P_{\text{Dense}} = D \times M + M$$

Dado que una imagen original del proyecto contiene:

$D = 256 \times 256 \times 3$

196,608

si hipotéticamente se conectara directamente a una capa densa de 256 neuronas:

$$P_{\text{Dense}} = 196,608 \times 256 + 256$$

$$P_{\text{Dense}} = 50,331,648 + 256$$

$$P_{\text{Dense}} = 50,331,904$$

En contraste, el ejemplo anterior de una convolución de 32 filtros de tamaño 3×3 utiliza:

$$P_{\text{CNN}} = 896$$

No corresponde comparar ambos valores como si fueran arquitecturas equivalentes, ya que realizan operaciones diferentes. Sin embargo, permiten ilustrar una característica fundamental:

Las convoluciones pueden extraer características espaciales
utilizando un número reducido de parámetros compartidos.

5. Conectividad local frente a conectividad completa

Una MLP utiliza conectividad completa:

cada neurona $\longleftrightarrow$ todas las entradas

En cambio, una CNN utiliza inicialmente conectividad local:

neurona convolucional $\longleftrightarrow$ región local

Esta diferencia se conoce como **campo receptivo local**.

Por ejemplo, con un kernel de:

$$3 \times 3$$

una activación observa inicialmente una región local de la imagen.

Al apilar varias capas convolucionales, el campo receptivo efectivo aumenta progresivamente, permitiendo combinar información de regiones cada vez mayores.

Conceptualmente:

Patrones locales $\rightarrow$ Patrones regionales $\rightarrow$ Representación global

Esto permite conservar una relación directa entre la arquitectura del modelo y la naturaleza espacial del problema.

6. Relación con el problema de PlantVillage

En este proyecto, la CNN deberá distinguir **10 clases** a partir de imágenes de hojas de tomate.

La clasificación puede depender de características como:

Color + Textura + Manchas + Bordos + Forma + Distribución espacial

Estas propiedades pueden aparecer en distintas posiciones de una imagen.

Por ello, la posibilidad de aplicar los mismos filtros sobre toda la superficie resulta especialmente útil.

Por ejemplo:

Filtro detector de lesión → aplicación sobre toda la hoja

sin necesitar aprender un detector independiente para cada posición.

7. Comparación conceptual MLP vs CNN

La diferencia fundamental entre las dos arquitecturas seleccionadas puede resumirse como:

MLP

Imagen → Flatten → Vector → Capas densas → Clase

La organización espacial original deja de estar representada explícitamente.

CNN

Imagen → Convoluciones → Mapas de características → Representaciones jerárquicas → Clase

La CNN incorpora directamente en su arquitectura:

Localidad + Compartición de pesos + Jerarquía espacial

Por ello, la comparación experimental puede interpretarse como:

MLP: aprender desde una representación vectorizada

frente a:

CNN: aprender explotando explícitamente la geometría de la imagen

8. Relación con los datos ya procesados

Los resultados obtenidos hasta esta etapa indican que el experimento se realizará sobre un conjunto de:

18,146 imágenes únicas

correspondientes a:

10 clases

con imágenes originales de:

$256 \times 256 \times 3$

La estructura espacial presente en estas imágenes constituye precisamente la información que una CNN está diseñada para aprovechar.

Por tanto, existe una correspondencia directa entre:

Modalidad de los datos → Arquitectura

específicamente:

Imágenes → CNN

Conclusión de la sección 2.4

La ventaja específica de una CNN para este proyecto se fundamenta en cuatro propiedades principales:

1. Detección de patrones locales 2. Compartición de pesos 3. Aprendizaje jerárquico de características 4

Estas propiedades permiten que una CNN procese las imágenes de hojas conservando y utilizando su organización espacial, mientras aprende automáticamente características visuales relevantes para la clasificación.

En términos conceptuales:

Imagen → Patrones locales → Características jerárquicas → Representación discriminativa → Clase sanitaria

Por tanto, la CNN constituye una arquitectura técnicamente adecuada para analizar las imágenes de PlantVillage y ofrece una **ventaja arquitectónica teórica** frente a una MLP basada en imágenes aplanadas.

No obstante, esta sección establece únicamente la **justificación teórica previa al modelamiento**. La afirmación de que la CNN presenta efectivamente mejor rendimiento o mayor capacidad de generalización **no debe darse por demostrada todavía**.

Esa hipótesis deberá comprobarse experimentalmente mediante:

MLP vs. CNN

utilizando los mismos subconjuntos de datos, las mismas clases y métricas comunes de evaluación.

La conclusión final deberá determinar si:

la ventaja arquitectónica teórica de la CNN → una ventaja observable y generalizable en los resultados

Fase 3 - Auditoría y descripción del dataset

Construir un DataFrame maestro con una fila por imagen. El objetivo es documentar fuente, tamaño, clases, calidad y estructura del dataset utilizado.

3.1 Construcción del DataFrame maestro

Una vez completadas las etapas de selección de clases, consolidación de los directorios originales, verificación de integridad y separación de archivos duplicados, se construirá un **DataFrame maestro** con una fila por cada imagen que permanece en el dataset limpio.

El objetivo de esta estructura es disponer de una representación tabular y reproducible del conjunto de datos utilizado durante el proyecto.

Cada fila representará una imagen:

$$X_i, \quad i = 1, 2, \dots, N$$

y contendrá información relacionada con:

- fuente del dataset;
- ruta de la imagen;
- nombre del archivo;
- clase;
- extensión;
- tamaño del archivo;
- formato;
- modo de color;
- ancho y alto;
- número de canales;
- estado de integridad;
- hash SHA-256.

De esta manera, el dataset de imágenes queda representado mediante una estructura tabular:

$$D = \{X_1, X_2, \dots, X_N\}$$

donde cada observación mantiene asociada su correspondiente clase:

$$(X_i, y_i)$$

siendo y_i la categoría sanitaria de la hoja de tomate.

3.2 Tamaño efectivo del dataset

Antes de la eliminación de duplicados, el conjunto consolidado contenía:

$$N_{\text{antes}} = 18\,160$$

imágenes.

Durante la auditoría mediante hash SHA-256 se identificaron:

$$N_{\text{duplicados}} = 14$$

copias redundantes exactas.

Estas imágenes fueron apartadas del directorio principal, por lo que el tamaño efectivo del dataset limpio es:

$$N_{\text{limpio}} = N_{\text{antes}} - N_{\text{duplicados}}$$

$$N_{\text{limpio}} = 18\,160 - 14$$

$$\boxed{N_{\text{limpio}} = 18\,146}$$

El porcentaje global de copias redundantes detectadas fue:

$$P_{\text{duplicados}} = \frac{N_{\text{duplicados}}}{N_{\text{antes}}} \times 100$$

$$P_{\text{duplicados}} = \frac{14}{18\,160} \times 100 \approx \boxed{0.08\%}$$

3.3 Número de clases

Después del filtrado se conservaron exclusivamente las categorías correspondientes a hojas de tomate.

El número definitivo de clases es:

$$K = 10$$

El dataset limpio cumple:

$$N_{\text{limpio}} = \sum_{i=1}^K N_i$$

donde N_i representa la cantidad de imágenes pertenecientes a la clase i .

Para los datos procesados:

$$18\,146 = \sum_{i=1}^{10} N_i$$

3.4 Distribución de imágenes por clase

La cantidad final de imágenes por categoría es:

Clase	Imágenes	Participación
Tomato__Bacterial_spot	2.127	11,72 %
Tomato__Early_blight	1.000	5,51 %
Tomato__Late_blight	1.901	10,48 %
Tomato__Leaf_Mold	952	5,25 %
Tomato__Septoria_leaf_spot	1.771	9,76 %
Tomato__Spider_mites Two-spotted_spider_mite	1.676	9,24 %
Tomato__Target_Spot	1.404	7,74 %
Tomato__Tomato_Yellow_Leaf_Curl_Virus	5.357	29,52 %
Tomato__Tomato_mosaic_virus	373	2,06 %
Tomato__healthy	1.585	8,73 %
Total	18.146	100 %

La participación de cada clase se calcula mediante:

$$P_i = \frac{N_i}{N_{\text{limpio}}} \times 100$$

donde:

- N_i corresponde al número de imágenes de la clase i ;
- N_{limpio} corresponde al número total de imágenes únicas.

3.5 Clase mayoritaria

La clase mayoritaria puede definirse como:

$$c_{\text{max}} = \underset{i}{\operatorname{argmax}} N_i$$

En el dataset procesado corresponde a:

$$c_{\max} = \text{Tomato_Tomato_Yellow_Leaf_Curl_Virus}$$

con:

$$N_{\max} = 5\,357$$

imágenes, equivalentes aproximadamente al:

$$29.52\%$$

del dataset limpio.

3.6 Clase minoritaria

La clase minoritaria se define como:

$$c_{\min} = \underset{i}{\operatorname{argmin}} N_i$$

Para el dataset procesado:

$$c_{\min} = \text{Tomato_Tomato_mosaic_virus}$$

con:

$$N_{\min} = 373$$

imágenes, equivalentes aproximadamente al:

$$2.06\%$$

del conjunto de datos.

La relación entre la clase mayoritaria y minoritaria puede expresarse mediante:

$$IR = \frac{N_{\max}}{N_{\min}}$$

$$IR = \frac{5\,357}{373} \approx 14.36$$

Por tanto, la clase mayoritaria contiene aproximadamente **14,4 veces más imágenes que la clase minoritaria**, lo que confirma la existencia de desbalance entre categorías.

3.7 Duplicados apartados

El análisis mediante hash SHA-256 identificó **14 copias redundantes exactas**.

Los duplicados se concentraron en dos clases:

Clase	Duplicados apartados
Tomato___Late_blight	8
Tomato___healthy	6
Total	14

No se detectaron imágenes idénticas asociadas a clases diferentes.

Después de apartar las copias redundantes se verificó que:

$$N_{\text{duplicados restantes}} = 0$$

en el dataset principal.

3.8 Control de consistencia

La relación entre el dataset original consolidado, las imágenes conservadas y los duplicados debe satisfacer:

$$N_{\text{antes}} = N_{\text{limpio}} + N_{\text{duplicados}}$$

En este caso:

$$18\,160 = 18\,146 + 14$$

Por lo tanto:

$$18\,160 = 18\,160$$

lo que confirma la consistencia cuantitativa del proceso de depuración.

3.9 Conclusión de la auditoría

La Fase 3 permite establecer que el dataset efectivo utilizado en las siguientes etapas del proyecto está compuesto por:

$$\begin{aligned} N_{\text{antes}} &= 18\,160 \\ N_{\text{duplicados}} &= 14 \\ N_{\text{limpio}} &= 18\,146 \\ K &= 10 \\ N_{\text{max}} &= 5\,357 \\ N_{\text{min}} &= 373 \\ IR &\approx 14.36 \end{aligned}$$

El DataFrame maestro proporcionará una fuente única y estructurada para las siguientes etapas del proyecto, especialmente el análisis exploratorio, la división estratificada en train, validation y test, el control de fuga de información y el posterior entrenamiento de las arquitecturas MLP y CNN.

Fase 4 - EDA visual

Explorar el dataset antes del entrenamiento para entender la distribución y la diversidad visual.

Gráfico 1 - Distribución por clase

Barplot con clase en el eje X y número de imágenes en el eje Y. Permitirá detectar desbalance.

4.1 Gráfico 1 — Distribución de imágenes por clase

El primer análisis visual tiene como objetivo estudiar la **distribución de las imágenes entre las clases** del conjunto de datos depurado. Esta etapa permite identificar si todas las categorías poseen una cantidad similar de observaciones o si existen clases significativamente sobrerrepresentadas o subrepresentadas.

Después de las etapas anteriores de consolidación, verificación de integridad y eliminación de duplicados exactos, el dataset contiene:

$$N = 18\,146$$

imágenes únicas, distribuidas en:

$$K = 10$$

clases correspondientes a diferentes enfermedades foliares del tomate y al estado saludable.

Sea:

$$n_k$$

la cantidad de imágenes pertenecientes a la clase k . Entonces, el tamaño total del conjunto de datos puede expresarse como:

$$N = \sum_{k=1}^K n_k$$

La participación porcentual de cada clase se calcula mediante:

$$p_k = \frac{n_k}{N} \times 100$$

donde:

- n_k representa el número de imágenes de la clase k ;
- N representa el número total de imágenes;
- p_k representa la participación porcentual de esa clase en el dataset.

Si las imágenes estuvieran distribuidas perfectamente de forma uniforme entre las diez categorías, la cantidad teórica de observaciones por clase sería:

$$n_{\text{eq}} = \frac{N}{K}$$

Para el conjunto de datos actual:

$$n_{\text{eq}} = \frac{18\,146}{10} = 1\,814.6$$

Este valor no representa una cantidad real obligatoria de imágenes por clase, sino una **referencia teórica de equilibrio** que permite visualizar qué categorías se encuentran por encima o por debajo de una distribución uniforme.

También puede cuantificarse el grado de desbalance mediante la **razón de desbalance**:

$$IR = \frac{n_{\max}}{n_{\min}}$$

donde:

- $n_{\max}$ corresponde al número de imágenes de la clase mayoritaria;
- $n_{\min}$ corresponde al número de imágenes de la clase minoritaria.

En los datos actualmente procesados:

$$n_{\max} = 5\,357$$

para la clase:

Tomato__Tomato_Yellow_Leaf_Curl_Virus

mientras que:

$$n_{\min} = 373$$

para:

Tomato__Tomato_mosaic_virus

Por tanto:

$$IR = \frac{5\,357}{373} \approx 14.36$$

Esto significa que la clase con mayor número de observaciones contiene aproximadamente **14.36 veces más imágenes** que la clase con menor representación.

El gráfico de barras permitirá observar visualmente esta distribución y determinar si el desbalance debe ser considerado posteriormente durante la partición de los datos, el entrenamiento y la selección de métricas de evaluación.

Interpretación del Gráfico 1 — Distribución por clase

El gráfico evidencia que las **18.146 imágenes** del conjunto de datos depurado no se encuentran distribuidas uniformemente entre las diez categorías.

La clase con mayor representación corresponde a:

Tomato__Tomato_Yellow_Leaf_Curl_Virus

con:

$$n_{\max} = 5\,357$$

imágenes, equivalentes aproximadamente al:

$$29.52\%$$

del conjunto de datos.

En contraste, la clase con menor número de observaciones es:

Tomato__Tomato_mosaic_virus

con:

$$n_{\min} = 373$$

imágenes, correspondientes aproximadamente al:

$$2.06\%$$

del dataset.

La razón entre la clase mayoritaria y la minoritaria es:

$$IR = \frac{5\,357}{373} \approx 14.36$$

Por tanto, la categoría con mayor representación posee aproximadamente **14.36 veces más imágenes** que la categoría con menor representación.

Si las observaciones estuvieran distribuidas uniformemente entre las diez clases, cada categoría tendría teóricamente:

$$n_{eq} = \frac{18\,146}{10} = 1\,814.6$$

imágenes.

El gráfico muestra que varias categorías se encuentran considerablemente por encima o por debajo de esta referencia. En particular, `Tomato__Tomato_Yellow_Leaf_Curl_Virus` presenta una representación claramente superior al resto, mientras que `Tomato__Tomato_mosaic_virus` posee una cantidad mucho menor de observaciones.

En consecuencia, se concluye que el dataset presenta **desbalance entre clases**. Esta característica deberá considerarse en las siguientes fases del proyecto, especialmente al realizar la división estratificada de **Train / Validation / Test** y al seleccionar las métricas de evaluación.

Debido a este desbalance, la **accuracy global no debería utilizarse como única medida de rendimiento**, ya que un modelo podría obtener un resultado aparentemente favorable al predecir mejor las clases con mayor representación. Posteriormente será conveniente complementar la evaluación con métricas como:

- Balanced Accuracy.
- Precision Macro.
- Recall Macro.
- F1-score Macro.
- Matriz de confusión.

Este análisis no implica todavía modificar la distribución de los datos. Su objetivo en esta fase es **diagnosticar y documentar el desbalance antes de tomar decisiones de preprocesamiento o entrenamiento**.

Gráfico 2 - Ejemplos aleatorios

Mostrar una muestra representativa de imágenes con su clase correspondiente.

4.2 Gráfico 2 — Ejemplos aleatorios de imágenes

Después de analizar la distribución cuantitativa de las clases, se realiza una inspección visual del conjunto de datos mediante una **muestra aleatoria de imágenes**.

El objetivo de esta etapa es observar directamente las características visuales presentes en PlantVillage y comprobar si el conjunto depurado presenta condiciones adecuadas para el posterior entrenamiento de los modelos MLP y CNN.

Sea el conjunto de datos limpio:

$$\mathcal{D} = \{(X_i, y_i)\}_{i=1}^N$$

donde:

- X_i corresponde a la imagen i ;
- y_i corresponde a la clase asociada a esa imagen;
- N corresponde al número total de imágenes válidas y únicas.

En el dataset actualmente procesado:

$$N = 18\,146$$

y existen:

$$K = 10$$

clases.

Para realizar la inspección visual se selecciona una muestra aleatoria sin reemplazo:

$$\mathcal{S} \subset \mathcal{D}$$

tal que:

$$|\mathcal{S}| = m$$

donde m representa el número de imágenes que serán mostradas.

La selección se realiza **sin reemplazo**, por lo que una misma imagen no puede aparecer dos veces dentro de la muestra:

$$X_i \neq X_j \quad \forall i \neq j$$

Se utiliza una semilla fija:

$$SEED = 42$$

para que la selección sea reproducible. Esto significa que, mientras el dataset no cambie, volver a ejecutar la celda producirá la misma muestra de imágenes.

Objetivos de la inspección visual

La muestra permitirá observar preliminarmente:

- diferencias de color entre hojas;
- presencia de manchas, lesiones y cambios de textura;
- distribución espacial de los síntomas;
- variaciones de iluminación;
- orientación y tamaño aparente de las hojas;
- homogeneidad de los fondos;
- similitud visual entre algunas enfermedades;
- posibles anomalías que no hayan sido detectadas durante la auditoría automática.

Esta visualización constituye un análisis exploratorio cualitativo. Por tanto, **no se utilizará todavía para modificar, normalizar o aumentar artificialmente las imágenes**.

Interpretación del Gráfico 2 — Ejemplos aleatorios

La inspección de una muestra aleatoria permite complementar el análisis cuantitativo realizado previamente con una evaluación cualitativa de las imágenes.

La selección fue realizada sin reemplazo y utilizando:

$$SEED = 42$$

por lo que la muestra puede reproducirse mientras se mantenga sin modificaciones el conjunto de datos.

Cada observación del problema puede representarse mediante:

$$(X_i, y_i)$$

donde X_i corresponde a la información visual contenida en la imagen y y_i representa la enfermedad o estado sanitario asociado.

Esta inspección permite estudiar características que no están explícitamente representadas como variables tabulares, entre ellas:

- coloración de las hojas;
- manchas y lesiones;
- patrones de textura;
- bordes y formas;
- distribución espacial de los síntomas;
- orientación de la hoja;
- iluminación;
- características del fondo.

Estas propiedades resultan relevantes para la hipótesis central del proyecto porque la clasificación depende de información espacial distribuida sobre la superficie de la imagen.

Una MLP utilizará posteriormente una representación vectorizada de estos píxeles:

$$X \in \mathbb{R}^{H \times W \times C} \rightarrow x \in \mathbb{R}^{HWC}$$

mientras que una CNN conservará inicialmente la estructura:

$$X \in \mathbb{R}^{H \times W \times C}$$

permitiendo aprender patrones locales relacionados con bordes, texturas, manchas y otras características visuales.

Por tanto, la inspección visual de ejemplos aleatorios aporta evidencia preliminar de que la información relevante para la clasificación no está contenida solamente en valores individuales de píxeles, sino también en **sus relaciones espaciales**.

En esta etapa el análisis tiene carácter exploratorio. Las conclusiones sobre similitud entre clases y variabilidad dentro de cada categoría serán estudiadas con mayor detalle en el siguiente gráfico, donde se mostrarán múltiples ejemplos organizados explícitamente por clase.

Gráfico 3 - Ejemplos por clase

Mostrar varias imágenes por categoría para observar variabilidad intra-clase y similitud inter-clase.

4.3 Gráfico 3 — Ejemplos por clase

Después de analizar la distribución cuantitativa de las categorías y observar una muestra aleatoria global, se realiza una inspección visual **controlada por clase**.

El objetivo de esta visualización es mostrar varias imágenes pertenecientes a cada categoría para estudiar dos propiedades importantes del conjunto de datos:

- **Variabilidad intra-clase:** grado de diversidad visual existente entre imágenes que poseen la misma etiqueta.
- **Similitud inter-clase:** grado de parecido visual que puede existir entre imágenes pertenecientes a categorías diferentes.

El conjunto de datos limpio puede representarse como:

$$\mathcal{D} = \{(X_i, y_i)\}_{i=1}^N$$

donde:

$$N = 18\,146$$

representa el número total de imágenes válidas y únicas, y:

$$K = 10$$

corresponde al número de clases.

Para cada clase:

$$c_k \in \mathcal{C}, \quad k = 1, \dots, K$$

se define el subconjunto:

$$\mathcal{D}_k = \{(X_i, y_i) \in \mathcal{D} : y_i = c_k\}$$

que contiene todas las imágenes asociadas a la categoría c_k .

A diferencia del muestreo aleatorio global utilizado anteriormente, en esta visualización se selecciona el **mismo número de imágenes para cada clase**.

Sea:

$$m = 3$$

el número de imágenes seleccionadas por categoría. Para cada clase se obtiene entonces una muestra:

$$\mathcal{S}_k \subset \mathcal{D}_k$$

tal que:

$$|\mathcal{S}_k| = m$$

La muestra visual completa estará formada por:

$$\mathcal{S} = \bigcup_{k=1}^K \mathcal{S}_k$$

y el número total de imágenes visualizadas será:

$$|\mathcal{S}| = K \cdot m$$

Por tanto:

$$|\mathcal{S}| = 10 \cdot 3 = 30$$

imágenes.

La selección dentro de cada categoría se realiza **sin reemplazo**, por lo que una misma imagen no puede aparecer repetida dentro de una clase:

$$X_i \neq X_j, \quad \forall i \neq j$$

Además, se utiliza una semilla fija:

$$SEED = 42$$

para garantizar la reproducibilidad de la selección.

Variabilidad intra-clase

Para una determinada categoría c_k , interesa estudiar visualmente la diversidad existente entre:

$$X_i, X_j \in \mathcal{D}_k$$

aunque ambas observaciones compartan:

$$y_i = y_j = c_k$$

Las diferencias pueden encontrarse en características como color, textura, extensión de las lesiones, orientación de la hoja, iluminación o distribución espacial de los síntomas.

Similitud inter-clase

También interesa comparar imágenes correspondientes a clases diferentes:

$$X_i \in \mathcal{D}_a, \quad X_j \in \mathcal{D}_b, \quad a \neq b$$

para determinar si existen patrones visuales similares capaces de dificultar posteriormente la clasificación.

Esta inspección resulta especialmente relevante para la comparación experimental entre MLP y CNN, debido a que las CNN pueden aprender relaciones espaciales locales mediante operaciones de convolución, mientras que la MLP trabajará posteriormente con una representación vectorizada de la imagen.

En esta etapa las imágenes se muestran **sin normalización, redimensionamiento para entrenamiento ni data augmentation**, ya que el objetivo es inspeccionar el contenido visual original del dataset depurado.

4.4 Preguntas de análisis del EDA visual

Una vez estudiada la distribución de las clases, una muestra aleatoria global y ejemplos organizados por categoría, es posible realizar un diagnóstico exploratorio del conjunto de datos antes de proceder a la partición **Train / Validation / Test**.

El análisis debe distinguir entre dos conceptos diferentes:

1. **Homogeneidad técnica**, relacionada con resolución, formato, canales e integridad de los archivos.
2. **Variabilidad visual**, relacionada con color, textura, forma, lesiones y distribución espacial de los síntomas.

Estas propiedades no son contradictorias. Un dataset puede ser técnicamente homogéneo y, al mismo tiempo, presentar suficiente diversidad visual para constituir un problema de clasificación no trivial.

1. ¿Las imágenes son visualmente homogéneas? En términos técnicos, sí existe un alto grado de homogeneidad. Sin embargo, el contenido visual presenta una variabilidad relevante.

Después de la depuración, el conjunto contiene:

$$N = 18\,146$$

imágenes válidas y únicas.

Todas las imágenes presentan:

$$H = 256$$

píxeles de alto,

$$W = 256$$

píxeles de ancho y:

$$C = 3$$

canales de color.

Por tanto, cada imagen posee la misma estructura:

$$X_i \in \mathbb{R}^{256 \times 256 \times 3}$$

Además:

$$100\%$$

de las imágenes utilizan formato **JPEG**, modo de color **RGB** y fueron verificadas satisfactoriamente durante la auditoría de integridad.

La homogeneidad estructural puede expresarse mediante:

$$H_{\text{dim}} = \frac{N_{\text{dimensión modal}}}{N} \times 100$$

En este dataset:

$$H_{\text{dim}} = \frac{18\,146}{18\,146} \times 100 = 100\%$$

De manera análoga:

$$H_{\text{formato}} = 100\%$$

y:

$$H_{\text{RGB}} = 100\%$$

Desde el punto de vista visual, los ejemplos muestran también condiciones de adquisición relativamente controladas: generalmente aparece una hoja como objeto principal y los fondos son visualmente simples.

Sin embargo, **las imágenes no son idénticas ni completamente uniformes en su apariencia**, lo cual es favorable para el aprendizaje. Se observan diferencias en:

- forma y orientación de las hojas;
- intensidad del color verde;
- amarillamiento;
- extensión y localización de lesiones;
- tamaño y distribución de manchas;
- textura;
- grado de deterioro;
- iluminación y tonalidad del fondo.

Por tanto, la conclusión más precisa es:

alta homogeneidad técnica + variabilidad visual útil

Esta combinación resulta apropiada para un problema supervisado de clasificación de imágenes.

2. ¿Hay diferencias evidentes entre las enfermedades? **Sí. La inspección visual muestra diferencias apreciables entre varias categorías, aunque no todas presentan una separación visual igualmente clara.**

Los ejemplos organizados por clase muestran que las enfermedades pueden diferenciarse mediante patrones relacionados con:

color + textura + manchas + necrosis + deformación + distribución espacial

Por ejemplo, en la muestra exploratoria:

- la clase healthy presenta hojas visualmente más uniformes y sin lesiones prominentes;
- Tomato__Tomato_Yellow_Leaf_Curl_Virus muestra alteraciones visibles de coloración y forma;
- Tomato__Late_blight presenta en varios ejemplos áreas extensas de deterioro o necrosis;
- Tomato__Bacterial_spot y Tomato__Septoria_leaf_spot presentan patrones de manchas distribuidas sobre la hoja;
- otras categorías presentan modificaciones más sutiles asociadas principalmente a textura, moteado o cambios de color.

Estas observaciones apoyan la hipótesis de que la variable objetivo depende de información visual estructurada.

Conceptualmente:

$$X_i \longrightarrow \{\text{bordes, colores, texturas, lesiones, formas}\} \longrightarrow y_i$$

Por tanto, existen diferencias visuales potencialmente aprendibles por una red neuronal.

No obstante, esta conclusión es **exploratoria**. La verdadera capacidad de separación entre categorías deberá verificarse posteriormente mediante el rendimiento de los modelos y la matriz de confusión.

3. ¿Existen clases visualmente similares? **Sí. El Gráfico 3 sugiere que algunas categorías comparten características visuales y podrían resultar más difíciles de separar.**

Entre los ejemplos observados aparecen similitudes potenciales entre enfermedades que presentan:

- manchas pequeñas o distribuidas;
- regiones amarillentas;
- cambios de textura;
- lesiones oscuras;
- patrones de deterioro que pueden superponerse parcialmente.

En particular, la inspección visual sugiere semejanzas entre algunos ejemplos de:

- Tomato__Bacterial_spot y Tomato__Septoria_leaf_spot;
- Tomato__Early_blight y Tomato__Late_blight;
- Tomato__Spider_mites Two-spotted_spider_mite y Tomato__Target_Spot.

Estas semejanzas deben considerarse **hipótesis visuales preliminares**, no como evidencia definitiva de confusión.

Posteriormente podrá comprobarse si una similitud observada durante el EDA se traduce realmente en errores del modelo:

$$\text{similitud visual} \not\Rightarrow \text{confusión obligatoria}$$

pero puede plantearse la hipótesis:

$$\text{mayor similitud visual} \longrightarrow \text{mayor dificultad de separación}$$

Esta relación deberá contrastarse posteriormente mediante:

$$\text{Matriz de confusión} + \text{Recall por clase} + \text{F1 por clase}$$

Por esta razón, el análisis visual realizado ahora será útil para interpretar los errores futuros de la MLP y la CNN.

4. ¿Existe desbalance de clases? **Sí. El dataset presenta un desbalance de clases claramente identificable y cuantificable.**

El conjunto contiene:

$$N = 18\,146$$

imágenes distribuidas entre:

$$K = 10$$

clases.

Si existiera una distribución perfectamente uniforme, cada clase tendría aproximadamente:

$$n_{\text{eq}} = \frac{N}{K} = \frac{18\,146}{10} = 1\,814.6$$

imágenes.

Sin embargo, la clase mayoritaria es:

Tomato__Tomato_Yellow_Leaf_Curl_Virus

con:

$$n_{\text{max}} = 5\,357$$

imágenes, que representan:

$$29.52\%$$

del dataset.

La clase minoritaria es:

Tomato___Tomato_mosaic_virus

con:

$$n_{\min} = 373$$

imágenes, equivalentes a:

$$2.06\%$$

del total.

La razón de desbalance es:

$$IR = \frac{n_{\max}}{n_{\min}}$$

por lo tanto:

$$IR = \frac{5\,357}{373} \approx 14.36$$

Esto significa que la clase mayoritaria contiene aproximadamente:

$$14.36$$

veces más imágenes que la categoría minoritaria.

En consecuencia, el dataset debe considerarse **desbalanceado**.

Este resultado tendrá consecuencias metodológicas importantes:

1. la futura división **Train / Validation / Test** deberá ser estratificada;
2. la accuracy no deberá utilizarse como única métrica;
3. deberán analizarse métricas sensibles al rendimiento por clase, como:

Balanced Accuracy

Precision_{macro}

Recall_{macro}

F1_{macro}

además de la matriz de confusión.

En esta etapa, sin embargo, el desbalance solamente se **diagnostica y documenta**. La estrategia para tratarlo durante el entrenamiento deberá decidirse posteriormente y no debe confundirse con el EDA.

5. ¿La calidad visual parece suficientemente consistente para entrenamiento? Sí. La evidencia disponible indica que la calidad técnica del dataset es suficientemente consistente para avanzar hacia el modelamiento.

La auditoría realizada previamente produjo:

$$N_{\text{válidas}} = 18\,146$$

y:

$$N_{\text{problemáticas}} = 0$$

por lo que la tasa de integridad del dataset limpio es:

$$Q = \frac{N_{\text{válidas}}}{N} \times 100$$

$$Q = \frac{18\,146}{18\,146} \times 100 = 100\%$$

Además:

- todas las imágenes tienen resolución 256×256 ;
- todas poseen 3 canales;
- todas utilizan modo RGB;
- todas corresponden al formato JPEG;
- no existen rutas ausentes en las visualizaciones de la Fase 4;
- después de apartar las copias redundantes quedan:

$$N_{\text{duplicados restantes}} = 0$$

Por tanto, desde el punto de vista técnico:

integridad + resolución uniforme + color uniforme + ausencia de duplicados exactos

constituyen condiciones favorables para continuar con la preparación de los datos.

La inspección visual también muestra imágenes suficientemente claras para reconocer estructuras foliares y patrones de lesión.

No obstante, una calidad técnica elevada **no garantiza automáticamente una buena generalización**. Las imágenes parecen provenir de condiciones relativamente controladas, por lo que el rendimiento obtenido posteriormente sobre PlantVillage no necesariamente representará el comportamiento del modelo frente a fotografías tomadas en condiciones reales de terreno.

Síntesis del EDA Los resultados de la Fase 4 pueden resumirse como:

Pregunta	Diagnóstico
¿Las imágenes son visualmente homogéneas?	Sí en estructura técnica; existe variabilidad visual útil
¿Hay diferencias evidentes entre enfermedades?	Sí, aunque con diferente grado de separación visual
¿Existen clases visualmente similares?	Sí, se observan posibles solapamientos visuales
¿Existe desbalance de clases?	Sí, $IR \approx 14.36$
¿La calidad parece suficiente para entrenamiento?	Sí, integridad técnica del 100%

En conjunto:

Dataset técnicamente consistente + problema visual no trivial + desbalance relevante

Por tanto, existe evidencia suficiente para avanzar a la siguiente etapa del pipeline, manteniendo especial atención sobre:

estratificación + métricas balanceadas + generalización

Fase 5 - Control de calidad

Antes de crear los subconjuntos de datos deben identificarse problemas capaces de sesgar el entrenamiento o producir fallos de lectura.

Fase 5 — Control de calidad del dataset

Antes de crear los subconjuntos de entrenamiento, validación y prueba, se realiza una revisión final de calidad destinada a identificar problemas capaces de producir errores durante el entrenamiento, sesgar las métricas o generar fuga de información.

En esta fase se controlan cinco aspectos:

1. archivos vacíos, corruptos, truncados o ilegibles;
2. dimensiones y canales de color;
3. formatos reales de las imágenes;
4. duplicados exactos mediante hash SHA-256;
5. imágenes idénticas asociadas a clases diferentes.

El análisis utiliza los resultados generados previamente durante la auditoría y depuración del dataset, evitando repetir operaciones innecesarias o volver a mover archivos que ya fueron apartados.

5.1 Integridad de los archivos

Sea:

$$N_{\text{total}}$$

el número total de imágenes examinadas y:

$$N_{\text{válidas}}$$

el número de imágenes que superan todos los controles de integridad.

La tasa de integridad se define como:

$$Q_{\text{integridad}} = \frac{N_{\text{válidas}}}{N_{\text{total}}} \times 100$$

En el dataset original consolidado se analizaron:

$$N_{\text{total}} = 18\,160$$

imágenes.

Todas superaron las verificaciones de apertura, estructura y decodificación:

$$N_{\text{válidas}} = 18\,160$$

por lo tanto:

$$Q_{\text{integridad}} = \frac{18\,160}{18\,160} \times 100 = 100\%$$

No se detectaron:

$$N_{\text{vacías}} = 0$$

ni archivos con dimensiones inválidas:

$$N_{\text{dimensiones inválidas}} = 0$$

La configuración:

```
python ImageFile.LOAD_TRUNCATED_IMAGES = False
```

5.7 Interpretación final del control de calidad

El proceso de control de calidad confirma que el dataset se encuentra en condiciones adecuadas para avanzar hacia la división **Train / Validation / Test**.

La auditoría inicial examinó:

18 160

imágenes y determinó que:

18 160

eran válidas, obteniéndose:

$$Q_{\text{integridad}} = 100\%$$

No se identificaron archivos vacíos, imágenes con dimensiones inválidas ni archivos que no pudieran ser correctamente decodificados.

Asimismo, todas las imágenes presentan una estructura uniforme:

$$256 \times 256 \times 3$$

con modo de color:

RGB

y formato interno:

JPEG

lo que reduce la necesidad de resolver inconsistencias técnicas antes del preprocesamiento.

La detección mediante SHA-256 identificó inicialmente:

14

grupos de duplicados exactos, correspondientes a:

$$28$$

archivos.

Al conservar una copia de cada grupo se apartaron:

$$14$$

archivos redundantes, por lo que el tamaño final del dataset es:

$$18\,160 - 14 = 18\,146$$

imágenes únicas.

La verificación posterior confirma:

$$N_{\text{duplicados restantes}} = 0$$

Además, no se encontraron imágenes idénticas asociadas a clases distintas:

$$N_{\text{conflictos}} = 0$$

por lo que no existe evidencia de conflictos de etiquetado derivados de duplicados exactos.

Desde el punto de vista metodológico, este resultado es especialmente importante porque la eliminación de duplicados se realizó **antes de generar los subconjuntos de entrenamiento, validación y prueba**.

De este modo se reduce el riesgo de que una misma imagen aparezca simultáneamente en diferentes particiones:

$$X_i^{train} = X_j^{test}$$

situación que produciría **data leakage** y podría generar una estimación artificialmente optimista de la capacidad de generalización.

En consecuencia:

$$\boxed{\text{integridad} = 100\%}$$

$$\boxed{\text{duplicados exactos restantes} = 0}$$

$$\boxed{\text{conflictos entre clases} = 0}$$

y se concluye que el dataset ha superado satisfactoriamente la **Fase 5 — Control de calidad**.

El siguiente paso corresponde a crear una nueva partición estratificada:

$$70\% \text{ Train} \quad 15\% \text{ Validation} \quad 15\% \text{ Test}$$

manteniendo el conjunto de prueba completamente aislado hasta la evaluación final.

Fase 6 - División Train / Validation / Test

Crear una nueva partición controlada a partir del dataset consolidado y depurado. Se propone una división estratificada 70/15/15 con semilla fija.

Fase 6 — División Train / Validation / Test

Una vez finalizadas las etapas de consolidación, depuración, análisis exploratorio y control de calidad, se procede a crear la **partición experimental definitiva** del conjunto de datos.

El dataset limpio contiene:

$$N = 18\,146$$

imágenes únicas, distribuidas entre:

$$K = 10$$

clases.

La partición propuesta es:

$$70\% \text{ Train} \quad 15\% \text{ Validation} \quad 15\% \text{ Test}$$

o equivalentemente:

$$p_{\text{train}} = 0.70$$

$$p_{\text{val}} = 0.15$$

$$p_{\text{test}} = 0.15$$

cumpléndose:

$$p_{\text{train}} + p_{\text{val}} + p_{\text{test}} = 1$$

6.1 Función de cada subconjunto

El conjunto de entrenamiento:

$$\mathcal{D}_{\text{train}}$$

será utilizado para ajustar los parámetros internos de los modelos.

El conjunto de validación:

$$\mathcal{D}_{\text{val}}$$

permitirá evaluar el comportamiento del modelo durante el desarrollo, seleccionar configuraciones y detectar posibles problemas de sobreajuste.

Finalmente:

$$\mathcal{D}_{\text{test}}$$

se mantendrá completamente aislado del proceso de selección de modelos y se utilizará únicamente para estimar la capacidad final de generalización.

Conceptualmente:

$$\mathcal{D} = \mathcal{D}_{train} \cup \mathcal{D}_{val} \cup \mathcal{D}_{test}$$

y los tres subconjuntos deben ser mutuamente excluyentes:

$$\mathcal{D}_{train} \cap \mathcal{D}_{val} = \emptyset$$

$$\mathcal{D}_{train} \cap \mathcal{D}_{test} = \emptyset$$

$$\mathcal{D}_{val} \cap \mathcal{D}_{test} = \emptyset$$

Además, la unión de los tres subconjuntos debe reconstruir exactamente el dataset limpio:

$$|\mathcal{D}_{train}| + |\mathcal{D}_{val}| + |\mathcal{D}_{test}| = N$$

6.2 Partición estratificada

Debido al desbalance observado durante el EDA, no se utilizará una división aleatoria simple.

La clase mayoritaria contiene:

$$5\,357$$

imágenes, mientras que la clase minoritaria contiene:

$$373$$

por lo que se obtuvo:

$$IR = \frac{5\,357}{373} \approx 14.36$$

La partición será entonces **estratificada según la variable objetivo clase**.

Sea:

$$n_k$$

el número total de observaciones pertenecientes a la clase k .

Idealmente, cada categoría conservará aproximadamente:

$$n_{k,train} \approx 0.70 n_k$$

$$n_{k,val} \approx 0.15 n_k$$

$$n_{k,test} \approx 0.15 n_k$$

Debido a que el número de imágenes debe ser entero, pueden existir pequeñas diferencias producidas por redondeo.

Otra forma de expresar el objetivo de la estratificación es que la participación de una clase dentro de cada subconjunto permanezca aproximadamente igual a su participación en el dataset completo:

$$P(y = c_k \mid Train) \approx P(y = c_k)$$

$$P(y = c_k \mid Validation) \approx P(y = c_k)$$

$$P(y = c_k \mid Test) \approx P(y = c_k)$$

Es importante señalar que **estratificar no significa balancear las clases**.

La estratificación conserva la distribución original:

Stratified Split $\neq$ Class Balancing

El tratamiento del desbalance durante el entrenamiento será una decisión metodológica posterior.

6.3 Procedimiento de división

La división se realizará en dos etapas.

Primero:

$$\mathcal{D} \longrightarrow 70\% Train + 30\% Temporal$$

Posteriormente, el subconjunto temporal se divide en partes iguales:

$$30\% Temporal \longrightarrow 15\% Validation + 15\% Test$$

porque:

$$0.30 \times 0.50 = 0.15$$

La semilla utilizada será:

$$SEED = 42$$

para garantizar que la misma ejecución sobre el mismo dataset produzca exactamente la misma partición.

6.4 Prevención de Data Leakage

Además de verificar que las rutas sean diferentes, se comprobará que los hashes SHA-256 tampoco aparezcan en más de una partición.

Sean:

$$H_{train}, \quad H_{val}, \quad H_{test}$$

los conjuntos de hashes correspondientes a cada subconjunto.

Debe cumplirse:

$$H_{train} \cap H_{val} = \emptyset$$

$$H_{train} \cap H_{test} = \emptyset$$

$$H_{val} \cap H_{test} = \emptyset$$

Este control complementa la eliminación de duplicados realizada previamente y protege la evaluación frente a posibles fugas de información.

6.5 Interpretación de la partición obtenida

La división estratificada generó tres subconjuntos independientes a partir de las:

$$N = 18\,146$$

imágenes únicas del dataset depurado.

La distribución obtenida fue aproximadamente:

$$70\% \text{ Train} \quad 15\% \text{ Validation} \quad 15\% \text{ Test}$$

correspondiente a:

$$N_{train} = 12\,702$$

$$N_{val} = 2\,722$$

$$N_{test} = 2\,722$$

y se verifica:

$$12\,702 + 2\,722 + 2\,722 = 18\,146$$

Por tanto, no se perdieron ni se agregaron observaciones durante la partición.

Conservación de la distribución de clases

La utilización de una estrategia estratificada permitió mantener aproximadamente la misma proporción de cada categoría dentro de Train, Validation y Test.

Para cada clase c_k :

$$P(y = c_k \mid \text{Train}) \approx P(y = c_k \mid \text{Validation}) \approx P(y = c_k \mid \text{Test}) \approx P(y = c_k)$$

Las pequeñas diferencias observadas son consecuencia del redondeo necesario para asignar cantidades enteras de imágenes.

La estratificación resulta especialmente importante debido al desbalance previamente identificado:

$$IR \approx 14.36$$

ya que una partición aleatoria no controlada podría reducir excesivamente la representación de las clases minoritarias en Validation o Test.

Ausencia de solapamiento

Las verificaciones realizadas confirman:

$$\mathcal{D}_{train} \cap \mathcal{D}_{val} = \emptyset$$

$$\mathcal{D}_{train} \cap \mathcal{D}_{test} = \emptyset$$

$$\mathcal{D}_{val} \cap \mathcal{D}_{test} = \emptyset$$

Asimismo, la comparación mediante hashes SHA-256 confirma que ninguna imagen binariamente idéntica se encuentra presente en dos subconjuntos diferentes.

Por tanto:

$$\boxed{\text{solapamiento} = 0}$$

y no existe evidencia de fuga de información causada por imágenes duplicadas entre las particiones.

Función del conjunto Test

A partir de esta fase, el conjunto:

$$\mathcal{D}_{test}$$

debe mantenerse aislado.

No deberá utilizarse para:

selección de arquitectura

ajuste de hiperparámetros

selección de época

decisiones de regularización

Su única función será proporcionar posteriormente una estimación final de la capacidad de generalización.

En consecuencia, las decisiones durante el desarrollo deberán basarse exclusivamente en:

$$\mathcal{D}_{train} \quad \text{y} \quad \mathcal{D}_{val}$$

reservando:

$$\mathcal{D}_{test}$$

para la evaluación final.

Conclusión de la Fase 6

La partición cumple los requisitos fundamentales de:

reproducibilidad + estratificación + disjunción + cobertura completa

Por tanto, el dataset se encuentra preparado para avanzar a la **Fase 7 — Preprocesamiento**, donde se definirán la resolución de entrada, la normalización y las transformaciones aplicadas exclusivamente al conjunto de entrenamiento.

6.1 División estratificada 70 % / 15 % / 15 %

Después de completar la auditoría, depuración y control de calidad, el dataset definitivo contiene:

$$N = 18\,146$$

imágenes únicas, distribuidas en:

$$K = 10$$

clases.

El conjunto de datos se divide en tres subconjuntos mutuamente excluyentes:

$$\mathcal{D} = \mathcal{D}_{train} \cup \mathcal{D}_{val} \cup \mathcal{D}_{test}$$

utilizando las proporciones:

$$p_{train} = 0.70$$

$$p_{val} = 0.15$$

$$p_{test} = 0.15$$

cumpléndose:

$$p_{train} + p_{val} + p_{test} = 1$$

Por tanto, conceptualmente:

$$70\% \text{ Train} \quad 15\% \text{ Validation} \quad 15\% \text{ Test}$$

Función de cada subconjunto

El conjunto de entrenamiento:

$$\mathcal{D}_{train}$$

se utilizará para ajustar los parámetros internos de los modelos.

El conjunto de validación:

$$\mathcal{D}_{val}$$

se utilizará para controlar el proceso de entrenamiento, seleccionar configuraciones y analizar sobreajuste.

Finalmente:

$$\mathcal{D}_{test}$$

se mantendrá aislado hasta la evaluación final y se utilizará únicamente para estimar la capacidad de generalización del modelo seleccionado.

Estratificación por clase

Debido al desbalance previamente identificado:

$$IR = \frac{n_{\max}}{n_{\min}} = \frac{5357}{373} \approx 14.36$$

la división no se realiza mediante un muestreo aleatorio simple, sino utilizando **estratificación por la variable objetivo clase**.

Sea:

$$n_k$$

el número de imágenes pertenecientes a la clase k .

Idealmente, cada categoría se distribuye aproximadamente como:

$$n_{k,train} \approx 0.70 n_k$$

$$n_{k,val} \approx 0.15 n_k$$

$$n_{k,test} \approx 0.15 n_k$$

Debido a que el número de imágenes debe ser entero, pueden producirse pequeñas diferencias ocasionadas por redondeo.

La propiedad que se desea conservar es:

$$P(y = c_k \mid Train) \approx P(y = c_k)$$

$$P(y = c_k \mid Validation) \approx P(y = c_k)$$

$$P(y = c_k \mid Test) \approx P(y = c_k)$$

Por tanto:

$$\text{Estratificación} \neq \text{Balanceo de clases}$$

La estratificación conserva aproximadamente la distribución original; no modifica artificialmente la cantidad de observaciones de cada categoría.

Reproducibilidad

Se utiliza una semilla fija:

$$SEED = 42$$

de modo que, manteniendo el mismo dataset y el mismo procedimiento, la partición pueda reproducirse.

Independencia entre subconjuntos

Para evitar fuga de información debe cumplirse:

$$\mathcal{D}_{train} \cap \mathcal{D}_{val} = \emptyset$$

$$\mathcal{D}_{train} \cap \mathcal{D}_{test} = \emptyset$$

$$\mathcal{D}_{val} \cap \mathcal{D}_{test} = \emptyset$$

Además, dado que durante las fases anteriores se calculó un hash SHA-256 para cada imagen, también se verificará:

$$H_{train} \cap H_{val} = \emptyset$$

$$H_{train} \cap H_{test} = \emptyset$$

$$H_{val} \cap H_{test} = \emptyset$$

donde H representa el conjunto de hashes de cada partición.

Este segundo control es más estricto que comparar únicamente las rutas, ya que permite detectar imágenes binariamente idénticas aunque posean nombres diferentes.

6.2 Interpretación de la partición estratificada

La división del dataset se realizó satisfactoriamente utilizando:

$$SEED = 42$$

y una estrategia estratificada:

$$70\% \text{ Train} \quad 15\% \text{ Validation} \quad 15\% \text{ Test}$$

A partir de:

$$N = 18\,146$$

imágenes únicas se obtuvieron:

$$N_{train} = 12\,702$$

$$N_{val} = 2\,722$$

$$N_{test} = 2\,722$$

verificándose:

$$12\,702 + 2\,722 + 2\,722 = 18\,146$$

Por tanto, la partición conserva exactamente todas las observaciones del dataset limpio.

Conservación de la distribución de clases La estratificación permitió mantener prácticamente constante la proporción de cada clase dentro de los tres subconjuntos.

La mayor desviación encontrada respecto de la distribución global fue de solamente:

$$0.0242$$

puntos porcentuales.

Este resultado indica que:

$$P(y = c_k | Train) \approx P(y = c_k | Validation) \approx P(y = c_k | Test) \approx P(y = c_k)$$

para las diez categorías.

La estratificación resulta especialmente importante debido al desbalance previamente identificado:

$$IR \approx 14.36$$

ya que evita que las clases minoritarias pierdan representación de forma desproporcionada en Validation o Test.

Ausencia de solapamiento La comparación de las rutas produjo:

$$|Train \cap Validation| = 0$$

$$|Train \cap Test| = 0$$

$$|Validation \cap Test| = 0$$

La misma comprobación mediante hashes SHA-256 produjo:

$$|H_{train} \cap H_{val}| = 0$$

$$|H_{train} \cap H_{test}| = 0$$

$$|H_{val} \cap H_{test}| = 0$$

Por tanto, ninguna imagen aparece en más de un subconjunto y no existe evidencia de fuga de información ocasionada por archivos o contenidos duplicados.

Conclusión La división cumple simultáneamente cuatro condiciones metodológicas fundamentales:

reproducibilidad + estratificación + disjunción + cobertura completa

En consecuencia, la **Fase 6 queda aprobada** y las particiones pueden utilizarse como base común para el entrenamiento posterior de la MLP y la CNN.

El conjunto:

$$\mathcal{D}_{test}$$

debe permanecer aislado hasta la evaluación final. Las decisiones de arquitectura, regularización e hiperparámetros deberán realizarse exclusivamente utilizando:

$$\mathcal{D}_{train} \quad \text{y} \quad \mathcal{D}_{val}$$

De esta forma, la evaluación sobre Test proporcionará una estimación independiente de la capacidad de generalización.

6.3 Creación física de los conjuntos Train, Validation y Test

Una vez realizada la división estratificada del dataset en los conjuntos de entrenamiento, validación y prueba, se procede a materializar físicamente dicha partición en directorios independientes.

La división ya fue determinada previamente mediante los DataFrames:

- df_train
- df_val
- df_test

Por lo tanto, **no se realiza una nueva división aleatoria**. Esta etapa únicamente copia las imágenes correspondientes a cada partición, conservando la asignación de clases definida previamente.

La estructura resultante será:

```
dataset_split/
├── train/
│   ├── Tomato__Bacterial_spot/
│   ├── Tomato__Early_blight/
│   └── ...
├── validation/
│   ├── Tomato__Bacterial_spot/
│   ├── Tomato__Early_blight/
│   └── ...
└── test/
    ├── Tomato__Bacterial_spot/
    ├── Tomato__Early_blight/
    └── ...
```

Las imágenes originales **no se eliminan ni se modifican**. Se utiliza `shutil.copy2()` para crear una copia de cada archivo.

Este procedimiento facilita posteriormente la construcción de datasets con TensorFlow/Keras mediante `image_dataset_from_directory()` y mantiene una separación explícita entre entrenamiento, validación y prueba.

La cantidad total de archivos debe satisfacer:

$$N = N_{\{train\}} + N_{\{validation\}} + N_{\{test\}}$$

Para el dataset actualmente utilizado:

$$\text{\$ } 18,146 = 12,702 + 2,722 + 2,722 \text{ \$}$$

La separación física constituye únicamente una representación en disco de la partición ya definida; no modifica el protocolo experimental.

Fase 7 - Preprocesamiento

Estandarizar las imágenes y preparar las transformaciones necesarias sin introducir fuga de información.

Fase 7 — Preprocesamiento de imágenes

En esta fase se prepara el pipeline definitivo de entrada para los modelos MLP y CNN. El objetivo es estandarizar las imágenes y aplicar transformaciones que favorezcan el entrenamiento sin introducir fuga de información.

Las imágenes originales presentan una resolución de:

$$256 \times 256 \times 3$$

y serán redimensionadas dinámicamente a:

$$128 \times 128 \times 3$$

Esta reducción disminuye la dimensionalidad desde:

$$256 \times 256 \times 3 = 196\,608$$

hasta:

$$128 \times 128 \times 3 = 49\,152$$

lo que representa una reducción del:

$$75\%$$

en la cantidad de valores procesados por imagen.

Posteriormente, los valores de píxel se normalizan mediante:

$$x' = \frac{x}{255}$$

transformando:

$$x \in [0, 255]$$

en:

$$x' \in [0, 1]$$

La normalización es determinista y se aplica por igual a Train, Validation y Test.

Finalmente se incorpora **Data Augmentation únicamente al conjunto de entrenamiento** mediante transformaciones geométricas moderadas:

- RandomFlip

- RandomRotation
- RandomZoom

Por tanto, el protocolo definitivo es:

$$\mathcal{D}_{train} \rightarrow Resize \rightarrow Normalización \rightarrow Data Augmentation$$

mientras que:

$$\mathcal{D}_{validation} \rightarrow Resize \rightarrow Normalización$$

y:

$$\mathcal{D}_{test} \rightarrow Resize \rightarrow Normalización$$

Validation y Test no reciben transformaciones aleatorias, ya que deben permanecer estables y reproducibles para evaluar correctamente la capacidad de generalización de los modelos.

Los datasets resultantes serán:

- train_ds_final: Resize + Normalización + Data Augmentation.
- val_ds_final: Resize + Normalización.
- test_ds_final: Resize + Normalización.

Estos tres datasets constituirán la entrada común para la comparación posterior entre MLP y CNN.

7.4 Determinación y demostración de las transformaciones de Data Augmentation

El **Data Augmentation** permite introducir variabilidad controlada en el conjunto de entrenamiento mediante transformaciones aleatorias de las imágenes originales.

En este proyecto se analizarán tres transformaciones:

- RandomFlip
- RandomRotation
- RandomZoom

Estas operaciones se aplicarán exclusivamente sobre imágenes pertenecientes al conjunto de entrenamiento:

$$\mathcal{D}_{train}$$

Validation y Test permanecerán sin modificaciones aleatorias.

1. RandomFlip

RandomFlip genera una reflexión de la imagen.

En este proyecto se utilizará reflexión horizontal:

$$X_{\text{flip}} = T_{\text{flip}}(X)$$

Conceptualmente, para una imagen con ancho W :

$$X_{\text{flip}}(i, j, c) = X(i, W - 1 - j, c)$$

donde:

- i representa la posición vertical;

- j representa la posición horizontal;
- c corresponde al canal RGB.

Esta transformación es razonable para imágenes de hojas, porque invertir horizontalmente la orientación de una hoja no modifica la enfermedad que representa.

2. RandomRotation

RandomRotation introduce pequeñas variaciones angulares.

Se utilizará:

$$factor = 0.05$$

Keras interpreta este valor como una fracción de una vuelta completa. Por tanto:

$$\theta_{\max} = 0.05 \times 360^\circ = 18^\circ$$

La transformación se encuentra aproximadamente dentro de:

$$-18^\circ \leq \theta \leq 18^\circ$$

y puede expresarse conceptualmente como:

$$X_{\text{rot}} = T_\theta(X)$$

Esta transformación permite disminuir la dependencia del modelo respecto de la orientación exacta de la hoja.

3. RandomZoom

RandomZoom modifica moderadamente la escala aparente de la imagen.

Se utilizará un intervalo de aproximadamente $\pm 10\%$:

$$z \in [-0.10, 0.10]$$

de manera que puedan generarse pequeñas variaciones de acercamiento o alejamiento:

$$X_{\text{zoom}} = T_z(X)$$

El objetivo es evitar que el modelo aprenda que una enfermedad está asociada a un tamaño o encuadre específico de la hoja.

Transformación combinada

Durante el entrenamiento, las transformaciones pueden combinarse:

$$X_{\text{aug}} = T_{\text{zoom}}(T_{\text{rotation}}(T_{\text{flip}}(X)))$$

Por tanto:

$$X \rightarrow \text{RandomFlip} \rightarrow \text{RandomRotation} \rightarrow \text{RandomZoom} \rightarrow X_{\text{aug}}$$

Las transformaciones son aleatorias, por lo que una misma imagen puede generar variantes diferentes en distintas iteraciones:

$$T_1(X) \neq T_2(X) \neq T_3(X)$$

Sin embargo, la etiqueta debe permanecer inalterada:

$$y_{\text{aug}} = y$$

El propósito no es crear nuevas clases, sino aumentar la diversidad visual de los ejemplos disponibles durante el entrenamiento.

Criterio de selección

Las transformaciones elegidas son deliberadamente moderadas.

Se evita aplicar modificaciones excesivas que puedan alterar características relevantes de las enfermedades, como:

- manchas;
- lesiones;
- patrones de color;
- textura;
- distribución espacial de los síntomas.

La transformación debe modificar la apariencia de la imagen sin modificar su significado semántico.

Por tanto, el conjunto seleccionado para los experimentos será:

$$\text{RandomFlip} + \text{RandomRotation} + \text{RandomZoom}$$

aplicado exclusivamente a:

$$\text{Train}$$

Fase 8 - Baseline

Incluir, cuando sea razonable, un modelo simple para contextualizar el aporte de Deep Learning. El baseline no reemplaza ninguna de las dos redes obligatorias.

Fase 8 — Baseline convencional

8.1 Objetivo del modelo baseline

Antes de entrenar las arquitecturas profundas MLP y CNN se construirá un modelo convencional que permita establecer un nivel de referencia.

La comparación general del proyecto será:

$$\text{Baseline} \rightarrow \text{MLP} \rightarrow \text{CNN}$$

El propósito del baseline no es competir necesariamente con las redes profundas, sino responder una pregunta metodológica fundamental:

$$\text{¿Cuánto valor adicional aporta Deep Learning?}$$

En este proyecto se utilizará como baseline:

HOG + Regresión Logística

Esta estrategia corresponde a un enfoque clásico de visión por computador.

A diferencia de una red neuronal profunda, que aprende automáticamente las representaciones relevantes a partir de los píxeles, el baseline requiere una etapa explícita de extracción de características.

El flujo será:

Imagen $\rightarrow$ Resize $\rightarrow$ Normalización $\rightarrow$ HOG $\rightarrow$ Regresión Logística $\rightarrow \hat{y}$

8.2 ¿Por qué utilizar HOG?

Las imágenes normalizadas provenientes de la Fase 7 tienen dimensiones:

$$X \in [0, 1]^{128 \times 128 \times 3}$$

Si se utilizaran directamente todos los píxeles como variables para una regresión logística, cada imagen tendría:

$$128 \times 128 \times 3 = 49\,152$$

características.

Esto generaría un espacio de entrada considerablemente grande para un modelo convencional.

Por este motivo se utilizará **Histogram of Oriented Gradients (HOG)** como método clásico de extracción de características.

HOG describe principalmente la distribución local de:

- bordes;
- contornos;
- orientaciones;
- cambios de intensidad;
- estructuras locales.

Para cada región de la imagen se calcula aproximadamente la magnitud del gradiente:

$$G = \sqrt{G_x^2 + G_y^2}$$

y su orientación:

$$\theta = \text{atan2}(G_y, G_x)$$

donde:

- G_x representa el cambio horizontal de intensidad;
- G_y representa el cambio vertical;
- G representa la magnitud del gradiente;
- θ representa su orientación.

Posteriormente, estas orientaciones se agrupan en histogramas locales, produciendo un vector de características:

$$\mathbf{h} = [h_1, h_2, \dots, h_p]$$

La diferencia conceptual con Deep Learning es importante:

Baseline → características diseñadas manualmente

mientras que:

CNN → características aprendidas automáticamente

Esta diferencia permitirá posteriormente analizar si el aprendizaje automático de representaciones espaciales proporciona una ventaja sobre una descripción visual convencional.

8.3 Clasificador: Regresión Logística multiclase

Una vez obtenidas las características HOG, se utilizará una regresión logística para realizar la clasificación de las diez enfermedades o estados foliares.

Para una clase k , el modelo calcula un valor lineal:

$$z_k = \mathbf{w}_k^\top \mathbf{h} + b_k$$

La probabilidad asociada a cada clase puede expresarse mediante Softmax:

$$P(y = k \mid \mathbf{h}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

donde:

$$K = 10$$

es el número de clases.

La predicción final corresponde a:

$$\hat{y} = \arg \max_k P(y = k \mid \mathbf{h})$$

8.4 Tratamiento del desbalance de clases

El análisis exploratorio mostró que el dataset presenta desbalance entre sus categorías.

Por esta razón se utilizará:

`class_weight="balanced"`

La ponderación asignada aproximadamente a cada clase k es:

$$w_k = \frac{N}{K N_k}$$

donde:

- N es el número total de observaciones de entrenamiento;
- K es el número total de clases;
- N_k es el número de observaciones pertenecientes a la clase k .

De esta forma, las clases minoritarias reciben un peso proporcionalmente mayor durante el ajuste del modelo.

8.5 Protocolo experimental

El baseline respetará exactamente la división determinada previamente:

$$\mathcal{D}_{train} \cap \mathcal{D}_{validation} = \emptyset$$

y:

$$\mathcal{D}_{test}$$

permanecerá aislado.

En esta fase:

$$\mathcal{D}_{train} \rightarrow \text{entrenamiento del baseline}$$

$$\mathcal{D}_{validation} \rightarrow \text{evaluación preliminar}$$

mientras que:

$$\boxed{\mathcal{D}_{test} \text{ no se utiliza todavía}}$$

El conjunto Test será reservado para la comparación final entre:

$$\boxed{\text{Baseline vs. MLP vs. CNN}}$$

El baseline utilizará `train_ds_norm` y `val_ds_norm`, es decir, imágenes redimensionadas y normalizadas pero **sin Data Augmentation aleatorio**.

Esto permite mantener un modelo convencional determinista y reproducible.

8.6 Métricas de evaluación

Debido al desbalance de clases, Accuracy no será utilizada como única medida.

Se calcularán:

Accuracy

$$Accuracy = \frac{\text{predicciones correctas}}{N}$$

Balanced Accuracy

$$Balanced Accuracy = \frac{1}{K} \sum_{k=1}^K Recall_k$$

Precision Macro

$$Precision_{macro} = \frac{1}{K} \sum_{k=1}^K Precision_k$$

Recall Macro

$$Recall_{macro} = \frac{1}{K} \sum_{k=1}^K Recall_k$$

F1 Macro

$$F1_{macro} = \frac{1}{K} \sum_{k=1}^K F1_k$$

También se analizará la matriz de confusión para identificar qué clases presentan mayor dificultad para el modelo convencional.

El resultado de esta fase constituirá el punto de referencia para determinar posteriormente si las arquitecturas profundas proporcionan una mejora real.

Fase 9 - Modelo A: MLP

La MLP será la primera familia de red y trabajará sobre una representación vectorizada de las imágenes.

9.1 Entrada de la MLP: imágenes redimensionadas y normalizadas

La entrada de la red neuronal multicapa corresponde a las imágenes del conjunto **PlantVillage**, restringido a las diez clases de hojas de tomate. Estas imágenes ya fueron auditadas, depuradas y divididas mediante una partición estratificada en:

- **Entrenamiento:** 12.702 imágenes.
- **Validación:** 2.722 imágenes.
- **Prueba:** 2.722 imágenes.
- **Total:** 18.146 imágenes únicas y válidas.

Las imágenes originales poseen una resolución de:

$$256 \times 256 \times 3$$

donde el valor 3 representa los tres canales del espacio de color RGB. Para reducir el costo computacional y permitir el entrenamiento en Google Colab, las imágenes fueron redimensionadas a:

$$H \times W \times C = 128 \times 128 \times 3$$

Por lo tanto, cada imagen de entrada puede representarse mediante un tensor:

$$\mathbf{X}_i \in \mathbb{R}^{128 \times 128 \times 3}$$

donde:

- $H = 128$ corresponde a la altura.
- $W = 128$ corresponde al ancho.
- $C = 3$ corresponde a los canales rojo, verde y azul.
- i identifica una imagen del dataset.

Normalización de los píxeles

Antes de ingresar al modelo, los valores originales de los píxeles, almacenados en el intervalo $[0, 255]$, se transforman al intervalo $[0, 1]$. La normalización se define como:

$$x'_{h,w,c} = \frac{x_{h,w,c}}{255}$$

donde:

- $x_{h,w,c}$ es el valor original del píxel ubicado en la fila h , columna w y canal c .
- $x'_{h,w,c}$ es el valor normalizado.
- 255 es el valor máximo representable en una imagen RGB de 8 bits.

Como resultado:

$$x_{h,w,c} \in [0, 255] \implies x'_{h,w,c} \in [0, 1]$$

La normalización evita que la red trabaje con valores numéricos excesivamente grandes y facilita la optimización mediante descenso de gradiente. Además, mantiene una escala homogénea para todos los píxeles.

Representación por lotes

Las imágenes se procesan en lotes de 32 observaciones. Por tanto, cada lote utilizado por la red posee la forma:

$$\mathbf{X}_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 128 \times 3}$$

Las etiquetas del mismo lote presentan la forma:

$$\mathbf{y}_{\text{batch}} \in \{0, 1, \dots, 9\}^{32}$$

Cada valor de $\mathbf{y}_{\text{batch}}$ es un número entero que identifica una de las diez clases sanitarias del tomate.

Entrada específica de la MLP

Aunque las imágenes se conservan inicialmente como tensores tridimensionales, una MLP recibe vectores. Por ello, la primera operación de la arquitectura será una capa Flatten, encargada de transformar cada imagen de:

$$128 \times 128 \times 3$$

en un vector de:

$$D = 128 \cdot 128 \cdot 3 = 49.152$$

características. Matemáticamente:

$$\text{Flatten} : \mathbb{R}^{128 \times 128 \times 3} \longrightarrow \mathbb{R}^{49.152}$$

Por consiguiente, un lote se transformará de:

$$(32, 128, 128, 3)$$

a:

$$(32, 49.152)$$

Esta transformación no elimina los valores de los píxeles, pero deja de representar explícitamente su organización bidimensional. La MLP deberá aprender las relaciones entre las 49.152 entradas mediante capas densamente conectadas.

Esta limitación constituye precisamente una de las hipótesis del experimento: se espera que la CNN pueda aprovechar mejor la localidad espacial mediante convoluciones, mientras que la MLP operará sobre una representación vectorizada.

Tratamiento de cada partición

El pipeline conserva una separación metodológica estricta:

Conjunto	Redimensionamiento	Normalización	Data augmentation
Train	Sí	Sí	Sí
Validation	Sí	Sí	No
Test	Sí	Sí	No

El data augmentation se aplica exclusivamente a entrenamiento. Los conjuntos de validación y prueba permanecen sin transformaciones aleatorias, de modo que la evaluación se realice siempre sobre imágenes estables.

En esta fase se reutilizan directamente los objetos creados anteriormente:

- `train_ds_norm`: entrenamiento redimensionado y normalizado.
- `val_ds_norm`: validación redimensionada y normalizada.
- `test_ds_norm`: prueba redimensionada y normalizada.
- `train_ds_final`: entrenamiento con normalización y data augmentation.
- `val_ds_final`: validación normalizada y sin augmentation.
- `test_ds_final`: prueba normalizada y sin augmentation.

No se realiza una nueva división ni se vuelve a cargar el dataset. Esto garantiza que la MLP y la CNN trabajen posteriormente con las mismas observaciones y bajo el mismo protocolo experimental.

9.2 Aplanamiento de las imágenes mediante Flatten

Las imágenes procesadas en la Fase 7 poseen una resolución de:

$$H \times W \times C = 128 \times 128 \times 3$$

donde: - $H = 128$ corresponde a la altura. - $W = 128$ corresponde al ancho. - $C = 3$ representa los canales RGB.

Por tanto, cada imagen normalizada se representa mediante un tensor tridimensional:

$$\mathbf{X}_i \in \mathbb{R}^{128 \times 128 \times 3}$$

Las redes neuronales multicapa o **MLP** operan mediante capas densamente conectadas. Una capa densa espera recibir un vector de características por cada observación y no una matriz tridimensional.

Por esta razón, antes de ingresar a las capas densas se utiliza una capa Flatten, cuya función es reorganizar los píxeles de cada imagen en un único vector.

Transformación matemática

La operación de aplanamiento puede representarse como:

$$\text{Flatten} : \mathbb{R}^{H \times W \times C} \longrightarrow \mathbb{R}^D$$

donde la dimensión del vector resultante es:

$$D = H \cdot W \cdot C$$

Para las imágenes utilizadas en este proyecto:

$$\begin{aligned} D &= 128 \cdot 128 \cdot 3 \\ &= 49.152 \end{aligned}$$

Por consiguiente:

$$\text{Flatten} : \mathbb{R}^{128 \times 128 \times 3} \longrightarrow \mathbb{R}^{49.152}$$

Cada imagen se transforma en un vector:

$$\mathbf{x}_i = [x_{i,1} \quad x_{i,2} \quad \cdots \quad x_{i,49.152}]^T$$

donde cada componente corresponde a un valor normalizado de uno de los píxeles y canales de la imagen.

Transformación de un lote

El pipeline utiliza lotes de 32 imágenes. Antes del aplanamiento, un lote tiene la forma:

$$\mathbf{X}_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 128 \times 3}$$

Después de aplicar Flatten, la forma se transforma en:

$$\mathbf{X}_{\text{flat}} \in \mathbb{R}^{32 \times 49.152}$$

Es importante observar que la dimensión correspondiente al número de imágenes del lote no cambia:

$$32 \times 128 \times 128 \times 3 \longrightarrow 32 \times 49.152$$

La transformación se aplica independientemente a cada observación.

¿Flatten modifica los valores de los píxeles?

La capa Flatten no aprende parámetros y no modifica los valores numéricos de los píxeles. Solamente cambia la organización de los datos.

Por tanto:

$$\text{Flatten}(\mathbf{X}_i) = \text{reshape}(\mathbf{X}_i)$$

y se conserva la cantidad total de elementos:

$$128 \cdot 128 \cdot 3 = 49.152$$

En consecuencia, la suma, la media, el mínimo y el máximo de los valores de cada imagen permanecen iguales antes y después del aplanamiento.

Cantidad de parámetros de Flatten

La capa Flatten no contiene pesos ni sesgos entrenables:

$$N_{\text{parámetros de Flatten}} = 0$$

Su función es exclusivamente estructural. Los parámetros entrenables aparecen en la capa densa posterior.

Si el vector aplanado de 49.152 características se conecta a una capa densa con n neuronas, la cantidad de parámetros de esa capa será:

$$N_{\text{Dense}} = D \cdot n + n$$

donde: - $D = 49.152$ es la cantidad de entradas. - n es la cantidad de neuronas. - $D \cdot n$ corresponde a los pesos. - n corresponde a los sesgos.

De manera equivalente:

$$N_{\text{Dense}} = (D + 1)n$$

Por ejemplo, para una primera capa con 128 neuronas:

$$N_{\text{Dense}} = (49.152 + 1) \cdot 128 = 6.291.584$$

Esto evidencia que una MLP aplicada directamente sobre imágenes puede generar una cantidad considerable de parámetros, incluso cuando la primera capa densa posee un número moderado de neuronas.

Limitación espacial de Flatten

El aplanamiento conserva todos los valores, pero deja de representar explícitamente las relaciones espaciales entre los píxeles.

Antes del aplanamiento, un píxel puede identificarse mediante:

$$x_{h,w,c}$$

donde h , w y c indican su posición espacial y canal. Después del aplanamiento, el mismo valor se representa mediante una posición lineal:

$$x_j, \quad j \in \{1, 2, \dots, 49.152\}$$

La MLP recibe el vector completo, pero no incorpora de manera explícita conceptos como: - Vecindad entre píxeles. - Patrones locales. - Bordes. - Texturas. - Distribución espacial de las lesiones. - Invariancia aproximada frente a desplazamientos.

En una capa densa, cada neurona se conecta con todas las entradas:

$$z_j = \sum_{k=1}^D w_{jk} x_k + b_j$$

y su activación puede calcularse mediante ReLU:

$$a_j = \text{ReLU}(z_j) = \max(0, z_j)$$

La red deberá aprender las relaciones espaciales indirectamente a partir de una gran cantidad de pesos.

Esta característica constituye una parte central del experimento:

MLP $\rightarrow$ imagen aplanada $\rightarrow$ conexiones globales

CNN $\rightarrow$ imagen espacial $\rightarrow$ filtros locales y pesos compartidos

La comparación posterior permitirá determinar si conservar y explotar explícitamente la estructura espacial proporciona una ventaja de generalización para la clasificación de enfermedades foliares del tomate.

9.4 Función de pérdida: SparseCategoricalCrossentropy

El entrenamiento de una red neuronal requiere una **función de pérdida** que cuantifique la diferencia entre las predicciones generadas por el modelo y las clases verdaderas.

En este proyecto se resuelve un problema de clasificación multiclase con:

\$ K=10 \$

categorías mutuamente excluyentes. Cada imagen pertenece exactamente a una clase de enfermedad foliar o a la categoría saludable.

Las etiquetas fueron generadas por `image_dataset_from_directory` utilizando codificación entera. Por tanto:

\$ y_i \in \{0, 1, \dots, K-1\} \$

y, para este proyecto:

\$ y_i \in \{0, 1, \dots, 9\} \$

Por ejemplo, una etiqueta entera puede interpretarse como:

\$ y_i=0 \implies \text{class_names}[0] \$

\$ y_i=9 \implies \text{class_names}[9] \$

Dado que las etiquetas se encuentran representadas mediante números enteros, la función de pérdida apropiada es:

```
“python tf.keras.losses.SparseCategoricalCrossentropy( from_logits=False )
```

9.5 Optimizador inicial: Adam

Una vez calculada la función de pérdida, el entrenamiento debe actualizar los pesos y sesgos de la MLP con el objetivo de reducir progresivamente el error.

El algoritmo encargado de realizar esta actualización se denomina **optimizador**. Para la configuración inicial de la MLP se utilizará Adam:

```
“python tf.keras.optimizers.Adam( learning_rate=0.001 )
```

9.6 Registro de learning rate, batch size, épocas, parámetros y regularización

Antes de entrenar la MLP es necesario registrar formalmente su configuración. Esto permite:

- Reproducir el experimento.
- Comparar posteriormente la MLP con la CNN.
- Interpretar diferencias de rendimiento.
- Controlar la complejidad del modelo.
- Detectar posibles problemas de sobreajuste.
- Evitar cambios de hiperparámetros no documentados.

La configuración inicial propuesta es:

Componente	Configuración
Resolución de entrada	$128 \times 128 \times 3$
Características después de Flatten	49.152
Primera capa oculta	128 neuronas, ReLU
Dropout	0.30
Segunda capa oculta	64 neuronas, ReLU
Capa de salida	10 neuronas, softmax
Optimizador	Adam
Learning rate	0.001
Batch size	32
Épocas máximas	50
Early stopping	Sí
Paciencia	7 épocas
Función de pérdida	Sparse categorical cross-entropy
Regularización L2 inicial	No aplicada
Data augmentation	Solo en train

Arquitectura propuesta

La arquitectura inicial de la MLP puede representarse como:

$$\mathbf{X}_i \in \mathbb{R}^{128 \times 128 \times 3}$$

$$\begin{aligned} \mathbf{X}_i &\longrightarrow \text{Flatten} \longrightarrow \text{Dense}(128, \text{ReLU}) \longrightarrow \text{Dropout}(0.30) \\ &\longrightarrow \text{Dense}(64, \text{ReLU}) \longrightarrow \text{Dense}(10, \text{softmax}) \end{aligned}$$

En forma resumida:

$$128 \times 128 \times 3 \longrightarrow 49.152 \longrightarrow 128 \longrightarrow 64 \longrightarrow 10$$

Learning rate

La tasa de aprendizaje inicial será:

$$\alpha = 0.001 = 10^{-3}$$

Este valor controla la magnitud de las actualizaciones realizadas por Adam:

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}$$

Una tasa demasiado alta puede generar inestabilidad, mientras que una tasa demasiado baja puede producir una convergencia lenta.

El valor 10^{-3} se utilizará como configuración inicial. Cualquier ajuste posterior deberá justificarse mediante resultados de validación y nunca mediante el conjunto test.

Tamaño de lote

El tamaño de lote será:

$$B = 32$$

El conjunto de entrenamiento contiene:

$$N_{\text{train}} = 12.702$$

imágenes. Por tanto, el número de lotes por época será:

$$S = \left\lceil \frac{N_{\text{train}}}{B} \right\rceil$$

$$S = \left\lceil \frac{12.702}{32} \right\rceil = 397$$

Adam realizará aproximadamente 397 actualizaciones de parámetros durante cada época.

Épocas máximas

Se define inicialmente:

$$E_{\text{max}} = 50$$

Si el entrenamiento alcanzara las 50 épocas, el número máximo aproximado de actualizaciones sería:

$$T_{\text{max}} = E_{\text{max}} \cdot S$$

$$T_{\text{max}} = 50 \cdot 397 = 19.850$$

Sin embargo, este valor representa un máximo. El entrenamiento podrá finalizar antes mediante EarlyStopping.

Parámetros entrenables

Para una capa densa con n_{in} entradas y n_{out} neuronas, la cantidad de parámetros es:

$$N_{\text{Dense}} = n_{\text{in}} n_{\text{out}} + n_{\text{out}}$$

o equivalentemente:

$$N_{\text{Dense}} = (n_{\text{in}} + 1) n_{\text{out}}$$

El primer término corresponde a los pesos y el segundo a los sesgos.

1. Primera capa densa La entrada contiene 49.152 características y la primera capa posee 128 neuronas:

$$N_1 = (49.152 + 1) \cdot 128$$

$$N_1 = 6.291.584$$

2. Segunda capa densa La segunda capa recibe 128 entradas y contiene 64 neuronas:

$$N_2 = (128 + 1) \cdot 64$$

$$N_2 = 8.256$$

3. Capa de salida La salida recibe 64 entradas y contiene 10 neuronas:

$$N_3 = (64 + 1) \cdot 10$$

$$N_3 = 650$$

4. Parámetros totales La cantidad total es:

$$N_{\text{total}} = N_1 + N_2 + N_3$$

$$N_{\text{total}} = 6.291.584 + 8.256 + 650$$

$$N_{\text{total}} = 6.300.490$$

Las capas Flatten y Dropout no incorporan parámetros:

$$N_{\text{Flatten}} = 0, \quad N_{\text{Dropout}} = 0$$

La mayor parte de los parámetros se concentra en la primera capa densa debido a la vectorización de la imagen:

$$\frac{N_1}{N_{\text{total}}} \cdot 100 \approx 99.86\%$$

Esto anticipa un riesgo de sobreajuste y constituye una diferencia estructural importante respecto de la CNN.

Regularización mediante Dropout

Se utilizará:

$$p_{\text{drop}} = 0.30$$

Durante el entrenamiento, Dropout desactiva aleatoriamente una proporción de las activaciones de la capa anterior.

Si a_j representa la activación de la neurona j , se genera una máscara:

$$r_j \sim \text{Bernoulli}(q)$$

donde la probabilidad de conservación es:

$$q = 1 - p_{\text{drop}}$$

En este proyecto:

$$q = 1 - 0.30 = 0.70$$

La activación regularizada durante el entrenamiento puede expresarse como:

$$\tilde{a}_j = \frac{r_j}{q} a_j$$

Esta formulación, denominada *inverted dropout*, conserva aproximadamente el valor esperado de las activaciones:

$$\mathbb{E} [\tilde{a}_j] = a_j$$

Durante validación, prueba e inferencia, Dropout se desactiva.

Regularización mediante data augmentation

El conjunto de datos de entrenamiento incorpora técnicas de aumento de datos (data augmentation) para mejorar la capacidad de generalización del modelo y mitigar el sobreajuste.

Fase 10 - Modelo B: CNN

La CNN utilizará convoluciones para aprovechar explícitamente la estructura espacial de las imágenes.

10.1 Mismos conjuntos de datos utilizados por la MLP

La comparación entre la red neuronal multicapa y la red neuronal convolucional debe realizarse bajo un protocolo experimental común.

La única diferencia principal debe ser la manera en que cada arquitectura procesa las imágenes:

MLP $\rightarrow$ imagen aplanada $\rightarrow$ capas densas

CNN $\rightarrow$ imagen espacial $\rightarrow$ convoluciones

Para evitar que las diferencias de rendimiento se deban a cambios en los datos, la MLP y la CNN utilizarán exactamente las mismas particiones.

Dataset depurado

Después del control de integridad y la eliminación de duplicados exactos, el dataset contiene:

$$N = 18.146$$

imágenes válidas y únicas, distribuidas en:

$$K = 10$$

clases de enfermedades foliares y hojas saludables de tomate.

Particiones comunes

El dataset fue dividido mediante una partición estratificada:

$$70\% \text{ Train} \quad 15\% \text{ Validation} \quad 15\% \text{ Test}$$

Los tamaños obtenidos son:

$$N_{\text{train}} = 12.702$$

$$N_{\text{validation}} = 2.722$$

$$N_{\text{test}} = 2.722$$

y:

$$N_{\text{train}} + N_{\text{validation}} + N_{\text{test}} = 18.146$$

Por tanto:

$$12.702 + 2.722 + 2.722 = 18.146$$

Igualdad de las particiones entre modelos

Sea:

$$\mathcal{D}_{\text{train}}^{\text{MLP}}$$

el conjunto de entrenamiento utilizado por la MLP y:

$$\mathcal{D}_{\text{train}}^{\text{CNN}}$$

el conjunto de entrenamiento utilizado por la CNN.

Para garantizar una comparación justa debe cumplirse:

$$\mathcal{D}_{\text{train}}^{\text{MLP}} = \mathcal{D}_{\text{train}}^{\text{CNN}}$$

De la misma manera:

$$\mathcal{D}_{\text{validation}}^{\text{MLP}} = \mathcal{D}_{\text{validation}}^{\text{CNN}}$$

$$\mathcal{D}_{\text{test}}^{\text{MLP}} = \mathcal{D}_{\text{test}}^{\text{CNN}}$$

Esto significa que: - Las mismas imágenes de entrenamiento se utilizan en ambos modelos. - Las mismas imágenes de validación se utilizan en ambos modelos. - Las mismas imágenes de prueba se utilizan en ambos modelos. - Ninguna imagen se traslada entre particiones. - No se crea un nuevo split para la CNN. - No se utiliza una semilla diferente para generar otra división.

Disjunción entre las particiones

Además de ser iguales entre modelos, los conjuntos deben permanecer disjuntos:

$$\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{validation}} = \emptyset$$

$$\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset$$

$$\mathcal{D}_{\text{validation}} \cap \mathcal{D}_{\text{test}} = \emptyset$$

Esta condición evita que una imagen utilizada para ajustar los pesos aparezca también durante la evaluación.

Mismo orden de las clases

La relación entre índice y nombre de clase debe ser idéntica:

$$f_{\text{MLP}}(k) = f_{\text{CNN}}(k) = \text{class_names}[k]$$

para:

$$k \in \{0, 1, \dots, 9\}$$

Por ejemplo, la neurona de salida ubicada en la posición k debe representar la misma clase en ambos modelos.

Si el orden fuera diferente, las predicciones no serían comparables, aunque ambos modelos tuvieran diez neuronas de salida.

Mismo preprocesamiento

Las imágenes utilizadas por ambos modelos mantienen:

$$H \times W \times C = 128 \times 128 \times 3$$

y valores normalizados mediante:

$$x'_{h,w,c} = \frac{x_{h,w,c}}{255}$$

de modo que:

$$x'_{h,w,c} \in [0, 1]$$

Por tanto, la entrada posee la misma escala y resolución para ambas arquitecturas.

Pipeline común

Los datasets definidos en la Fase 7 se reutilizan directamente:

Objeto común	Contenido	Uso
<code>train_ds_final</code>	Resize + normalización + augmentation	Entrenamiento
<code>val_ds_final</code>	Resize + normalización	Selección y validación
<code>test_ds_final</code>	Resize + normalización	Evaluación final

Para documentar explícitamente su uso por la CNN se definirán alias:

```
“python train_ds_cnn = train_ds_final val_ds_cnn = val_ds_final test_ds_cnn = test_ds_final
```

10.2 Métricas comunes para permitir una comparación justa

La comparación entre la MLP y la CNN debe realizarse utilizando exactamente las mismas métricas y los mismos conjuntos de datos.

El protocolo común será:

$$\text{MLP} \rightarrow \mathcal{M}(\mathcal{D}_{\text{validation}})$$

$$\text{CNN} \rightarrow \mathcal{M}(\mathcal{D}_{\text{validation}})$$

donde: - $\mathcal{M}$ representa el conjunto común de métricas. - $\mathcal{D}_{\text{validation}}$ es el mismo conjunto de validación para ambas arquitecturas.

Después de seleccionar las configuraciones finales, el mismo procedimiento se aplicará una sola vez sobre test:

$$\text{MLP final} \rightarrow \mathcal{M}(\mathcal{D}_{\text{test}})$$

$$\text{CNN final} \rightarrow \mathcal{M}(\mathcal{D}_{\text{test}})$$

Protocolo común de evaluación

Las métricas seleccionadas son:

Tipo	Métrica	Función
Optimización	Loss	Medir el error probabilístico
Rendimiento global	Accuracy	Proporción total de aciertos
Rendimiento balanceado	Balanced accuracy	Promedio del recall por clase
Rendimiento macro	Precision macro	Promedio de precision por clase
Rendimiento macro	Recall macro	Promedio de recall por clase
Métrica principal	F1 macro	Equilibrio macro entre precision y recall
Métrica secundaria	F1 weighted	Rendimiento ponderado por soporte
Análisis detallado	Métricas por clase	Identificar clases difíciles
Análisis gráfico	Matriz de confusión	Examinar confusiones sistemáticas

Notación

Sea:

$$N = |\mathcal{D}|$$

la cantidad de imágenes del conjunto evaluado y:

$$K = 10$$

el número de clases.

Para cada clase k , se definen: - TP_k : verdaderos positivos. - FP_k : falsos positivos. - FN_k : falsos negativos. - TN_k : verdaderos negativos.

En un problema multiclase, estas cantidades se calculan tratando cada clase mediante una estrategia one-vs-rest.

Función de pérdida

La pérdida común será:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log(\hat{p}_{i,y_i})$$

donde: - y_i es la clase verdadera. - $\hat{p}_{i,y_i}$ es la probabilidad asignada a la clase correcta.

Tanto la MLP como la CNN utilizarán:

“python SparseCategoricalCrossentropy(from_logits=False)

10.3 Protocolo de entrenamiento comparable

La comparación entre la MLP y la CNN no debe limitarse a entrenar dos arquitecturas diferentes. Ambos modelos deben utilizar un protocolo experimental común que controle las condiciones externas al diseño de la red.

El protocolo compartido puede representarse mediante:

$$\mathcal{P} = \{\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{validation}}, \mathcal{L}, \mathcal{O}, \alpha, B, E_{\text{max}}, \mathcal{C}, \mathcal{M}, s\}$$

donde: - $\mathcal{D}_{\text{train}}$ es el conjunto de entrenamiento. - $\mathcal{D}_{\text{validation}}$ es el conjunto de validación. - $\mathcal{L}$ es la función de pérdida. - $\mathcal{O}$ es el optimizador. - α es el learning rate inicial. - B es el batch size. - E_{max} es el número máximo de épocas. - $\mathcal{C}$ es el conjunto de callbacks. - $\mathcal{M}$ es el conjunto de métricas. - s es la semilla de reproducibilidad.

La condición de comparabilidad es:

$$\mathcal{P}_{\text{MLP}} = \mathcal{P}_{\text{CNN}}$$

excepto por los componentes que necesariamente definen cada arquitectura:

$$\mathcal{A}_{\text{MLP}} \neq \mathcal{A}_{\text{CNN}}$$

Elementos controlados

Componente	MLP	CNN
Train	12.702 imágenes	12.702 imágenes
Validation	2.722 imágenes	2.722 imágenes
Resolución	$128 \times 128 \times 3$	$128 \times 128 \times 3$
Batch size	32	32
Pérdida	Sparse cross-entropy	Sparse cross-entropy
Optimizador	Adam	Adam
Learning rate inicial	0.001	0.001
Épocas máximas	50	50
Early stopping	val_loss	val_loss
Paciencia	7	7
min_delta	10^{-4}	10^{-4}
Mejor checkpoint	Mínima val_loss	Mínima val_loss
Semilla	42	42
Métricas	Comunes	Comunes
Test durante desarrollo	No	No

Diferencia arquitectónica controlada

La entrada de ambas redes es:

$$\mathbf{X}_i \in \mathbb{R}^{128 \times 128 \times 3}$$

La MLP transforma la imagen mediante:

$$128 \times 128 \times 3 \longrightarrow \text{Flatten} \longrightarrow 49.152 \longrightarrow \text{capas densas}$$

La CNN conserva la estructura espacial:

$$128 \times 128 \times 3 \longrightarrow \text{convoluciones} \longrightarrow \text{pooling} \longrightarrow \text{clasificación}$$

La arquitectura constituye la principal variable experimental:

Variable independiente = tipo de red

Variable dependiente = rendimiento y generalización

Entrenamiento por mini-batches

El conjunto de entrenamiento contiene:

$$N_{\text{train}} = 12.702$$

imágenes y se utiliza:

$$B = 32$$

El número de pasos por época es:

$$S = \left\lceil \frac{N_{\text{train}}}{B} \right\rceil$$

$$S = \left\lceil \frac{12.702}{32} \right\rceil = 397$$

En cada paso se realiza:

forward pass $\rightarrow$ loss $\rightarrow$ backpropagation $\rightarrow$ Adam $\rightarrow$ actualización

Si ambos modelos alcanzaran las 50 épocas:

$$T_{\max} = 50 \cdot 397 = 19.850$$

actualizaciones como máximo.

El número real puede ser inferior debido a EarlyStopping.

Función de pérdida común

Ambos modelos utilizarán:

`python SparseCategoricalCrossentropy( from_logits=False )`

10.4 Salida de K clases mediante Softmax

El problema corresponde a una tarea de clasificación supervisada multiclase de clase única. Cada imagen debe asignarse exactamente a una de las:

$$K = 10$$

categorías sanitarias del tomate.

La capa final de la CNN se define como:

`python Dense( units=NUM_CLASSES, activation="softmax", name="salida_cnn" )`

Fase 11 - Entrenamiento

Registrar el comportamiento por época tanto en entrenamiento como en validación.

11.1 EarlyStopping: detención del entrenamiento cuando no exista mejora

Durante el entrenamiento de una red neuronal, aumentar continuamente el número de épocas no garantiza una mejor capacidad de generalización. Después de cierto punto, el modelo puede seguir reduciendo el error de entrenamiento mientras el error de validación deja de mejorar o comienza a aumentar. Este comportamiento constituye una señal de **sobreajuste**.

Para controlar este problema se utilizará el callback EarlyStopping de Keras. Un callback es un objeto que supervisa el entrenamiento y ejecuta determinadas acciones al finalizar cada época.

Métrica supervisada

El criterio de detención será la pérdida de validación:

$$L_{\text{val}}^{(e)},$$

donde e representa la época actual.

El mejor valor observado hasta la época e se define como:

$$L_{\text{val}}^*(e) = \min_{1 \leq j \leq e} L_{\text{val}}^{(j)}.$$

Como la pérdida es una magnitud que debe minimizarse, una época presenta una mejora significativa cuando:

$$L_{\text{val}}^{(e)} < L_{\text{val}}^*(e-1) - \delta,$$

donde δ corresponde a la mejora mínima exigida mediante el parámetro `min_delta`.

En este proyecto se establece:

$$\delta = 10^{-4} = 0.0001.$$

Por lo tanto, una reducción de `val_loss` inferior a 0.0001 no se considerará una mejora suficiente.

Paciencia

El parámetro `patience` determina cuántas épocas consecutivas puede continuar el entrenamiento sin observar una mejora significativa.

Se utilizará:

$$p = 7.$$

Sea c_e el contador de épocas consecutivas sin mejora:

$$c_e = \begin{cases} 0, & \text{si } L_{\text{val}}^{(e)} < L_{\text{val}}^*(e-1) - \delta, \\ c_{e-1} + 1, & \text{en caso contrario.} \end{cases}$$

El entrenamiento se detiene cuando:

$$c_e \geq p.$$

La paciencia de siete épocas evita detener el entrenamiento debido a pequeñas fluctuaciones aleatorias de la pérdida de validación.

Restauración de los mejores pesos

Se utilizará:

`python restore_best_weights=True`

11.2 ModelCheckpoint: conservación del mejor modelo

Durante el entrenamiento, los parámetros de una red neuronal cambian en cada época. Sin embargo, la última época ejecutada no necesariamente corresponde al modelo con mejor capacidad de generalización.

Por esta razón se utilizará el callback `ModelCheckpoint`, cuya función es supervisar una métrica de validación y guardar el modelo solamente cuando se obtiene una mejora.

Criterio de selección

Para mantener el mismo protocolo experimental utilizado por `EarlyStopping`, se supervisará la pérdida de validación:

$$L_{\text{val}}^{(e)},$$

donde e representa la época actual.

El mejor valor observado hasta la época e se define como:

$$L_{\text{val}}^*(e) = \min_{1 \leq j \leq e} L_{\text{val}}^{(j)}.$$

Como la pérdida debe minimizarse, el modelo de la época actual se guarda cuando:

$$L_{\text{val}}^{(e)} < L_{\text{val}}^*(e - 1).$$

La mejor época queda determinada por:

$$e^* = \arg \min_{1 \leq e \leq E} L_{\text{val}}^{(e)},$$

donde E corresponde al número efectivo de épocas ejecutadas.

El modelo guardado será, por lo tanto:

$$\mathcal{M}^* = \mathcal{M}^{(e^*)}.$$

Configuración de ModelCheckpoint

Parámetro	Valor	Función
filepath	Archivo .keras	Ruta donde se guardará el modelo
monitor	val_loss	Métrica utilizada para seleccionar el mejor modelo
mode	min	Una pérdida menor representa una mejora
save_best_only	True	Conserva únicamente el mejor modelo
save_weights_only	False	Guarda la arquitectura y los parámetros
verbose	1	Informa cuándo se actualiza el archivo

Archivos independientes

Para evitar que un modelo sobrescriba al otro, se utilizarán rutas diferentes:

“text/content/plantvillage/modelos/mejor_modelo_mlp.keras/content/plantvillage/modelos/mejor_modelo_cnn.keras”

11.3 Registro de la mejor época y los mejores valores de validación

El entrenamiento genera un historial que registra el comportamiento del modelo en cada época. Para la época e , se almacenan las siguientes métricas:

$$L_{\text{train}}^{(e)}, \quad L_{\text{val}}^{(e)}, \quad A_{\text{train}}^{(e)}, \quad A_{\text{val}}^{(e)},$$

donde:

- $L_{\text{train}}^{(e)}$ es la pérdida de entrenamiento;
- $L_{\text{val}}^{(e)}$ es la pérdida de validación;
- $A_{\text{train}}^{(e)}$ es la exactitud de entrenamiento;
- $A_{\text{val}}^{(e)}$ es la exactitud de validación.

Criterio principal de selección

En este proyecto, EarlyStopping y ModelCheckpoint supervisan `val_loss`. Por lo tanto, la mejor época se define como aquella que minimiza la pérdida de validación:

$$e^* = \arg \min_{1 \leq e \leq E} L_{\text{val}}^{(e)},$$

donde E corresponde al número efectivo de épocas ejecutadas.

La mejor pérdida de validación será:

$$L_{\text{val}}^* = L_{\text{val}}^{(e^*)}.$$

En la misma época se registrarán:

$$L_{\text{train}}^{(e^*)}, \quad A_{\text{train}}^{(e^*)}, \quad A_{\text{val}}^{(e^*)}.$$

De esta manera, todas las métricas principales corresponden al mismo estado del modelo guardado por ModelCheckpoint.

Mejor exactitud de validación

La mayor exactitud de validación observada se calcula de forma complementaria:

$$e_A^* = \arg \max_{1 \leq e \leq E} A_{\text{val}}^{(e)},$$

$$A_{\text{val,max}} = A_{\text{val}}^{(e_A^*)}.$$

No necesariamente se cumple que:

$$e^* = e_A^*.$$

Esto significa que la época con menor pérdida de validación puede ser diferente de la época con mayor exactitud.

La pérdida evalúa la calidad completa de las probabilidades producidas por Softmax, mientras que la exactitud considera solamente si la clase con mayor probabilidad coincide con la etiqueta verdadera.

Brechas de generalización

Para analizar posibles señales de sobreajuste se calcularán las siguientes brechas en la mejor época:

$$\Delta_A = A_{\text{train}}^{(e^*)} - A_{\text{val}}^{(e^*)},$$

$$\Delta_L = L_{\text{val}}^{(e^*)} - L_{\text{train}}^{(e^*)}.$$

Una brecha positiva y elevada puede indicar que el modelo presenta mejor rendimiento sobre entrenamiento que sobre validación. Sin embargo, estas diferencias deben interpretarse conjuntamente con las curvas de aprendizaje y no como una prueba aislada de sobreajuste.

Regla metodológica

La selección oficial del mejor modelo se realizará mediante:

$$e^* = \arg \min_e L_{\text{val}}^{(e)}$$

La mayor exactitud de validación se registrará como información complementaria, pero no reemplazará el criterio utilizado por EarlyStopping y ModelCheckpoint.

El conjunto de prueba no participa en esta selección. Se utilizará una sola vez, después de congelar todas las decisiones experimentales.

Fase 12 - Análisis de sobreajuste

Analizar la diferencia entre el rendimiento de entrenamiento y validación para identificar problemas de generalización

Fase 12 — Análisis de sobreajuste

12.1 Brecha de generalización: Accuracy de entrenamiento vs. validación

Una primera forma de estudiar la presencia de **sobreajuste** consiste en comparar el rendimiento del modelo sobre el conjunto de entrenamiento con su rendimiento sobre el conjunto de validación.

Para cada época e , se define la **brecha de generalización de Accuracy** como:

$$\text{Gap}_{\text{acc}}^{(e)} = \text{Accuracy}_{\text{train}}^{(e)} - \text{Accuracy}_{\text{val}}^{(e)}$$

La misma diferencia puede expresarse en **puntos porcentuales**:

$$\text{Gap}_{\text{pp}}^{(e)} = 100 \left(\text{Accuracy}_{\text{train}}^{(e)} - \text{Accuracy}_{\text{val}}^{(e)} \right)$$

Interpretación

Si:

$$\text{Gap}_{\text{acc}}^{(e)} > 0$$

entonces el rendimiento en entrenamiento es superior al obtenido en validación.

Cuando esta diferencia es **grande, positiva y persistente durante varias épocas**, puede indicar que el modelo está aprendiendo patrones demasiado específicos del conjunto de entrenamiento y que estos no se transfieren adecuadamente a datos no utilizados para actualizar sus parámetros.

Conceptualmente:

$$\text{Accuracy}_{\text{train}} \gg \text{Accuracy}_{\text{val}} \implies \text{posible sobreajuste}$$

Por el contrario, un gap cercano a cero indica que entrenamiento y validación presentan rendimientos similares:

$$\text{Gap}_{\text{acc}}^{(e)} \approx 0$$

Un gap negativo significa:

$$\text{Accuracy}_{\text{val}} > \text{Accuracy}_{\text{train}}$$

y no constituye, por sí mismo, evidencia de sobreajuste. En este proyecto esto puede ocurrir porque el conjunto de entrenamiento utiliza **Data Augmentation** y los modelos incorporan mecanismos de regularización como Dropout, mientras que Validation se evalúa sin esas perturbaciones aleatorias.

Umbral descriptivo para el análisis

No existe un valor universal que determine automáticamente cuándo un gap constituye sobreajuste. Para facilitar el análisis exploratorio de este experimento se utilizará únicamente como **señal de alerta descriptiva**:

$$\text{Gap}_{\text{pp}} \geq 10$$

es decir, una diferencia igual o superior a **10 puntos porcentuales** entre Train y Validation.

Este umbral **no representa una regla matemática universal**. El diagnóstico final deberá complementarse con:

- la evolución de loss;
- la evolución de val_loss;
- la persistencia del gap entre épocas;
- la mejor época seleccionada mediante val_loss;
- y posteriormente las métricas del conjunto Test.

En esta etapa el conjunto Test permanece completamente aislado.

12.2 Divergencia entre pérdida de entrenamiento y pérdida de validación

Además de comparar las exactitudes de entrenamiento y validación, una de las señales más importantes para detectar **sobreajuste** consiste en estudiar la evolución de las funciones de pérdida.

Para una época e , se dispone de:

$$L_{\text{train}}^{(e)}$$

que representa la pérdida calculada sobre el conjunto de entrenamiento, y:

$$L_{\text{val}}^{(e)}$$

que corresponde a la pérdida obtenida sobre el conjunto de validación.

Un modelo que aprende adecuadamente debería reducir inicialmente ambas pérdidas:

$$L_{\text{train}} \downarrow \quad \text{y} \quad L_{\text{val}} \downarrow$$

Sin embargo, cuando el modelo comienza a especializarse excesivamente en los datos de entrenamiento puede ocurrir que:

$$L_{\text{train}} \downarrow \quad \text{mientras} \quad L_{\text{val}} \uparrow$$

Este comportamiento constituye una **señal clásica de posible sobreajuste**.

Variación de la pérdida entre épocas consecutivas

Para evaluar este comportamiento de manera cuantitativa se define:

$$\Delta L_{\text{train}}^{(e)} = L_{\text{train}}^{(e)} - L_{\text{train}}^{(e-1)}$$

y:

$$\Delta L_{\text{val}}^{(e)} = L_{\text{val}}^{(e)} - L_{\text{val}}^{(e-1)}$$

Por lo tanto, existe una señal de divergencia compatible con sobreajuste cuando simultáneamente se cumple:

$$\Delta L_{\text{train}}^{(e)} < 0 \quad \wedge \quad \Delta L_{\text{val}}^{(e)} > 0$$

Es decir:

- la pérdida de entrenamiento disminuye;
- la pérdida de validación aumenta.

Brecha de pérdida

También puede analizarse la separación existente entre ambas pérdidas:

$$D_L^{(e)} = L_{\text{val}}^{(e)} - L_{\text{train}}^{(e)}$$

Si:

$$D_L^{(e)} > 0$$

la pérdida de validación es mayor que la pérdida de entrenamiento.

Un crecimiento progresivo de esta brecha puede indicar que el modelo continúa mejorando sobre Train, pero pierde capacidad de generalización sobre Validation.

Conceptualmente:

$$L_{\text{train}} \downarrow + L_{\text{val}} \uparrow + D_L \uparrow \implies \text{posible sobreajuste}$$

Criterio de interpretación

Una única época con este comportamiento no es suficiente para demostrar sobreajuste. La evidencia es más fuerte cuando la divergencia aparece de forma **persistente durante varias épocas consecutivas**, especialmente después de haber alcanzado el mínimo de pérdida de validación.

En este proyecto, la mejor época se define mediante:

$$e^* = \arg \min_e L_{\text{val}}^{(e)}$$

Por ello, se analizará especialmente lo que ocurre después de e^* .

El conjunto **Test no participa en este análisis**. Se mantiene reservado para la evaluación final después de cerrar las decisiones experimentales.

12.3 Pérdidas altas y bajo rendimiento: posible subajuste

El **subajuste** (underfitting) ocurre cuando un modelo no logra aprender adecuadamente los patrones existentes en los datos, incluso sobre el propio conjunto de entrenamiento.

A diferencia del sobreajuste, donde el modelo puede alcanzar un rendimiento muy elevado en Train y deteriorarse en Validation, en un escenario de subajuste se observa típicamente:

$$L_{\text{train}} \text{ alta} \quad \wedge \quad L_{\text{val}} \text{ alta}$$

acompañado de:

$$\text{Accuracy}_{\text{train}} \text{ baja} \quad \wedge \quad \text{Accuracy}_{\text{val}} \text{ baja}$$

Conceptualmente:

$$L_{\text{train}} \uparrow + L_{\text{val}} \uparrow + \text{Accuracy}_{\text{train}} \downarrow + \text{Accuracy}_{\text{val}} \downarrow \implies \text{posible subajuste}$$

Una característica importante es que la diferencia entre Train y Validation puede ser relativamente pequeña:

$$|\text{Accuracy}_{\text{train}} - \text{Accuracy}_{\text{val}}| \approx 0$$

pero esto **no implica necesariamente una buena generalización**.

Por ejemplo, dos accuracies igualmente bajas significan que el modelo presenta un rendimiento similar en ambos conjuntos, pero insuficiente para resolver correctamente el problema.

Referencia para clasificación multiclase

En este proyecto existen:

$$K = 10$$

clases.

Si un clasificador Softmax asignara inicialmente la misma probabilidad a todas las clases:

$$p_k = \frac{1}{K}$$

la pérdida de entropía cruzada asociada sería:

$$L_{\text{uniforme}} = -\log\left(\frac{1}{K}\right) = \log(K)$$

Para $K = 10$:

$$L_{\text{uniforme}} = \log(10) \approx 2.3026$$

y la accuracy asociada a una elección uniforme sería aproximadamente:

$$\text{Accuracy}_{\text{uniforme}} = \frac{1}{10} = 0.10$$

Estas cantidades constituyen únicamente **referencias descriptivas** y no umbrales universales de subajuste.

Debido al desbalance de PlantVillage, también resulta más informativo comparar el modelo con:

- la clase mayoritaria del dataset;
- el baseline convencional HOG + Regresión Logística;
- y la otra arquitectura entrenada bajo el mismo protocolo.

Diagnóstico

El subajuste no debe diagnosticarse utilizando únicamente un valor absoluto de loss, porque la magnitud de la función de pérdida depende del problema, del número de clases y de la función de coste utilizada.

La evidencia es más convincente cuando aparecen simultáneamente:

1. pérdida de entrenamiento relativamente alta;
2. pérdida de validación relativamente alta;
3. baja Accuracy en entrenamiento;
4. baja Accuracy en validación;
5. ausencia de una gran brecha Train-Validation;
6. rendimiento claramente inferior a referencias razonables.

En este proyecto el análisis se realizará utilizando exclusivamente los historiales de Train y Validation.

El conjunto **Test permanece aislado** hasta la evaluación final.

Interpretación: posible subajuste

El análisis de las curvas de Train y Validation permite distinguir entre **sobreajuste** y **subajuste**. La mejor época se define mediante el mínimo valor de `val_loss`.

MLP En la mejor época de la MLP:

$$e_{\text{MLP}}^* = 25,$$

se obtuvo:

$$L_{\text{train}} \approx 1.823611, \quad L_{\text{val}} \approx 1.622516,$$

junto con:

$$\text{Accuracy}_{\text{train}} \approx 0.3418, \quad \text{Accuracy}_{\text{val}} \approx 0.4603.$$

La brecha de Accuracy es:

$$\text{Gap}_{\text{MLP}} = 0.3418 - 0.4603 \approx -0.1186,$$

es decir, aproximadamente:

$$\boxed{-11.86 \text{ puntos porcentuales}}.$$

Por este criterio no existe el patrón clásico de sobreajuste, porque la Accuracy de Train no es muy superior a la de Validation. Sin embargo, el rendimiento de la MLP continúa siendo bajo en comparación con el baseline convencional HOG + Regresión Logística:

$$\text{Accuracy}_{\text{baseline, val}} \approx 68.52\%.$$

La diferencia respecto de la MLP en su checkpoint seleccionado es:

$$68.52\% - 46.03\% = \boxed{22.49 \text{ puntos porcentuales}}.$$

Esto constituye evidencia compatible con **subajuste o representación insuficientemente discriminativa** bajo la arquitectura y configuración evaluadas.

Una explicación arquitectónica plausible es que la MLP transforma:

$$128 \times 128 \times 3 \rightarrow 49\,152$$

valores mediante Flatten, por lo que la vecindad espacial deja de estar representada explícitamente.

CNN utilizada en la Fase 11 Para la CNN regularizada utilizada previamente, la mejor época fue:

$$e_{\text{CNN B}}^* = 2.$$

En ese checkpoint se obtuvo aproximadamente:

$$L_{\text{train}} = 0.486538, \quad L_{\text{val}} = 1.451495,$$

con:

$$\text{Accuracy}_{\text{train}} \approx 83.35\%, \quad \text{Accuracy}_{\text{val}} \approx 65.58\%.$$

La brecha es:

$$\text{Gap}_{\text{CNN B}} \approx +17.77 \text{ pp},$$

por lo que existe una señal de sobreajuste en el checkpoint seleccionado. Las curvas también muestran divergencia posterior entre Train y Validation.

Conclusión del diagnóstico

La MLP presenta un rendimiento global insuficiente y queda por debajo del baseline, mientras que la CNN B aprende patrones más discriminativos pero muestra una brecha Train-Validation considerable.

Este diagnóstico justifica la Fase 13, donde se comparan explícitamente dos configuraciones CNN para determinar cuál ofrece mejor generalización antes de abrir Test.

Fase 13 - Experimento de regularización

Realizar uno o dos experimentos acotados de regularización o hiperparámetros. Evitar búsquedas masivas.

13.1 Comparación de Accuracy de validación

El primer experimento de regularización tiene como objetivo determinar si la estrategia utilizada en la CNN definitiva mejora su capacidad de generalización sobre el conjunto de validación.

Se comparan dos configuraciones de la misma arquitectura convolucional:

Configuración	Data Augmentation	Dropout	EarlyStopping
CNN A	No	0.10	Sí
CNN B	Sí	0.30	Sí

La **CNN A** representa una configuración con regularización reducida, mientras que la **CNN B** corresponde al modelo utilizado previamente en el proyecto.

Se mantiene EarlyStopping en ambas configuraciones para conservar un **criterio común de selección del checkpoint** y evitar que una diferencia en el número de épocas actúe como una variable de confusión.

Por tanto, las principales diferencias experimentales son:

$$\text{CNN A} = \text{sin Data Augmentation} + \text{Dropout}(0.10)$$

frente a:

$$\text{CNN B} = \text{Data Augmentation} + \text{Dropout}(0.30)$$

El resto de las condiciones permanece controlado:

$$\text{Learning Rate} = 10^{-3}$$

$$\text{Batch Size} = 32$$

$$\text{Max Epochs} = 50$$

$$\text{Loss} = \text{SparseCategoricalCrossentropy}$$

y se mantiene:

$$SEED = 42$$

Accuracy de validación

Para un conjunto de validación formado por N_{val} observaciones, la Accuracy se define como:

$$\text{Accuracy}_{val} = \frac{\sum_{i=1}^{N_{val}} \mathbb{1}(\hat{y}_i = y_i)}{N_{val}}$$

donde:

- y_i corresponde a la clase real de la imagen i ;
- $\hat{y}_i$ corresponde a la clase predicha;
- $\mathbb{1}(\cdot)$ vale 1 cuando la predicción es correcta y 0 cuando es incorrecta.

En este proyecto:

$$N_{val} = 2722$$

Criterio común de selección

La comparación no utilizará arbitrariamente la época con mayor Accuracy.

De acuerdo con el protocolo definido previamente, el checkpoint de cada modelo se selecciona mediante la mínima pérdida de validación:

$$e^* = \arg \min_e L_{val}^{(e)}$$

Posteriormente se recupera la Accuracy correspondiente a esa misma época:

$$A_{val}^* = \text{Accuracy}_{val}^{(e^*)}$$

Este procedimiento mantiene consistente el criterio utilizado desde la Fase 11.

Diferencia entre configuraciones

La ganancia absoluta producida por la regularización se calculará como:

$$\Delta A_{val} = A_{val}^{(B)} - A_{val}^{(A)}$$

y en puntos porcentuales:

$$\Delta A_{pp} = 100 \left(A_{val}^{(B)} - A_{val}^{(A)} \right)$$

Si:

$$\Delta A_{pp} > 0$$

la CNN B presenta una mejor Accuracy de validación.

Si:

$$\Delta A_{pp} < 0$$

la CNN A presenta una mayor Accuracy de validación.

No obstante, una diferencia de Accuracy por sí sola no será suficiente para concluir qué configuración generaliza mejor. Posteriormente deberán compararse también:

- val_loss;
- la brecha Train-Validation;
- y el comportamiento de las curvas de aprendizaje.

El conjunto **Test permanece completamente aislado** durante toda la Fase 13.

Resultado ejecutado

La comparación real produjo:

$$\text{Accuracy}_{val,A} = 96.14\%, \quad \text{Accuracy}_{val,B} = 65.58\%.$$

Por tanto:

$$\Delta A_{val} = 65.58 - 96.14 = -30.57 \text{ pp},$$

resultado favorable a **CNN A**.

13.2 Comparación de Loss de validación

Además de comparar la Accuracy de validación, es necesario analizar la **función de pérdida sobre Validation**, ya que esta incorpora no solo si la predicción final fue correcta o incorrecta, sino también la confianza con que el modelo asignó probabilidades a las distintas clases.

En este proyecto se utiliza:

SparseCategoricalCrossentropy

como función de pérdida para la clasificación multiclase.

Para un conjunto de validación formado por N_{val} imágenes y K clases, la pérdida puede expresarse conceptualmente como:

$$L_{val} = -\frac{1}{N_{val}} \sum_{i=1}^{N_{val}} \log(p_{i,y_i})$$

donde:

- N_{val} es el número de imágenes del conjunto de validación;
- y_i representa la clase verdadera de la imagen i ;
- p_{i,y_i} corresponde a la probabilidad que el modelo asigna a la clase correcta.

Una pérdida menor indica que el modelo asigna, en promedio, una probabilidad más elevada a las clases correctas:

$$L_{val} \downarrow \implies \text{mejor ajuste sobre Validation}$$

Selección del mejor checkpoint

Para mantener el mismo protocolo utilizado en las fases anteriores, la mejor época de cada configuración se define mediante:

$$e^* = \arg \min_e L_{val}^{(e)}$$

Por tanto:

$$L_{val}^{*(A)} = \min_e L_{val,A}^{(e)}$$

para la CNN A, y:

$$L_{val}^{*(B)} = \min_e L_{val,B}^{(e)}$$

para la CNN B.

La diferencia entre ambas configuraciones se calculará como:

$$\Delta L_{val} = L_{val}^{*(B)} - L_{val}^{*(A)}$$

Si:

$$\Delta L_{val} < 0$$

entonces la CNN B presenta una pérdida de validación menor.

Si:

$$\Delta L_{val} > 0$$

la CNN A presenta una pérdida de validación menor.

También se calculará la reducción relativa de pérdida de CNN B respecto de CNN A:

$$R_L = \frac{L_{val}^{*(A)} - L_{val}^{*(B)}}{L_{val}^{*(A)}} \times 100$$

Un valor positivo de R_L significa que CNN B redujo la pérdida de validación respecto de CNN A.

Interpretación metodológica

La comparación de `val_loss` complementa a `val_accuracy`.

Dos modelos pueden presentar Accuracy similares, pero pérdidas diferentes. Esto ocurre porque la Accuracy solo evalúa si la clase final fue correcta, mientras que la Cross-Entropy considera también la distribución de probabilidades predicha.

Por esta razón:

$$\text{menor } L_{val} + \text{Accuracy competitiva}$$

constituye una señal favorable de generalización.

No obstante, la decisión final de la Fase 13 requerirá todavía analizar la brecha entre entrenamiento y validación.

El conjunto **Test permanece completamente aislado** y no participa en esta comparación.

Resultado ejecutado

$$L_{val,A} = 0.150224, \quad L_{val,B} = 1.451495.$$

Por tanto:

$$\Delta L_{val} = 1.451495 - 0.150224 = 1.301271 > 0,$$

resultado favorable a **CNN A**.

13.3 Comparación de la brecha Train-Validation

La comparación entre Accuracy de entrenamiento y Accuracy de validación permite estudiar cuánto difiere el rendimiento del modelo entre los datos utilizados para aprender y los datos no utilizados para actualizar sus pesos.

Para una configuración m , se define la **brecha de generalización** como:

$$G_m = \text{Accuracy}_{train,m} - \text{Accuracy}_{val,m}$$

En puntos porcentuales:

$$G_{m,pp} = 100 \left(\text{Accuracy}_{train,m} - \text{Accuracy}_{val,m} \right)$$

Si:

$$G_m > 0$$

el modelo obtiene mayor Accuracy en Train que en Validation.

Una brecha positiva grande puede indicar que el modelo se ha especializado excesivamente en los datos de entrenamiento:

$$\text{Accuracy}_{train} \gg \text{Accuracy}_{val} \implies \text{posible sobreajuste}$$

Por el contrario:

$$G_m \approx 0$$

indica rendimientos similares entre ambos conjuntos.

Comparación justa entre CNN A y CNN B

Existe una consideración metodológica importante en este experimento.

La CNN A utiliza:

Train normalizado + Dropout(0.10)

mientras que la CNN B utiliza:

Data Augmentation + Dropout(0.30)

Durante `model.fit()`, la Accuracy de entrenamiento de CNN B se calcula sobre imágenes modificadas mediante Data Augmentation y con Dropout activo.

Por tanto, comparar directamente:

$$\text{Accuracy}_{train,A}^{fit}$$

con:

$$\text{Accuracy}_{train,B}^{fit}$$

puede introducir una diferencia derivada del propio proceso de regularización.

Para realizar una comparación más justa, ambos modelos serán evaluados, después del entrenamiento, sobre el **mismo conjunto Train sin Data Augmentation** y sobre el mismo conjunto Validation.

La evaluación se realiza en modo inferencia, por lo que Dropout permanece desactivado.

De esta manera:

$$G_A^{eval} = Accuracy_{train,A}^{eval} - Accuracy_{val,A}^{eval}$$

y:

$$G_B^{eval} = Accuracy_{train,B}^{eval} - Accuracy_{val,B}^{eval}$$

son directamente comparables.

Magnitud de la brecha

También se estudiará la magnitud absoluta:

$$|G_m| = |Accuracy_{train,m} - Accuracy_{val,m}|$$

La reducción de la brecha producida por CNN B respecto de CNN A se define como:

$$\Delta G_{abs} = |G_A| - |G_B|$$

Si:

$$\Delta G_{abs} > 0$$

entonces CNN B presenta una menor separación entre Train y Validation.

Si:

$$\Delta G_{abs} < 0$$

CNN B presenta una brecha mayor.

En puntos porcentuales:

$$\Delta G_{abs,pp} = 100 (|G_A| - |G_B|)$$

Interpretación

Una brecha menor no constituye, por sí sola, evidencia suficiente de un mejor modelo.

Por ejemplo, un modelo con:

$$Accuracy_{train} \approx Accuracy_{val} \approx 0.30$$

presentaría una brecha pequeña, pero un rendimiento global insuficiente.

Por esta razón, la interpretación de la regularización debe considerar conjuntamente:

$$Accuracy_{val} + Loss_{val} + Gap_{train-val}$$

La regularización será considerada favorable si logra mantener o mejorar el rendimiento de Validation y, al mismo tiempo, controlar la separación entre Train y Validation.

El conjunto **Test permanece completamente aislado** durante este experimento.

Resultado ejecutado

Evaluando ambas configuraciones sobre los mismos conjuntos:

$$|G_A| = 0.55 \text{ pp}, \quad |G_B| = 0.56 \text{ pp}.$$

La diferencia es mínima, pero por este criterio CNN A presenta la menor magnitud de brecha.

13.4 Determinar si la regularización mejora la generalización

El objetivo final del experimento de regularización no consiste en maximizar exclusivamente la Accuracy de entrenamiento, sino en determinar qué configuración presenta una mejor capacidad para responder correctamente ante datos que no fueron utilizados para actualizar sus parámetros.

En este experimento se comparan:

$$\text{CNN A} = \text{sin Data Augmentation} + \text{Dropout}(0.10) + \text{EarlyStopping}$$

frente a:

$$\text{CNN B} = \text{Data Augmentation} + \text{Dropout}(0.30) + \text{EarlyStopping}$$

La CNN B incorpora una regularización más intensa.

¿Qué significa generalizar?

Un modelo presenta buena **generalización** cuando mantiene un rendimiento elevado sobre datos no utilizados para ajustar directamente sus pesos.

Por tanto, no resulta suficiente observar:

$$Accuracy_{train}$$

de forma aislada.

Un modelo podría alcanzar:

$$Accuracy_{train} \approx 1$$

pero presentar una disminución importante sobre Validation.

Este comportamiento sería compatible con sobreajuste.

En cambio, un modelo regularizado puede presentar una Accuracy de entrenamiento ligeramente inferior:

$$Accuracy_{train,B} < Accuracy_{train,A}$$

y, sin embargo, generalizar mejor si obtiene:

$$\text{Accuracy}_{val,B} > \text{Accuracy}_{val,A}.$$

Por esta razón:

mayor Accuracy de Train $\neq$ mejor generalización

Criterios utilizados

La evaluación de la regularización integrará tres indicadores principales.

1. Accuracy de validación Se define:

$$\Delta A_{val} = A_{val,B} - A_{val,A}.$$

Si:

$$\Delta A_{val} > 0,$$

CNN B presenta mayor Accuracy de validación.

2. Loss de validación Se define:

$$\Delta L_{val} = L_{val,B} - L_{val,A}.$$

Como una pérdida menor es preferible:

$$\Delta L_{val} < 0$$

constituye evidencia favorable para CNN B.

3. Brecha Train-Validation Para cada configuración:

$$G_A = A_{train,A} - A_{val,A}$$

y:

$$G_B = A_{train,B} - A_{val,B}.$$

Para comparar la magnitud de la separación se utilizará:

$$|G_A| \quad \text{y} \quad |G_B|.$$

La reducción de la brecha se define como:

$$\Delta G = |G_A| - |G_B|.$$

Por tanto:

$$\Delta G > 0$$

significa que CNN B presenta una menor separación entre Train y Validation.

Evidencia conjunta

Una situación especialmente favorable a la regularización se produciría si:

$$A_{val,B} > A_{val,A}$$

simultáneamente:

$$L_{val,B} < L_{val,A}$$

y además:

$$|G_B| < |G_A|.$$

En ese escenario, CNN B:

1. clasifica correctamente una mayor proporción de Validation;
2. presenta menor error probabilístico;
3. mantiene una menor separación entre Train y Validation.

Estas tres evidencias serían consistentes con una mejora de generalización.

El papel de la Accuracy de entrenamiento

La Accuracy de Train se utilizará como información diagnóstica, pero no como criterio principal para seleccionar el modelo.

Puede ocurrir que:

$$A_{train,B} < A_{train,A}$$

debido a la acción de mecanismos como:

- Data Augmentation;
- Dropout;
- EarlyStopping.

Esto no representa necesariamente un deterioro.

Si simultáneamente:

$$A_{val,B} \geq A_{val,A},$$

$$L_{val,B} \leq L_{val,A},$$

y:

$$|G_B| < |G_A|,$$

una menor Accuracy de entrenamiento puede interpretarse como el costo esperado de una regularización que evita una especialización excesiva en Train.

Conceptualmente:

$$\text{menor rendimiento en Train} + \text{mejor rendimiento fuera de Train} \Rightarrow \text{regularización efectiva}$$

Alcance de la conclusión

La conclusión de esta fase corresponde exclusivamente a la selección de hiperparámetros utilizando Train y Validation.

El conjunto Test todavía no participa.

Por tanto, incluso si CNN B resulta superior, la afirmación correcta será:

CNN B presenta mejor comportamiento de generalización sobre Validation.

La capacidad de generalización final solo deberá estimarse posteriormente mediante el conjunto Test, una vez cerradas todas las decisiones del modelo.

Resultado ejecutado

Los tres criterios definidos antes de Test favorecieron a CNN A:

$$Accuracy_{val,A} > Accuracy_{val,B},$$

$$L_{val,A} < L_{val,B},$$

$$|G_A| < |G_B|.$$

Por tanto, la configuración seleccionada para la evaluación final fue:

CNN A.

13.4 Interpretación final del experimento de regularización

El objetivo de la Fase 13 fue comparar dos configuraciones de la misma familia convolucional sin utilizar el conjunto Test:

Configuración	Características principales
CNN A	Sin Data Augmentation + Dropout 0.10 + EarlyStopping
CNN B	Data Augmentation + Dropout 0.30 + EarlyStopping

La comparación se realizó exclusivamente con **Train y Validation**.

Resultados reales obtenidos

Métrica	CNN A	CNN B	Diferencia B - A
Train Accuracy	96.69 %	65.02 %	-31.67 pp
Validation Accuracy	96.14 %	65.58 %	-30.57 pp
Validation Loss	0.150224	1.451495	+1.301271
$ Gap_{Train-Val} $	0.55 pp	0.56 pp	+0.01 pp

Los tres criterios definidos antes de Test favorecen a **CNN A**:

1. mayor Accuracy de Validation;
2. menor val_loss;
3. menor magnitud de la brecha Train-Validation.

Por tanto:

$$\text{criterios favorables a CNN B} = 0/3$$

y la configuración seleccionada es:

$$\text{CNN A}$$

La regularización más intensa de CNN B no produjo una mejora en esta ejecución. Esto no implica que Data Augmentation o Dropout sean perjudiciales de forma general; significa únicamente que, **bajo esta arquitectura, semilla, hiperparámetros y protocolo**, CNN A obtuvo mejor comportamiento sobre Validation.

La selección final se realizó antes de observar Test:

$$(Train, Validation) \rightarrow \text{selección de CNN A} \rightarrow Test$$

Conclusión de la Fase 13: CNN A es la configuración convolucional que continúa hacia la evaluación final.

Fase 14 - Evaluación final con TEST

Utilizar el conjunto test únicamente después de finalizar las decisiones de arquitectura e hiperparámetros.

Fase 14 — Evaluación final con TEST

14.1 Apertura controlada del conjunto Test

Test constituye la última partición del experimento y no participó en:

- entrenamiento;
- selección de la mejor época;
- selección de arquitectura;
- comparación de regularización;
- ajuste de hiperparámetros.

La partición final es:

$$70\% \text{ Train} \quad 15\% \text{ Validation} \quad 15\% \text{ Test}$$

con:

$$N_{Train} = 12\,702, \quad N_{Validation} = 2\,722, \quad N_{Test} = 2\,722.$$

Además:

$$Train \cap Validation = \emptyset,$$

$$Train \cap Test = \emptyset,$$

$$Validation \cap Test = \emptyset.$$

14.2 Configuración congelada antes de Test

La Fase 13 produjo:

Métrica	CNN A	CNN B
Validation Accuracy	96.14 %	65.58 %
Validation Loss	0.150224	1.451495
$ Gap_{Train-Val} $	0.55 pp	0.56 pp

Por tanto:

CNN A

fue seleccionada utilizando exclusivamente Train y Validation.

La regla metodológica aplicada fue:

$Train \rightarrow Validation \rightarrow \text{selección definitiva} \rightarrow Test$

14.3 Métricas finales sobre Test

Las predicciones almacenadas para las 2 722 observaciones de Test produjeron:

Métrica	MLP	CNN A	Diferencia CNN-MLP
Accuracy	46.07 %	95.81 %	+49.74 pp
Balanced Accuracy	27.90 %	94.11 %	+66.21 pp
Precision Macro	28.08 %	95.51 %	+67.44 pp
Recall Macro	27.90 %	94.11 %	+66.21 pp
F1 Macro	20.20 %	94.41 %	+74.21 pp

El promedio macro asigna el mismo peso a todas las clases, por lo que es especialmente informativo ante el desbalance del dataset.

14.4 Regla posterior a Test

Una vez abierto Test:

NO se modifican arquitectura, pesos ni hiperparámetros

Las Fases 15–20 reutilizan exactamente las predicciones generadas en esta fase.

Fase 15 - Matriz de confusión

Construir una matriz M_{ij} con filas como clases reales y columnas como clases predichas. Debe utilizarse para identificar qué enfermedades se confunden entre sí.

Fase 15 — Matriz de confusión

15.1 Matrices de confusión para MLP y CNN

Las métricas globales, como Accuracy o F1, permiten resumir el rendimiento general de un clasificador, pero no muestran **qué clases específicas son confundidas entre sí**.

Para estudiar este comportamiento se utiliza la **matriz de confusión**.

En un problema de clasificación multiclase con K clases, la matriz de confusión se define como:

$$M \in \mathbb{N}^{K \times K}$$

donde cada elemento M_{ij} representa el número de observaciones cuya clase real es i y que fueron predichas como clase j .

Formalmente:

$$M_{ij} = \sum_{n=1}^{N_{test}} \mathbb{1}(y_n = i \wedge \hat{y}_n = j)$$

donde:

- N_{test} es el número total de imágenes del conjunto Test;
- y_n representa la clase real de la observación n ;
- $\hat{y}_n$ representa la clase predicha;
- $\mathbb{1}(\cdot)$ es la función indicadora.

Organización de la matriz

En este proyecto se utilizará la convención:

Filas = clases reales

Columnas = clases predichas

Por tanto:

- los elementos de la **diagonal principal** corresponden a predicciones correctas;
- los elementos fuera de la diagonal representan errores de clasificación.

Conceptualmente:

$$M = \begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1K} \\ M_{21} & M_{22} & \cdots & M_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ M_{K1} & M_{K2} & \cdots & M_{KK} \end{bmatrix}.$$

La suma de todos los elementos debe coincidir con el tamaño de Test:

$$\sum_{i=1}^K \sum_{j=1}^K M_{ij} = N_{test}$$

En este proyecto:

$$N_{test} = 2722$$

Matriz normalizada

Debido a que PlantVillage presenta desbalance entre clases, también resulta útil representar una versión normalizada por filas.

La matriz normalizada se define como:

$$\widetilde{M}_{ij} = \frac{M_{ij}}{\sum_{j=1}^K M_{ij}}$$

De esta manera, cada fila suma aproximadamente:

$$\sum_{j=1}^K \widetilde{M}_{ij} = 1$$

y cada valor puede interpretarse como la proporción de ejemplos de una clase real que fueron asignados a cada clase predicha.

En particular, los valores de la diagonal:

$$\widetilde{M}_{ii}$$

representan la proporción correctamente recuperada para cada clase y están directamente relacionados con el **Recall por clase**.

Protocolo utilizado

Las matrices se construirán reutilizando las predicciones almacenadas durante la evaluación final:

$$y_{test}$$

$$\hat{y}_{MLP}$$

y:

$$\hat{y}_{CNN}.$$

Por tanto:

$$M^{MLP} = CM(y_{test}, \hat{y}_{MLP})$$

y:

$$M^{CNN} = CM(y_{test}, \hat{y}_{CNN}).$$

No se volverán a entrenar los modelos ni se generarán nuevas predicciones sobre Test.

Esto garantiza que las matrices de confusión, las métricas de la Fase 14 y el posterior análisis de errores correspondan exactamente al mismo experimento final.

15.2 Análisis de errores frecuentes

Después de construir las matrices de confusión de MLP y CNN, el siguiente paso consiste en estudiar los errores que aparecen con mayor frecuencia.

En una matriz de confusión:

$$M_{ij}$$

representa el número de observaciones cuya clase real es i y que fueron predichas como clase j . Los elementos de la diagonal principal:

$$M_{ii}$$

corresponden a clasificaciones correctas.

Por tanto, un error ocurre cuando:

$$i \neq j$$

y puede representarse como:

$$E_{ij} = M_{ij}, \quad i \neq j$$

Número de errores por clase real

Para una clase real i , el número total de observaciones es:

$$N_i = \sum_{j=1}^K M_{ij}$$

El número de predicciones incorrectas para esa clase corresponde a:

$$E_i = \sum_{\substack{j=1 \\ j \neq i}}^K M_{ij}$$

o, equivalentemente:

$$E_i = N_i - M_{ii}$$

donde M_{ii} representa el número de observaciones correctamente clasificadas de la clase i .

Tasa de error por clase

Debido a que PlantVillage presenta desbalance entre clases, no resulta suficiente comparar únicamente la cantidad absoluta de errores.

Por ello se calcula también la **tasa de error por clase**:

$$R_i = \frac{E_i}{N_i}$$

que también puede expresarse como:

$$R_i = 1 - \frac{M_{ii}}{N_i}$$

Como:

$$Recall_i = \frac{M_{ii}}{N_i},$$

entonces:

$$R_i = 1 - Recall_i$$

Por tanto, una clase con una tasa de error elevada corresponde a una clase que el modelo recupera con mayor dificultad.

Errores direccionales más frecuentes

También se estudiarán las confusiones específicas:

$$i \rightarrow j$$

donde:

- i es la clase real;
- j es la clase predicha incorrectamente.

La frecuencia absoluta de una confusión será:

$$F_{i \rightarrow j} = M_{ij}$$

mientras que su frecuencia relativa dentro de la clase real será:

$$P_{i \rightarrow j} = \frac{M_{ij}}{N_i}$$

para:

$$i \neq j.$$

Este segundo indicador es especialmente importante ante desbalance de clases.

Objetivo del análisis

Para cada arquitectura se identificarán:

1. el número total de errores;
2. la tasa global de error;
3. las clases reales con mayor número de errores;
4. las clases con mayor tasa proporcional de error;
5. las confusiones direccionales más frecuentes.

El análisis se realizará por separado para:

MLP

y:

CNN seleccionada

utilizando exclusivamente las matrices generadas a partir del conjunto Test.

No se generarán nuevas predicciones ni se modificarán los modelos.

15.3 Identificación de pares de clases con confusión sistemática

El análisis anterior permitió identificar errores direccionales frecuentes, es decir, situaciones de la forma:

$$C_i \rightarrow C_j$$

donde una imagen perteneciente realmente a la clase C_i fue clasificada incorrectamente como C_j .

Sin embargo, para estudiar si **dos clases se confunden sistemáticamente entre sí**, es necesario analizar conjuntamente las dos direcciones:

$$C_i \rightarrow C_j$$

y:

$$C_j \rightarrow C_i.$$

En la matriz de confusión:

$$M_{ij}$$

representa el número de imágenes de la clase real i clasificadas como j , mientras que:

$$M_{ji}$$

representa el error inverso.

Por tanto, para cada par no ordenado de clases:

$$\{C_i, C_j\}, \quad i < j$$

se analizarán simultáneamente ambos términos.

Confusión bidireccional absoluta

Se define el número total de errores intercambiados entre dos clases como:

$$B_{ij} = M_{ij} + M_{ji}$$

Un valor elevado de B_{ij} indica que ambas clases concentran una cantidad importante de errores entre sí.

Sin embargo, esta cantidad puede estar influida por el tamaño de las clases.

Confusión direccional normalizada

Sea:

$$N_i = \sum_{k=1}^K M_{ik}$$

el número real de observaciones pertenecientes a la clase i .

La proporción de imágenes reales de i confundidas con j se define como:

$$P_{i \rightarrow j} = \frac{M_{ij}}{N_i}$$

y, en la dirección contraria:

$$P_{j \rightarrow i} = \frac{M_{ji}}{N_j}$$

Estas proporciones permiten controlar el efecto producido por el desbalance entre clases.

Índice simétrico de confusión

Para representar mediante un único valor la intensidad relativa de la confusión entre dos clases se utilizará:

$$S_{ij} = \frac{1}{2} (P_{i \rightarrow j} + P_{j \rightarrow i})$$

donde:

$$0 \leq S_{ij} \leq 1.$$

Un valor elevado de S_{ij} indica que, proporcionalmente, ambas clases presentan una separación difícil para el modelo.

En porcentaje:

$$S_{ij}^{(\%)} = 100S_{ij}$$

Grado de bidireccionalidad

También resulta útil determinar si la confusión ocurre de manera equilibrada en ambas direcciones.

Se define:

$$Q_{ij} = \frac{\min(P_{i \rightarrow j}, P_{j \rightarrow i})}{\max(P_{i \rightarrow j}, P_{j \rightarrow i})}$$

cuando al menos una de las dos proporciones es distinta de cero.

La interpretación es:

$$Q_{ij} \approx 1$$

cuando ambas direcciones presentan intensidades similares, mientras que:

$$Q_{ij} \approx 0$$

indica que la confusión ocurre fundamentalmente en una sola dirección.

Criterio utilizado en este proyecto

No existe un umbral universal que permita afirmar automáticamente que una confusión es sistemática.

Por ello, el análisis utilizará conjuntamente:

$$B_{ij}$$

cantidad absoluta de errores,

$$S_{ij}$$

intensidad relativa de la confusión,

y:

$$Q_{ij}$$

grado de bidireccionalidad.

Se considerarán especialmente relevantes aquellos pares que:

- presentan errores en ambas direcciones;
- aparecen entre los mayores valores de B_{ij} ;
- mantienen valores elevados de S_{ij} ;
- y presentan una bidireccionalidad apreciable.

El análisis se realizará independientemente para MLP y CNN utilizando las matrices de confusión obtenidas previamente sobre el mismo conjunto Test.

No se generarán nuevas predicciones y no se volverá a consultar directamente el conjunto Test.

Interpretación de los pares con confusión sistemática

El análisis de pares permitió extender la interpretación de la matriz de confusión desde errores individuales:

$$C_i \rightarrow C_j$$

hacia confusiones bidireccionales:

$$C_i \leftrightarrow C_j.$$

Para la **MLP**, el par con mayor score simétrico correspondió a:

$$[\text{Clase A}] \leftrightarrow [\text{Clase B}]$$

Se observaron:

$$M_{AB} = [\text{valor}]$$

errores desde la clase A hacia B y:

$$M_{BA} = [\text{valor}]$$

errores en la dirección contraria.

Por tanto, la confusión bidireccional total fue:

$$B_{AB} = M_{AB} + M_{BA} = [\text{valor}]$$

con un score simétrico de:

$$S_{AB} = [\text{valor}] \%$$

y un índice de bidireccionalidad de:

$$Q_{AB} = [\text{valor}].$$

Para la **CNN**, el par con mayor confusión bidireccional correspondió a:

$$[\text{Clase A}] \leftrightarrow [\text{Clase B}]$$

con:

$$B_{AB} = [\text{valor}]$$

errores intercambiados y:

$$S_{AB} = [\text{valor}] \%.$$

La comparación MLP-CNN permite además distinguir entre dos tipos de confusiones.

En algunos pares:

$$S_{ij}^{CNN} < S_{ij}^{MLP},$$

lo que indica que la arquitectura convolucional logra separar mejor esas clases.

En otros pares, la confusión persiste incluso en la CNN:

$$S_{ij}^{CNN} > 0 \quad \text{y} \quad S_{ij}^{MLP} > 0.$$

Estos pares persistentes son especialmente relevantes porque sugieren que la dificultad podría no depender únicamente de la arquitectura, sino también de la semejanza visual entre las categorías.

Por tanto, los pares con mayor score simétrico y las confusiones que persisten en ambos modelos constituyen los candidatos prioritarios para el siguiente análisis.

El próximo paso consistirá en relacionar estas confusiones con características visuales observadas durante el EDA, especialmente:

color + textura + patrones de lesiones + forma de la hoja

para determinar si los errores encontrados son visualmente razonables o si revelan una limitación estructural de alguno de los modelos.

15.4 Relación entre errores de clasificación y similitudes visuales observadas en el EDA

La matriz de confusión permite determinar **qué clases son confundidas por los modelos**, pero no explica por sí sola por qué aparecen esas confusiones.

Para interpretar los errores es necesario volver al **Análisis Exploratorio de Datos (EDA)** y contrastar los pares problemáticos con las características visuales de las imágenes.

Durante el EDA se analizaron varias imágenes por categoría con el objetivo de observar:

- variabilidad dentro de una misma clase;
- similitud entre clases diferentes;
- color;
- textura;
- distribución de manchas;
- patrones de lesión;
- bordes;
- forma general de las hojas.

Estas características constituyen precisamente la información visual que los modelos deben aprender para realizar la clasificación.

Conceptualmente:

Píxeles → bordes → texturas → formas → patrones de lesión → clase

Hipótesis de interpretación

Si dos clases presentan características visuales similares, puede aumentar la probabilidad de que el modelo las confunda:

alta similitud visual $\Rightarrow$ mayor dificultad de separación

Sin embargo, esta relación debe interpretarse como una **hipótesis exploratoria** y no como una relación causal demostrada.

Una confusión también puede estar influida por:

- variabilidad dentro de una misma clase;
- diferencias de iluminación;
- orientación de las hojas;
- escala de la imagen;
- intensidad de las lesiones;
- resolución utilizada;
- desbalance de clases;
- capacidad de representación del modelo.

Características visuales consideradas

Para complementar la inspección cualitativa del EDA se calcularán algunos descriptores simples.

Color medio

Para una imagen con N píxeles se define:

$$\bar{R} = \frac{1}{N} \sum_{p=1}^N R_p$$

$$\bar{G} = \frac{1}{N} \sum_{p=1}^N G_p$$

$$\bar{B} = \frac{1}{N} \sum_{p=1}^N B_p$$

Estos valores proporcionan una descripción global aproximada de la distribución de color de la imagen.

Luminosidad

La luminosidad aproximada de cada píxel se calculará mediante:

$$L = 0.2126R + 0.7152G + 0.0722B$$

y posteriormente:

$$\bar{L} = \frac{1}{N} \sum_{p=1}^N L_p$$

Este indicador permite detectar si dos clases presentan niveles globales de iluminación similares.

Contraste

El contraste global se aproximará mediante la desviación estándar de la luminosidad:

$$C = \sigma_L$$

Valores similares de C indican distribuciones globales de intensidad comparables.

Textura aproximada

Como descriptor exploratorio de textura se utilizará el gradiente espacial de la intensidad:

$$G(x, y) = \sqrt{\left(\frac{\partial L}{\partial x}\right)^2 + \left(\frac{\partial L}{\partial y}\right)^2}$$

y su promedio:

$$\bar{G} = \frac{1}{N} \sum_{p=1}^N G_p$$

Un valor elevado indica una mayor presencia de cambios locales de intensidad, como bordes, manchas o estructuras de textura.

Este descriptor no sustituye las características aprendidas por una CNN; se utiliza solamente para apoyar cuantitativamente la interpretación visual del EDA.

Relación con la matriz de confusión

Para cada par problemático:

$$C_i \leftrightarrow C_j$$

se compararán:

1. el número de errores intercambiados;
2. el score simétrico de confusión;
3. imágenes representativas de ambas clases;
4. color medio;
5. luminosidad;
6. contraste;
7. saturación;
8. textura aproximada.

Cuando un par presente simultáneamente:

confusión elevada + apariencia visual similar

el error podrá interpretarse como razonable desde el punto de vista visual.

Por el contrario, si las clases son visualmente muy distintas pero el modelo las confunde con frecuencia, el error podría revelar una limitación más estructural de la representación aprendida.

Relación con MLP y CNN

Este análisis es especialmente importante para comparar ambas arquitecturas.

La MLP recibe:

$$128 \times 128 \times 3 \rightarrow 49\,152$$

valores aplanados, por lo que las relaciones espaciales entre píxeles no quedan representadas explícitamente.

La CNN, en cambio, conserva la organización espacial y aprende características locales mediante convoluciones.

Por tanto, si:

$$S_{ij}^{CNN} < S_{ij}^{MLP}$$

para clases visualmente similares, existe evidencia de que la representación espacial de la CNN contribuye a diferenciarlas.

Si la confusión persiste incluso en la CNN:

$$S_{ij}^{CNN} > 0$$

y además el EDA muestra similitudes claras entre las clases, entonces la dificultad puede estar asociada parcialmente a una similitud visual real del problema.

El análisis se realizará sin generar nuevas predicciones sobre Test.

15.4 Interpretación de la relación entre EDA y matriz de confusión

El análisis conjunto del EDA y de las matrices de confusión permite estudiar si los errores se concentran entre clases con características visuales semejantes.

En la ejecución actual, el par persistente en ambos modelos es:

- Tomato__Spider_mites Two-spotted_spider_mite;
- Tomato__Target_Spot.

Sus descriptores medios fueron:

Característica	Spider mites	Target Spot
R medio	0.4761	0.4713
G medio	0.4654	0.4732
B medio	0.4209	0.4398
Luminosidad	0.4644	0.4704
Contraste	0.1561	0.1592
Saturación	0.2221	0.2006
Textura	0.0430	0.0482

Las diferencias visuales agregadas son pequeñas:

$$D_{RGB} = 0.0209,$$

$$\Delta L = 0.0060, \quad \Delta C = 0.0032,$$

$$\Delta S = 0.0214, \quad \Delta T = 0.0052.$$

La confusión bidireccional asociada al par fue:

Modelo	Errores del par	Score simétrico
MLP	17	3.42 %
CNN A	10	2.22 %

Por tanto, la CNN reduce el score de confusión en aproximadamente:

$$3.42 - 2.22 = \boxed{1.21 \text{ pp}}.$$

La persistencia de cierta confusión es compatible con la similitud visual del par. Sin embargo, la CNN reduce la frecuencia del error, lo que es consistente con una mejor explotación de textura, localidad y estructura espacial.

Esta interpretación es descriptiva y no demuestra causalidad entre una característica visual aislada y el error.

Fase 16 - Análisis de errores

Complementar las métricas globales con ejemplos concretos de predicciones correctas e incorrectas.

Fase 16 — Análisis de errores

16.1 Similitud visual entre clases

El análisis de errores complementa las métricas globales mediante la inspección de clases visualmente próximas y predicciones concretas.

Las características visuales pueden representarse mediante un vector:

$$\mathbf{z}_i = [z_{i1}, z_{i2}, \dots, z_{ip}],$$

y una distancia entre clases puede expresarse como:

$$D_{ij} = \sqrt{\sum_{k=1}^p (z_{ik} - z_{jk})^2}.$$

Una transformación exploratoria de similitud es:

$$S_{ij}^{visual} = \frac{1}{1 + D_{ij}},$$

donde valores mayores indican mayor proximidad bajo los descriptores seleccionados.

En la ejecución actual, el par Spider_mites Two-spotted_spider_mite-Target_Spot presenta:

$$D_{RGB} = 0.0209, \quad \Delta L = 0.0060,$$

$$\Delta C = 0.0032, \quad \Delta S = 0.0214, \quad \Delta T = 0.0052.$$

El score de confusión se reduce de:

$$3.42\% \text{ en MLP}$$

a:

$$2.22\% \text{ en CNN A.}$$

La diferencia es:

$$1.21 \text{ puntos porcentuales}.$$

Esto sugiere que la similitud visual puede explicar parte de los errores persistentes, mientras que la CNN logra reducir su frecuencia.

La conclusión correcta es que existe **evidencia compatible** con una relación entre similitud visual y confusión; no que dicha similitud sea la única causa de los errores.

16.2 Revisión de iluminación, textura, color y calidad visual

Después de identificar las clases visualmente similares y los principales errores de clasificación, se analiza si determinadas propiedades de las imágenes pueden contribuir a explicar el comportamiento de los modelos.

Este análisis no pretende demostrar causalidad. Su objetivo es estudiar si los errores aparecen asociados a características visuales como:

$$\text{iluminación} + \text{contraste} + \text{color} + \text{saturación} + \text{textura} + \text{nitidez}$$

La comparación se realizará entre imágenes:

- correctamente clasificadas;
- incorrectamente clasificadas por la MLP;
- incorrectamente clasificadas por la CNN;
- confundidas por ambos modelos;
- incorrectas para la MLP pero correctas para la CNN.

De esta manera puede estudiarse si las condiciones visuales de las imágenes problemáticas difieren sistemáticamente de las imágenes correctamente clasificadas.

1. Iluminación

Una aproximación estándar a la luminancia de un píxel RGB normalizado se calcula mediante:

$$L = 0.2126R + 0.7152G + 0.0722B$$

La iluminación media de una imagen con N píxeles se define como:

$$\bar{L} = \frac{1}{N} \sum_{p=1}^N L_p$$

Valores pequeños de:

$$\bar{L}$$

indican imágenes globalmente más oscuras, mientras que valores elevados representan imágenes más luminosas.

Sin embargo, una imagen muy clara o muy oscura no debe considerarse automáticamente de mala calidad. La interpretación debe realizarse junto con el contraste y la distribución de intensidades.

2. Contraste

El contraste global se aproxima mediante la desviación estándar de la luminancia:

$$C = \sigma_L = \sqrt{\frac{1}{N} \sum_{p=1}^N (L_p - \bar{L})^2}$$

Un contraste reducido indica poca variación entre regiones claras y oscuras.

Un contraste elevado indica una distribución más amplia de intensidades.

En imágenes de hojas, el contraste puede influir en la visibilidad de:

- lesiones;
 - bordes;
 - nervaduras;
 - manchas;
 - diferencias de textura.
-

3. Color medio

La distribución global de color se resume mediante:

$$\bar{R} = \frac{1}{N} \sum_{p=1}^N R_p$$

$$\bar{G} = \frac{1}{N} \sum_{p=1}^N G_p$$

$$\bar{B} = \frac{1}{N} \sum_{p=1}^N B_p$$

Estos valores permiten identificar diferencias generales de tonalidad entre imágenes.

En enfermedades foliares, cambios en:

verde → amarillo → marrón

pueden constituir información discriminativa relevante.

No obstante, los promedios RGB no describen por sí solos la distribución espacial de las lesiones.

4. Saturación

La saturación describe aproximadamente la intensidad del color.

Se calculará a partir de la representación HSV:

$$\bar{S} = \frac{1}{N} \sum_{p=1}^N S_p$$

Una saturación baja corresponde a colores más cercanos al gris, mientras que una saturación mayor indica colores más intensos.

Diferencias de saturación pueden estar relacionadas con:

- amarillamiento;
 - pérdida de pigmentación;
 - zonas necróticas;
 - variaciones naturales de adquisición.
-

5. Textura

Para obtener una aproximación interpretable de la textura se utilizará la magnitud del gradiente de la luminancia:

$$G(x, y) = \sqrt{\left(\frac{\partial L}{\partial x}\right)^2 + \left(\frac{\partial L}{\partial y}\right)^2}$$

y su valor promedio:

$$T = \frac{1}{N} \sum_{p=1}^N G_p$$

Un valor mayor de T indica mayor cantidad de cambios locales de intensidad.

Estos cambios pueden corresponder a:

- bordes;
- manchas;
- lesiones;
- nervaduras;
- patrones de textura.

La medida es exploratoria y no representa directamente las características internas aprendidas por la CNN.

6. Nitidez como componente de calidad

La nitidez se aproximará mediante la variabilidad del Laplaciano de la luminancia:

$$Q_{sharp} = Var(\nabla^2 L)$$

Una imagen con poco detalle y bordes difusos suele producir un valor pequeño, mientras que una imagen con transiciones espaciales más definidas tiende a generar valores mayores.

Esta medida debe interpretarse como un **indicador de nitidez**, no como una medida absoluta de calidad de imagen.

7. Rango dinámico

También se calculará:

$$D = P_{95}(L) - P_5(L)$$

donde P_5 y P_{95} representan los percentiles 5 y 95 de la luminancia.

Un rango dinámico reducido puede indicar poca separación entre zonas claras y oscuras.

8. Posible subexposición y sobreexposición

Se calculará la proporción de píxeles extremadamente oscuros:

$$P_{dark} = \frac{\#\{L < 0.05\}}{N} \times 100$$

y de píxeles extremadamente claros:

$$P_{bright} = \frac{\#\{L > 0.95\}}{N} \times 100.$$

Estos valores ayudan a identificar posibles condiciones extremas de exposición.

Hipótesis de análisis

Si las imágenes mal clasificadas presentan sistemáticamente:

$$\bar{L}_{error} \neq \bar{L}_{correcto},$$

$$T_{error} \neq T_{correcto},$$

o:

$$Q_{sharp,error} < Q_{sharp,correcto},$$

podría existir evidencia descriptiva de que ciertas propiedades visuales contribuyen a aumentar la dificultad de clasificación.

Sin embargo:

$$\text{asociación} \neq \text{causalidad}$$

por lo que estas medidas deben interpretarse conjuntamente con las imágenes, las clases reales y las clases predichas.

Relación con MLP y CNN

La comparación también permitirá identificar situaciones en las que:

$$\text{MLP incorrecta} \wedge \text{CNN correcta}$$

Este tipo de observación resulta especialmente relevante porque ambas arquitecturas reciben la misma información visual, pero la CNN conserva y explota explícitamente las relaciones espaciales.

También se analizarán observaciones donde:

$$\text{MLP incorrecta} \wedge \text{CNN incorrecta}$$

ya que pueden corresponder a casos visualmente difíciles incluso para una arquitectura convolucional.

El objetivo final no es determinar si una imagen es simplemente “buena” o “mala”, sino comprender si iluminación, textura, color o nitidez ayudan a explicar determinados errores de clasificación.

16.3 ¿El error es razonable o revela una debilidad estructural?

No todos los errores poseen la misma interpretación. Se distinguen errores visualmente plausibles de patrones que pueden revelar limitaciones arquitectónicas.

Para cada observación puede definirse:

$$E_n^{(m)} = \mathbb{1} \left(\hat{y}_n^{(m)} \neq y_n \right),$$

donde m identifica el modelo.

La ejecución actual produjo:

Indicador	MLP	CNN A
Accuracy	46.07 %	95.81 %
Aciertos	1.254	2.608
Errores	1.468	114

De los 1 468 errores de la MLP:

$$1\,367$$

son imágenes que CNN A clasifica correctamente. Esto corresponde a:

$$\frac{1367}{1468} \times 100 \approx \boxed{93.12\%}.$$

Además:

- ambos modelos fallan en 101 imágenes;
- MLP acierta y CNN falla en 13 imágenes;
- CNN A comete 114 errores en total.

El diagnóstico automático clasifica la mayoría de los errores MLP como **posible debilidad estructural**, porque CNN resuelve la misma observación. Esto es consistente con la limitación de la representación aplanada.

Para la CNN, los errores son mucho menos frecuentes y una proporción importante puede relacionarse con ambigüedad visual entre clases. Aun así, errores de alta confianza requieren inspección específica.

Por tanto:

La evidencia es compatible con una limitación estructural de la MLP bajo la representación utilizada, pero no constituye una demostración causal de que Flatten explique cada error individual.

16.4 Evaluación multidimensional: más allá de la Accuracy

La Accuracy no debe analizarse de manera aislada porque las diez clases poseen frecuencias diferentes. La razón entre la clase mayoritaria y la minoritaria es:

$$\frac{5357}{373} \approx 14.36.$$

Por ello se consideran conjuntamente:

- Accuracy;
- Balanced Accuracy;
- Precision Macro;

- Recall Macro;
- F1 Macro;
- F1 Weighted;
- tasa de error;
- métricas por clase;
- matriz de confusión;
- confianza predictiva;
- inspección visual.

Resultados actuales de Test

Métrica	MLP	CNN A
Accuracy	46.07 %	95.81 %
Balanced Accuracy	27.90 %	94.11 %
Precision Macro	28.08 %	95.51 %
Recall Macro	27.90 %	94.11 %
F1 Macro	20.20 %	94.41 %
F1 Weighted	36.98 %	95.69 %
Error Rate	53.93 %	4.19 %

La tasa de error se define como:

$$ErrorRate = 1 - Accuracy.$$

Por tanto:

$$ErrorRate_{MLP} = 1 - 0.4607 = 0.5393,$$

mientras:

$$ErrorRate_{CNN} = 1 - 0.9581 = 0.0419.$$

La reducción relativa de error es:

$$RRE = \frac{0.5393 - 0.0419}{0.5393} \approx \boxed{92.23\%}.$$

La clase con menor F1 en la CNN es Tomato__Early_blight, con:

$$Precision = 0.9802, \quad Recall = 0.6600, \quad F1 = 0.7888.$$

En la MLP existen varias clases con F1 igual a cero, lo que muestra que la Accuracy global oculta una capacidad muy desigual entre categorías.

Conclusión

La CNN supera a la MLP no solo en Accuracy, sino también en métricas que asignan igual importancia a cada clase. La ventaja observada es, por tanto, global y balanceada.

Fase 17 - Comparación MLP vs CNN

La comparación constituye el núcleo experimental del proyecto. Debe considerar rendimiento, eficiencia y explicación arquitectónica.

Fase 17 — Comparación MLP vs CNN

17.1 Determinación del modelo con mejor desempeño

El objetivo de esta fase es determinar cuál de las dos arquitecturas presenta el mejor desempeño final en el conjunto **Test**:

- **MLP**: recibe cada imagen como un vector de 49.152 valores después de aplicar Flatten.
- **CNN**: conserva la estructura tridimensional de la imagen y aprende patrones locales mediante filtros convolucionales.

La comparación no debe limitarse a observar cuál modelo posee la mayor **accuracy**. Debido al desbalance existente entre las diez clases, se analizarán simultáneamente:

1. Accuracy.
2. Balanced Accuracy.
3. Precision Macro.
4. Recall Macro.
5. F1 Macro.
6. F1 Weighted.
7. Tasa de error.
8. Número de parámetros.
9. Tiempo de entrenamiento.
10. Mejor época.
11. Desempeño por clase.
12. Resultados de la matriz de confusión.

Accuracy

La accuracy representa la proporción total de predicciones correctas:

$$\text{Accuracy} = \frac{N_{\text{aciertos}}}{N_{\text{Test}}}.$$

Los resultados ya obtenidos indican:

$$\text{Accuracy}_{\text{MLP}} = \frac{831}{2722} \approx 0,3053,$$

mientras que:

$$\text{Accuracy}_{\text{CNN}} = \frac{2607}{2722} \approx 0,9578.$$

La diferencia absoluta de accuracy es:

$$\Delta_{\text{Accuracy}} = \text{Accuracy}_{\text{CNN}} - \text{Accuracy}_{\text{MLP}}.$$

Por lo tanto:

$$\Delta_{\text{Accuracy}} = 0,9578 - 0,3053 = 0,6525.$$

La CNN supera a la MLP en aproximadamente:

65,25 puntos porcentuales.

Balanced Accuracy

La balanced accuracy corresponde al promedio del recall obtenido en las K clases:

$$\text{BalancedAccuracy} = \frac{1}{K} \sum_{k=1}^K \text{Recall}_k.$$

Esta métrica asigna el mismo peso a cada clase, independientemente de su cantidad de imágenes. Por ello, resulta especialmente relevante en este proyecto, donde la razón de desbalance alcanza aproximadamente:

$$IR \approx 14,36.$$

Si la CNN supera a la MLP tanto en accuracy como en balanced accuracy, la mejora no puede atribuirse solamente a un mejor reconocimiento de las clases mayoritarias.

Precision Macro

La precision de una clase k se define como:

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}.$$

La precision macro corresponde al promedio no ponderado entre las clases:

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Precision}_k.$$

Esta métrica permite evaluar si las clases predichas por el modelo son confiables sin favorecer a las categorías con mayor soporte.

Recall Macro

El recall de una clase k se define como:

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k}.$$

Su promedio macro es:

$$\text{Recall}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Recall}_k.$$

En una clasificación multiclase de etiqueta única, el recall macro es equivalente a la balanced accuracy cuando se consideran las mismas clases.

F1 Macro

El F1-score de cada clase combina precision y recall:

$$F1_k = 2 \frac{\text{Precision}_k \text{ Recall}_k}{\text{Precision}_k + \text{Recall}_k}.$$

El promedio macro se calcula como:

$$F1_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K F1_k.$$

Esta será una de las métricas principales para determinar el mejor modelo porque:

- asigna el mismo peso a todas las clases;
- integra precision y recall;
- penaliza los resultados desequilibrados;
- es apropiada ante la distribución desigual del dataset.

Tasa de error

La tasa de error es el complemento de la accuracy:

$$\text{ErrorRate} = 1 - \text{Accuracy}.$$

Para la MLP:

$$\text{ErrorRate}_{\text{MLP}} = 1 - 0,4607 = 0,5393.$$

Para la CNN:

$$\text{ErrorRate}_{\text{CNN}} = 1 - 0,9581 = 0,0419.$$

La reducción relativa del error obtenida por la CNN se calcula como:

$$\text{RRE} = \frac{\text{ErrorRate}_{\text{MLP}} - \text{ErrorRate}_{\text{CNN}}}{\text{ErrorRate}_{\text{MLP}}}.$$

Sustituyendo los resultados:

$$\text{RRE} = \frac{0,5393 - 0,0419}{0,5393} \approx 0,939.$$

Esto significa que la CNN reduce aproximadamente un 92,23% de la tasa de error observada en la MLP.

Criterio de decisión

El modelo con mejor desempeño predictivo será aquel que presente simultáneamente:

$$\text{mayor } F1_{\text{macro}},$$

$$\text{mayor Balanced Accuracy},$$

mayor Accuracy,

y:

menor Error Rate.

El número de parámetros y el tiempo de entrenamiento se interpretarán como criterios de **eficiencia computacional**, pero no reemplazarán las métricas de generalización.

No se construirá un promedio arbitrario entre accuracy, tiempo y parámetros, porque estas variables poseen escalas y significados diferentes.

Hipótesis de comparación

Se establecen las siguientes hipótesis experimentales:

H_0 : la CNN no presenta una ventaja consistente sobre la MLP,

H_1 : la CNN presenta una ventaja consistente sobre la MLP.

La evidencia favorecerá a H_1 cuando la CNN supere a la MLP no solamente en accuracy, sino también en las métricas balanceadas y en el análisis por clase.

Resultado esperado

Considerando los resultados ya obtenidos, la CNN presenta una ventaja amplia:

- obtiene 2607 predicciones correctas frente a 831 de la MLP;
- comete 115 errores frente a 1891 de la MLP;
- resuelve correctamente 1786 imágenes que la MLP clasifica de manera incorrecta;
- corrige aproximadamente el 94,45% de los errores producidos por la MLP;
- conserva la estructura espacial de las imágenes;
- aprende patrones locales de color, bordes, textura y lesiones.

Por consiguiente, la CNN se perfila como el modelo con mejor capacidad de generalización. Esta conclusión deberá confirmarse automáticamente mediante la tabla consolidada de métricas.

Estado actualizado del experimento

La comparación final utiliza las predicciones ya congeladas de la Fase 14. CNN A supera a la MLP en los tres criterios principales:

$$\Delta Accuracy = +49.74 \text{ pp},$$

$$\Delta BalancedAccuracy = +66.21 \text{ pp},$$

$$\Delta F1_{macro} = +74.21 \text{ pp},$$

con una reducción relativa del error de:

$$\boxed{92.23\%}.$$

17.2 ¿Por qué la CNN funciona mejor?

La superioridad observada debe relacionarse con la arquitectura y no únicamente con la profundidad de la red.

Representación de entrada

La MLP transforma:

$$128 \times 128 \times 3 \longrightarrow 49\,152$$

valores mediante Flatten. Los valores de los píxeles se conservan, pero la vecindad bidimensional deja de estar representada explícitamente.

La CNN, en cambio, mantiene inicialmente el tensor:

$$X \in \mathbb{R}^{128 \times 128 \times 3}$$

y aplica filtros locales.

Localidad

Una convolución puede representarse como:

$$Z_{i,j,k} = \phi \left(\sum_{u,v,c} W_{u,v,c,k} X_{i+u,j+v,c} + b_k \right).$$

Cada filtro analiza regiones vecinas y puede responder a bordes, manchas, texturas y cambios de color.

Compartición de pesos

El mismo filtro se reutiliza en múltiples posiciones, por lo que una característica visual puede detectarse sin aprender pesos completamente independientes para cada ubicación.

Aprendizaje jerárquico

Las capas sucesivas permiten construir:

píxeles $\rightarrow$ bordes $\rightarrow$ texturas $\rightarrow$ manchas $\rightarrow$ patrones de lesión $\rightarrow$ clase.

Evidencia obtenida en Test

Indicador	MLP	CNN A
Predicciones correctas	1.254	2.608
Errores	1.468	114
Accuracy	46.07 %	95.81 %
Balanced Accuracy	27.90 %	94.11 %
F1 Macro	20.20 %	94.41 %

La CNN clasifica correctamente 1 367 imágenes que la MLP clasifica de forma incorrecta, equivalentes al:

$$\frac{1367}{1468} \times 100 \approx 93.12\%$$

de los errores de la MLP.

Además:

$$\Delta Accuracy = +49.74 \text{ pp},$$

$$\Delta BalancedAccuracy = +66.21 \text{ pp},$$

$$\Delta F1_{macro} = +74.21 \text{ pp}.$$

La CNN también supera a la MLP en Recall en las diez clases.

Interpretación

Los resultados son consistentes con la capacidad de las convoluciones para explotar localidad, pesos compartidos y jerarquía espacial.

No obstante, la comparación no demuestra que las convoluciones sean la única causa de toda la diferencia; también pueden influir regularización, hiperparámetros y distribución de parámetros.

17.3 Trade-offs de costo computacional, parámetros y generalización

Una comparación completa debe separar **rendimiento predictivo** y **eficiencia computacional**.

Parámetros

La MLP posee:

$$P_{MLP} = 6\,300\,490,$$

mientras CNN A posee:

$$P_{CNN} = 424\,042.$$

La razón es:

$$R_P = \frac{P_{MLP}}{P_{CNN}} \approx \boxed{14.86}.$$

Por tanto, la MLP utiliza aproximadamente 14.86 veces más parámetros.

La causa principal es la primera capa densa posterior a Flatten:

$$P_{Dense} = (49\,152 + 1) \times 128 = 6\,291\,584.$$

La CNN reduce drásticamente el número de conexiones independientes mediante filtros locales y compartición de pesos.

Generalización

Sobre Test:

$$\Delta Accuracy = +49.74 \text{ pp},$$

$$\Delta BalancedAccuracy = +66.21 \text{ pp},$$

$$\Delta F1_{macro} = +74.21 \text{ pp}.$$

La reducción relativa de la tasa de error alcanza:

$$\boxed{92.23\%}.$$

Tiempo de entrenamiento

El notebook actual no conserva un registro temporal comparable para MLP y CNN A bajo exactamente las mismas condiciones de medición. Por este motivo, **no se debe presentar una comparación numérica de tiempos como evidencia definitiva.**

El tiempo depende de:

- hardware asignado por Colab;
- operaciones convolucionales;
- Data Augmentation;
- tamaño de lote;
- número de épocas;
- carga de datos;
- estado de la sesión.

Por consiguiente:

Menos parámetros no implica necesariamente menor tiempo de entrenamiento.

Mejor época

La mejor época se define mediante:

$$e^* = \arg \min_e L_{\text{validation}}^{(e)}.$$

En la ejecución actual:

$$e_{\text{MLP}}^* = 25, \quad e_{\text{CNN A}}^* = 18.$$

Conclusión del trade-off

CNN A presenta simultáneamente:

- mejor rendimiento en Test;
- mejor rendimiento balanceado;
- menos parámetros;
- menor tasa de error.

La comparación temporal queda como una limitación porque no existe un registro directamente comparable para ambos modelos.

17.4 Relación entre los resultados y la hipótesis experimental inicial

La pregunta central del proyecto es:

¿La utilización de una CNN permite clasificar las enfermedades de las hojas con mayor capacidad de generalización que una MLP entrenada sobre una representación vectorizada de las mismas imágenes?

La hipótesis experimental puede expresarse como:

$$H_{\text{exp}} : \mathcal{G}_{\text{CNN}} > \mathcal{G}_{\text{MLP}},$$

y operacionalmente:

$$Accuracy_{\text{CNN}} > Accuracy_{\text{MLP}},$$

$$BalancedAccuracy_{\text{CNN}} > BalancedAccuracy_{\text{MLP}},$$

$$F1_{\text{macro,CNN}} > F1_{\text{macro,MLP}}.$$

Evidencia global

Métrica	MLP	CNN A	Diferencia
Accuracy	46.07 %	95.81 %	+49.74 pp
Balanced Accuracy	27.90 %	94.11 %	+66.21 pp
F1 Macro	20.20 %	94.41 %	+74.21 pp

Los modelos se evaluaron sobre las mismas $N = 2\,722$ imágenes.

Evidencia pareada

Los casos discordantes fueron:

$$n_{10} = 13$$

para imágenes correctas en MLP e incorrectas en CNN, y:

$$n_{01} = 1367$$

para imágenes incorrectas en MLP y correctas en CNN.

La diferencia pareada neta es:

$$D_{\text{pareada}} = \frac{1367 - 13}{2722} \approx \boxed{49.74 \text{ pp}}.$$

La prueba exacta de McNemar produjo un valor p numéricamente reportado como 0 por la implementación, es decir, extremadamente pequeño respecto del nivel usual de 0.05. Esto respalda que los patrones de error de ambos modelos son sistemáticamente distintos.

La prueba no demuestra que las convoluciones sean la única causa de la diferencia; para esa interpretación deben considerarse también arquitectura, regularización e hiperparámetros.

Reducción relativa del error

$$ErrorRate_{\text{MLP}} = 53.93\%,$$

$$ErrorRate_{\text{CNN}} = 4.19\%.$$

Por tanto:

$$RRE = \frac{0.5393 - 0.0419}{0.5393} \approx \boxed{92.23\%}.$$

Relación con la estructura espacial

La MLP utiliza:

$$X \in \mathbb{R}^{128 \times 128 \times 3} \rightarrow \text{vec}(X) \in \mathbb{R}^{49\,152},$$

mientras la CNN conserva estructura espacial y aprende filtros locales compartidos.

Los resultados son consistentes con la hipótesis de que dicha estructura aporta una ventaja observable para reconocer enfermedades foliares.

Conclusión

Los seis criterios operacionales evaluados en la Fase 17.4 fueron satisfechos y la evidencia respalda la hipótesis experimental inicial:

$$\mathcal{G}_{\text{CNN A}} > \mathcal{G}_{\text{MLP}}$$

dentro del Test interno de PlantVillage y bajo el protocolo utilizado.

Tabla comparativa final — MLP vs CNN

La CNN incluida corresponde a **CNN A**, seleccionada mediante Validation antes de abrir Test.

Métrica / criterio	MLP	CNN A
Accuracy	46.07 %	95.81 %
Balanced Accuracy	27.90 %	94.11 %
Precision Macro	28.08 %	95.51 %
Recall Macro	27.90 %	94.11 %
F1 Macro	20.20 %	94.41 %
Número de parámetros	6.300.490	424.042
Razón de parámetros MLP/CNN	14.86×	14.86×
Mejor época según val_loss	25	18
Tiempo comparable de entrenamiento	No disponible	No disponible

El tiempo de entrenamiento no se utiliza como criterio cuantitativo final porque el notebook no conserva una medición directamente comparable entre la MLP y CNN A.

Fase 18 - Interpretación arquitectónica

Si la CNN obtiene un mejor resultado, la conclusión debe vincular esa mejora con su capacidad de explotar localidad, pesos compartidos y jerarquía espacial, no simplemente con el hecho de ser una red “más profunda”.

Fase 18 — Interpretación arquitectónica

18.1 Resultado observado

CNN A supera ampliamente a la MLP:

Métrica	MLP	CNN A	Diferencia CNN-MLP
Accuracy	46.07 %	95.81 %	+49.74 pp
Balanced Accuracy	27.90 %	94.11 %	+66.21 pp
Precision Macro	28.08 %	95.51 %	+67.44 pp
Recall Macro	27.90 %	94.11 %	+66.21 pp
F1 Macro	20.20 %	94.41 %	+74.21 pp

Además:

$$P_{\text{MLP}} = 6\,300\,490, \quad P_{\text{CNN}} = 424\,042,$$

por lo que:

$$\frac{P_{\text{MLP}}}{P_{\text{CNN}}} \approx 14.86.$$

La MLP posee aproximadamente 14.86 veces más parámetros y, aun así, presenta un rendimiento muy inferior.

18.2 La profundidad no explica por sí sola la diferencia

No es correcto concluir que la CNN funciona mejor simplemente por ser “más profunda”:

mayor profundidad $\nRightarrow$ mejor desempeño.

La diferencia debe relacionarse con el tipo de operaciones que cada modelo aplica sobre las imágenes.

18.3 MLP: representación aplanada

La MLP transforma:

$$\mathbb{R}^{128 \times 128 \times 3} \xrightarrow{\text{Flatten}} \mathbb{R}^{49\,152}.$$

Flatten conserva los valores, pero la vecindad espacial deja de estar representada explícitamente. La primera capa densa contiene:

$$(49\,152 + 1) \times 128 = 6\,291\,584$$

parámetros, aproximadamente el 99.86% de todos los parámetros de la MLP.

18.4 CNN: localidad

Una convolución utiliza filtros locales, de modo que una neurona analiza una región pequeña de la imagen. Esto permite detectar:

- bordes;
- texturas;
- manchas;
- cambios cromáticos;
- patrones de lesión.

18.5 Pesos compartidos

El mismo filtro se desplaza por distintas posiciones. Esta compartición reduce el número de parámetros independientes y permite buscar un mismo patrón en múltiples regiones.

18.6 Jerarquía espacial

Las capas sucesivas pueden construir:

píxeles $\rightarrow$ bordes $\rightarrow$ texturas $\rightarrow$ manchas $\rightarrow$ patrones de lesión $\rightarrow$ clase.

18.7 Evidencia empírica

La CNN resuelve 1 367 imágenes que la MLP clasifica incorrectamente y reduce relativamente la tasa de error en:

$$\boxed{92.23\%}.$$

También obtiene mayor Recall en las diez clases.

Conclusión

La evidencia es consistente con que:

$$\boxed{\text{localidad} + \text{pesos compartidos} + \text{jerarquía espacial}}$$

constituyen un sesgo inductivo más apropiado para imágenes foliares que la representación completamente aplanada de la MLP.

La asociación es fuerte, pero no debe interpretarse como una demostración causal de que toda la diferencia provenga exclusivamente de las convoluciones.

Fase 19 - Limitaciones

Reconocer explícitamente aquello que el experimento no permite demostrar aumenta la validez de las conclusiones.

Fase 19 — Limitaciones del experimento

Reconocer explícitamente lo que el experimento **no permite demostrar** aumenta la validez de las conclusiones.

Dentro del Test interno de PlantVillage se obtuvo:

Métrica	MLP	CNN A
Accuracy	46.07 %	95.81 %
Balanced Accuracy	27.90 %	94.11 %
F1 Macro	20.20 %	94.41 %

Estos resultados respaldan la superioridad de CNN A **dentro de las condiciones evaluadas**, pero no garantizan el mismo desempeño en un entorno agrícola real.

19.1 Dataset potencialmente capturado en condiciones controladas

PlantVillage puede contener imágenes con fondos relativamente homogéneos, hojas centradas y buena visibilidad. Por ello:

$$(X, Y) \sim P_{PV}(X, Y)$$

no implica necesariamente:

$$P_{PV}(X, Y) = P_{campo}(X, Y).$$

El modelo podría aprender regularidades de captura además de síntomas biológicos.

19.2 Diferencia respecto de imágenes de terreno

Fotografías reales pueden contener:

- fondos complejos;
- sombras;
- reflejos;
- oclusiones;
- varias hojas;
- distintas cámaras;
- síntomas parciales;
- diferentes escalas y ángulos.

Por ello existe riesgo de **distribution shift**.

La Accuracy interna:

$$Accuracy_{CNA} = 95.81\%$$

no debe interpretarse como una estimación garantizada del rendimiento en terreno.

19.3 Desbalance de clases

El dataset limpio contiene:

$$N = 18\,146, \quad K = 10.$$

La clase mayoritaria posee:

$$N_{\max} = 5\,357$$

y la minoritaria:

$$N_{\min} = 373.$$

Por tanto:

$$IR = \frac{5357}{373} \approx 14.36.$$

Este desbalance justifica utilizar Balanced Accuracy y métricas macro.

19.4 Resolución reducida

La resolución original predominante es:

$$256 \times 256,$$

mientras el experimento utiliza:

$$128 \times 128.$$

En términos de píxeles espaciales:

$$\frac{128^2}{256^2} = 0.25.$$

Por tanto, se conserva el 25% de los píxeles espaciales y existe una reducción del 75%. Esto disminuye el costo computacional, pero puede eliminar detalles finos.

19.5 Exploración limitada de hiperparámetros

La búsqueda fue deliberadamente acotada para mantener el proyecto reproducible y compatible con el tiempo disponible. No se realizó una búsqueda exhaustiva de:

- learning rate;
- número de filtros;
- profundidad;
- Dropout;
- batch size;
- intensidad de augmentation.

No puede afirmarse que la configuración obtenida sea un óptimo global.

19.6 Ausencia de validación externa

Train, Validation y Test provienen de la misma fuente. Por ello Test mide **generalización interna**, no generalización entre dominios.

Se requiere un conjunto independiente de imágenes de campo para evaluar esa capacidad.

19.7 Recursos y tiempo limitados de Google Colab

El hardware, memoria y acelerador disponibles en Colab pueden variar entre sesiones. Estas restricciones limitaron:

- resolución;
- cantidad de experimentos;
- búsqueda de hiperparámetros;
- evaluación con múltiples semillas.

Alcance válido de la conclusión

La formulación respaldada por la evidencia es:

CNN A generaliza mejor que la MLP evaluada dentro del Test interno de PlantVillage.

No es válido afirmar todavía:

CNN A mantendrá automáticamente 95.81 % de Accuracy en condiciones agrícolas reales.

Fase 20 - Posibles mejoras

Proponer extensiones futuras sin convertirlas en requisitos del experimento actual.

Fase 20 — Posibles mejoras y extensiones futuras

Las mejoras propuestas son **trabajo futuro** y no requisitos pendientes del experimento actual.

El punto de referencia correcto de la ejecución actual es:

Indicador	MLP	CNN A seleccionada
Accuracy Test	46.07 %	95.81 %
Balanced Accuracy Test	27.90 %	94.11 %
Precision Macro Test	28.08 %	95.51 %
Recall Macro Test	27.90 %	94.11 %
F1 Macro Test	20.20 %	94.41 %
Parámetros	6.300.490	424.042

Las extensiones futuras deben compararse contra CNN A sin reutilizar Test para seleccionar hiperparámetros.

20.1 Transfer Learning

El Transfer Learning reutiliza representaciones visuales aprendidas previamente:

$$f(X) = g_{\phi}(h_{\theta}(X)).$$

Una primera etapa puede mantener congelado el extractor h_{θ} , y una segunda puede realizar fine-tuning con un learning rate menor.

Hipótesis futura:

$$H_{TL} : F1_{\text{macro, TL}} > 94.41\%.$$

La comparación debe realizarse primero en Validation y posteriormente confirmarse una sola vez sobre un Test externo o un nuevo protocolo de prueba.

20.2 EfficientNet

EfficientNet permite explorar un escalamiento conjunto de profundidad, ancho y resolución. EfficientNetB0 constituye una alternativa razonable para Colab por su relación entre capacidad y costo.

Una configuración futura puede ser:

Imagen $\rightarrow$ EfficientNetB0 $\rightarrow$ GlobalAveragePooling $\rightarrow$ Dropout $\rightarrow$ Softmax.

20.3 ResNet

Las conexiones residuales se expresan como:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \mathbf{W}) + \mathbf{x}.$$

ResNet50 preentrenada permitiría estudiar si representaciones más profundas y transferidas mejoran las métricas balanceadas.

20.4 Data Augmentation más amplio

Se pueden evaluar transformaciones moderadas adicionales:

- traslación;
- contraste;
- brillo;
- recorte;
- variaciones cromáticas;
- ruido leve.

Cada transformación debe conservar el significado de la clase.

20.5 Validación externa con imágenes de campo

Esta es la extensión de mayor prioridad porque permitiría evaluar:

$$P_{\text{campo}}(X, Y) \neq P_{\text{PV}}(X, Y).$$

La comparación externa debe incluir Accuracy, Balanced Accuracy, F1 Macro y análisis de errores.

20.6 Grad-CAM

Grad-CAM permitiría inspeccionar qué regiones contribuyen a una predicción y evaluar si la red atiende a lesiones y estructuras foliares relevantes.

20.7 Optimización sistemática de hiperparámetros

En lugar de una grilla exhaustiva se recomienda utilizar un presupuesto controlado con:

- búsqueda aleatoria;
- Hyperband;
- optimización bayesiana.

La selección debe realizarse únicamente con Train y Validation.

20.8 Evaluación con múltiples semillas

Repetir el experimento con varias semillas permitiría reportar:

$$\overline{F1}_{macro} \pm s_{F1},$$

y estimar la variabilidad debida a inicialización y orden de los datos.

Prioridades futuras

1. validación externa;
2. Grad-CAM;
3. Transfer Learning;
4. EfficientNet/ResNet;
5. Data Augmentation ampliado;
6. optimización de hiperparámetros;
7. múltiples semillas.

Estas propuestas amplían el trabajo, pero el experimento actual permanece cerrado.